Swift, iOS, and SwiftUI Interview FAQ: From Fundamentals to Confident Answers

Audience: iOS engineers with Swift experience preparing for iOS and SwiftUI interviews.

Assumption: Modern Swift and iOS 17+ unless noted. SwiftUI's Observation framework requires iOS 17, macOS 14, watchOS 10, tvOS 17, or newer.

Table of Contents

How To Use This Guide

Interviewers usually test three layers:

1. Can you state the practical answer clearly?
2. Do you understand why the abstraction exists?
3. Can you explain the failure modes in production code?

For each topic, practice the short answer first, then use the deep explanation to handle follow-up questions. Strong answers usually connect language rules to app behavior: performance, memory, correctness, user experience, and testability.

Swift Fundamentals

[Beginner] What is the difference between a struct and a class in Swift?

Interview answer: A `struct` is a value type: assigning or passing it creates an independent value. A `class` is a reference type: assigning or passing it shares the same instance. Classes support inheritance, identity checks with ===, deinitializers, and ARC-based reference counting; structs do not.

Deep explanation: Structs and classes can both store properties, define methods, conform to protocols, define initializers, and be extended. The major difference is semantics. A struct models a value, like `Date`, `URL`, `CGRect`, or a domain model. A class models identity and shared mutable lifetime, like a view controller, service object, cache, or object graph node.

Why it matters: This choice determines how state moves through your app. Value types make local reasoning easier because mutation happens to a specific copy. Reference types are necessary when identity, sharing, inheritance, or Objective-C/UIKit interoperability matters.

How it works: Struct values are copied semantically when assigned or passed. The compiler can optimize many copies away. Classes live as heap-allocated instances managed by ARC; variables hold references to those instances. Multiple references can point to the same object.

Use cases: Use structs for models, configuration, view state, request/response DTOs, and small data containers. Use classes for UIKit objects, reference-owned services, delegates, caches, shared mutable coordinators, and objects that need `deinit`.

Common mistakes: Using a class for every model because other languages do. Assuming `let` makes a class immutable. Forgetting that two references can mutate the same object from different places.

Best practices: Default to `struct`; use `final class` when reference semantics are intentional. Keep mutable class state protected by actor isolation, main-actor isolation, or explicit synchronization.

Red flag answers: "Structs are always on the stack and classes are always on the heap." That is an oversimplification. The compiler decides storage details; the semantic distinction is value vs reference.

Code example:

```
struct Profile {
    var name: String
}

final class Session {
    var token: String

    init(token: String) {
        self.token = token
    }
}

var a = Profile(name: "Ari")
var b = a
b.name = "Bo"
print(a.name) // Ari

let s1 = Session(token: "old")
let s2 = s1
s2.token = "new"
print(s1.token) // new
```

Possible follow-up questions: When would you still choose a class? How do copy-on-write collections fit this model? What does `let` mean for a class reference?

[Beginner] What are value types and reference types?

Interview answer: Value types carry their own independent value. Reference types carry a reference to shared identity. In Swift, structs, enums, and tuples are value types; classes, actors, and closures are reference-like.

Deep explanation: Value semantics means code can treat a value as independent after assignment. Reference semantics means two variables can be aliases for the same underlying instance. Both are useful; the mistake is using one accidentally.

Why it matters: Most iOS bugs involving surprising state changes, stale UI, or data races come from unclear ownership and aliasing. Value types reduce accidental aliasing.

How it works: The language promises value behavior even when the implementation shares storage internally. `Array`, `Dictionary`, `Set`, and `String` use copy-on-write so copies are cheap until mutation.

Use cases: Value types for app state snapshots, SwiftUI view input, diffable data source items, and request models. Reference types for object identity, delegate lifetimes, and shared services.

Common mistakes: Believing value type means "deep copy of everything." A struct can contain a class reference, and then the struct copy still points to the same referenced object.

Best practices: Make value-type properties value types where possible. If a struct wraps a class, document that reference behavior or implement copy-on-write.

Red flag answers: "Reference types are bad." They are not; they are just higher-risk when shared mutation is uncontrolled.

Code example:

```
final class Box {
    var count = 0
}

struct Wrapper {
    var box: Box
}

var first = Wrapper(box: Box())
var second = first
second.box.count = 10

print(first.box.count) // 10, because the wrapped class is shared
```

Possible follow-up questions: How do you design a value type that owns reference storage? How does this affect SwiftUI state?

[Intermediate] Why does Swift favor value types?

Interview answer: Swift favors value types because they make code easier to reason about, reduce accidental shared mutation, compose well with generics and protocols, and support efficient copy-on-write implementations.

Deep explanation: Value types fit Swift's goals: safety, performance, and clarity. If assigning a value produces independent behavior, fewer parts of the app can unexpectedly mutate it. This helps architecture, testing, concurrency, and SwiftUI rendering.

Why it matters: SwiftUI is built around value descriptions of UI. App state is often represented as values and changes are propagated intentionally. Concurrency also benefits because immutable or value-semantic data is easier to move between tasks.

How it works: The compiler can place, copy, inline, and optimize values aggressively. Standard library collections avoid eager deep copies by sharing buffers until mutation. At the language level, behavior still appears as if a copy was made.

Use cases: Domain models, reducers, form state, immutable snapshots, configuration objects, and SwiftUI views.

Common mistakes: Turning every object into a struct even when identity matters. Large structs with many mutable reference properties can be harder to reason about than a small explicit class.

Best practices: Use value types for data and reference types for identity/lifetime. Keep values small enough conceptually, not necessarily physically. Do not optimize away structs prematurely.

Red flag answers: "Structs are always faster." Performance depends on size, copying behavior, ARC traffic, cache locality, and mutation pattern.

Code example:

```
struct CheckoutState: Equatable {
    var items: [CartItem]
    var selectedPaymentID: Payment.ID?
    var isSubmitting = false
}

// A reducer-style update returns a new value snapshot.
func selectingPayment(_ id: Payment.ID, in state: CheckoutState) -> CheckoutState {
    var copy = state
    copy.selectedPaymentID = id
    return copy
}
```

Possible follow-up questions: How does value semantics help thread safety? When can a value type still be unsafe?

[Intermediate] What is copy-on-write?

Interview answer: Copy-on-write is an optimization where values share storage until one copy is mutated. Swift collections like `Array`, `Dictionary`, `Set`, and `String` behave like values but avoid copying their full storage on every assignment.

Deep explanation: Without copy-on-write, copying large arrays on every assignment would be expensive. With COW, two arrays can point to the same buffer. When one is mutated, Swift checks whether the buffer is uniquely referenced; if not, it copies before mutation so value semantics are preserved.

Why it matters: It explains why value types can be both safe and performant. It also explains performance cliffs when repeatedly mutating shared copies.

How it works: The standard library stores collection elements in reference-backed buffers. Mutation checks uniqueness, conceptually using `isKnownUniquelyReferenced` for custom COW types. If storage is unique, mutate in place. If shared, allocate and copy first.

Use cases: Large collections, strings, images or buffers wrapped in value types, immutable snapshots with occasional mutation.

Common mistakes: Assuming COW makes all copies free forever. Mutating after copying can trigger real allocation and copying. Also, COW does not automatically make shared mutable reference contents safe.

Best practices: Avoid unnecessary intermediate copies in hot paths. For custom COW, keep storage private and ensure all mutations go through uniqueness checks.

Red flag answers: "Copy-on-write means Swift never copies arrays." It copies when needed to preserve independent value behavior.

Code example:

```
final class Storage {
    var values: [Int]

    init(_ values: [Int]) {
        self.values = values
    }

    init(copying storage: Storage) {
        values = storage.values
    }
}

struct CowList {
    private var storage: Storage
```

```
    init(_ values: [Int]) {
        storage = Storage(values)
    }

    var values: [Int] { storage.values }

    mutating func append(_ value: Int) {
        if !isKnownUniquelyReferenced(&storage) {
            storage = Storage(copying: storage)
        }
        storage.values.append(value)
    }
}
```

Possible follow-up questions: How would you implement COW safely? What happens if the storage leaks outside the struct?

[Beginner] What are optionals and why does Swift use them?

Interview answer: An optional represents either a wrapped value or `nil`. Swift uses optionals to make absence explicit in the type system so you handle missing values safely instead of crashing through null references.

Deep explanation: `String` and `String?` are different types. A `String?` must be unwrapped before use because the compiler does not know whether it contains a value. Optional unwrapping is a safety boundary: `if let`, `guard let`, optional chaining, nil coalescing, and pattern matching all express how absence should be handled.

Why it matters: Many iOS APIs naturally have absence: missing network fields, optional delegates, unavailable UI state, not-yet-loaded data, nullable Objective-C imports, and failed lookups.

How it works: `Optional<Wrapped>` is an enum with `.some(Wrapped)` and `.none`. The `?` syntax is sugar for `Optional`.

Use cases: Parsing JSON, looking up dictionary values, weak references, optional callbacks, navigation selection, and UI text fields that may not have valid input yet.

Common mistakes: Force-unwrapping because a value "should exist." Overusing optionals where a non-optional default or explicit state enum would be clearer.

Best practices: Use `guard let` for required early exits. Use optional chaining for genuinely optional work. Model meaningful states with enums instead of multiple loosely related optionals.

Red flag answers: "Optionals are just null." They are typed containers that force handling at compile time.

Code example:

```
func displayName(from profile: Profile?) -> String {
    guard let profile else {
        return "Guest"
    }

    return profile.name.trimmingCharacters(in: .whitespacesAndNewlines)
}
```

Possible follow-up questions: What is implicitly unwrapped optional? When is `!` acceptable? How do optionals map from Objective-C nullability?

[Intermediate] What is protocol-oriented programming?

Interview answer: Protocol-oriented programming designs behavior around protocols and protocol extensions instead of only class inheritance. It lets unrelated types share capabilities while preserving value semantics and static dispatch where possible.

Deep explanation: A protocol defines requirements. Types conform by implementing those requirements. Protocol extensions can provide default behavior, and generic constraints can activate behavior only for matching types. This gives reuse without forcing a common superclass.

Why it matters: iOS apps frequently need testable boundaries: repositories, analytics trackers, clocks, network clients, coordinators, and formatters. Protocols let production and test implementations share contracts without inheritance.

How it works: When a generic function is constrained to a protocol, the compiler often specializes it for concrete types. When a value is stored as `any Protocol`, Swift uses an existential container and dynamic dispatch through witness tables.

Use cases: Dependency injection, mocking, reusable UI configuration, collection algorithms, domain abstractions, and framework extension points.

Common mistakes: Creating a protocol for every class even when there is one implementation and no boundary. Putting too many unrelated requirements into one protocol.

Best practices: Prefer small role-based protocols. Define protocols at the consumer boundary when testability or substitution is needed. Avoid protocol requirements that expose implementation details.

Red flag answers: "POP means never use classes." Protocols and classes solve different problems.

Code example:

```
protocol UserRepository {
    func user(id: User.ID) async throws -> User
}

@MainActor
final class ProfileViewModel: ObservableObject {
    @Published private(set) var name = ""
    private let repository: UserRepository

    init(repository: UserRepository) {
        self.repository = repository
    }

    func load(id: User.ID) async {
        do {
            name = try await repository.user(id: id).name
        } catch {
            name = "Unavailable"
        }
    }
}
```

Possible follow-up questions: What are protocol extensions? How do existentials differ from generics? What is a witness table?

[Intermediate] What are generics and why are they useful?

Interview answer: Generics let you write reusable, type-safe code that works with many concrete types without losing static type information.

Deep explanation: Instead of writing the same container, algorithm, or wrapper for each type, you write it once with a type parameter. The compiler checks constraints at compile time and can specialize generic code for performance.

Why it matters: Swift's standard library is generic: `Array<Element>`, `Dictionary<Key, Value>`, `Result<Success, Failure>`, `Optional<Wrapped>`. In app code, generics help build reusable networking, storage, and view components without casting.

How it works: Generic placeholders stand in for concrete types. Constraints such as `T: Decodable` or `T: Identifiable` tell the compiler which operations are allowed.

Use cases: Network clients decoding arbitrary DTOs, reusable SwiftUI rows, typed caches, result wrappers, and collection helpers.

Common mistakes: Using generics where a concrete type is simpler. Exposing too many generic parameters in public APIs. Replacing clear domain types with abstract generic machinery.

Best practices: Use generics when the behavior is truly identical across types and type safety matters. Add constraints only as needed.

Red flag answers: "Generics are just Any." Any erases type information; generics preserve it.

Code example:

```
struct APIClient {
    func get<Response: Decodable>(
        _ type: Response.Type,
        from url: URL
    ) async throws -> Response {
        let (data, response) = try await URLSession.shared.data(from: url)
        guard (response as? HTTPURLResponse)?.statusCode == 200 else {
            throw URLError(.badServerResponse)
        }
        return try JSONDecoder().decode(Response.self, from: data)
    }
}
```

Possible follow-up questions: What is generic specialization? How do generic constraints differ from protocol existentials?

[Senior] What are associated types in protocols?

Interview answer: An associated type is a placeholder type declared inside a protocol. The conforming type chooses the concrete associated type, letting the protocol express relationships between inputs, outputs, and stored values.

Deep explanation: Some protocols cannot be fully described without saying "this type has some related type." For example, `Sequence` has an `Element`. A repository might have an `Entity`. Associated types keep these relationships type-safe.

Why it matters: Associated types are common in SwiftUI, Combine, collections, repositories, and reusable architecture. They explain why some protocols cannot be used directly as simple existential types without `any` limitations or type erasure.

How it works: The conforming type either explicitly defines a `typealias` or the compiler infers the associated type from method signatures/properties. Generic constraints can refer to that associated type.

Use cases: Reusable data sources, parsers, repositories, sequences, reducers, and SwiftUI protocols.

Common mistakes: Trying to store `let repository: Repository` when `Repository` has associated types or `Self` requirements and expecting it to work like a Java interface. Use generics or type erasure instead.

Best practices: Use associated types when the relationship is part of the protocol's core meaning. If callers need heterogeneous storage, design a type-erased wrapper.

Red flag answers: "Associated types are generics for protocols" is a start, but incomplete. The key is that conforming types bind related types as part of conformance.

Code example:

```
protocol Repository {
    associatedtype Entity: Identifiable
    func fetch(id: Entity.ID) async throws -> Entity
}

struct UserRepository: Repository {
    func fetch(id: User.ID) async throws -> User {
        User(id: id, name: "Ari")
    }
}
```

Possible follow-up questions: Why can protocols with associated types be hard to store? How would you type-erase this repository?

[Beginner] What are extensions in Swift?

Interview answer: Extensions add functionality to an existing type, protocol, or generic specialization without modifying the original declaration.

Deep explanation: Extensions can add computed properties, methods, initializers, nested types, protocol conformance, and protocol default implementations. They cannot add stored instance properties to existing types.

Why it matters: Extensions help organize code by capability and keep files focused. They also make protocol-oriented programming powerful because behavior can be added conditionally.

How it works: The compiler treats extension members as part of the type's available interface, subject to access control and module boundaries. Conditional extensions apply only when constraints are satisfied.

Use cases: Organizing `// MARK:` sections, protocol conformance, test helpers, formatters, SwiftUI view modifiers, and collection utilities.

Common mistakes: Using extensions to scatter a type's core behavior across too many files. Adding broad extensions to standard types with ambiguous names.

Best practices: Keep extensions cohesive: one conformance or capability per extension. Prefer explicit names that do not conflict with future platform APIs.

Red flag answers: "Extensions are inheritance." They are static additions, not subclassing.

Code example:

```
extension Collection where Element: Identifiable {
    func element(id: Element.ID) -> Element? {
        first { $0.id == id }
    }
}
```

Possible follow-up questions: Can extensions override methods? Can they add stored properties? What are conditional conformances?

[Intermediate] How does Swift error handling work?

Interview answer: Swift uses typed throwing functions marked with `throws`. Callers use `try`, `try?`, or `try!`, and handle errors with `do/catch` or propagate them from another throwing function.

Deep explanation: Swift errors are values, typically enums conforming to `Error`. Throwing separates expected failure from impossible programmer mistakes. It avoids exceptions for ordinary control flow while keeping error paths explicit.

Why it matters: Networking, decoding, persistence, permissions, and file access all fail in normal app operation. Good error modeling helps UI show meaningful recovery, not just "Something went wrong."

How it works: A throwing function has an alternate exit path. `try` marks the call site where control flow can leave the current path. `try?` converts success to optional value and failure to `nil`.

Use cases: API clients, repositories, data migrations, image loading, Keychain access, and business validation.

Common mistakes: Catching and ignoring errors. Using `try!` for recoverable failures. Exposing raw low-level errors directly to user-facing UI.

Best practices: Model domain errors where useful. Preserve underlying errors for logging. Convert technical failures to user-friendly state at the presentation boundary.

Red flag answers: "Use `try?` everywhere to avoid crashes." That hides failure and makes debugging harder.

Code example:

```
enum LoginError: LocalizedError {
    case invalidCredentials
    case networkUnavailable

    var errorDescription: String? {
```

```
        switch self {
        case .invalidCredentials:
            return "The email or password is incorrect."
        case .networkUnavailable:
            return "Check your connection and try again."
        }
    }
}

func login(email: String, password: String) async throws -> Session {
    guard !email.isEmpty, !password.isEmpty else {
        throw LoginError.invalidCredentials
    }
    // Network call...
    return Session(token: "token")
}
```

Possible follow-up questions: When would you use `Result` instead of `throws`? How do async and throwing compose?

[Beginner] What is access control in Swift?

Interview answer: Access control limits where declarations can be used. Swift has `private`, `fileprivate`, `internal`, `package`, `public`, and `open`. `internal` is the default.

Deep explanation: Access control protects module boundaries and API design. `public` exposes a symbol outside the module, but `open` additionally allows subclassing and overriding outside the module. `package` is useful in Swift packages to share within a package but not expose publicly.

Why it matters: Good access control reduces accidental coupling. It makes refactoring easier and keeps app modules from depending on implementation details.

How it works: The compiler enforces access rules at compile time. A declaration cannot expose another declaration that is less visible than itself.

Use cases: Framework APIs, modular architecture, package internals, testable boundaries, and hiding implementation details.

Common mistakes: Marking everything `public`. Using `private` so aggressively that related extensions become awkward. Confusing `public` with `open`.

Best practices: Start with the most restrictive access that fits the use case. Make APIs public only when they are intentional contracts.

Red flag answers: "Private means private to the class." In Swift, `private` is scoped to the declaration and its extensions in the same file, while `fileprivate` is file-wide.

Code example:

```
public struct UserProfile {
    public let id: UUID
    public private(set) var displayName: String

    public init(id: UUID, displayName: String) {
        self.id = id
        self.displayName = displayName
    }
}
```

Possible follow-up questions: What is the difference between `public` and `open`? How does access control affect testing?

[Beginner] What is a mutating method?

Interview answer: A `mutating` method is a method on a struct or enum that can change `self` or its stored properties.

Deep explanation: Value types are immutable by default inside instance methods because methods receive `self` as immutable unless marked `mutating`. Marking a method `mutating` tells the compiler the method may replace or modify the value.

Why it matters: It makes mutation explicit. Callers must hold the value in a `var`, not a `let`, to call a mutating method.

How it works: For structs, mutation changes stored properties. For enums, mutation can replace `self` with another case.

Use cases: State updates, builders, domain model transitions, counters, and enum state machines.

Common mistakes: Forgetting `mutating` on value-type methods. Expecting a mutating method to work on a `let` value. Using classes just to avoid `mutating`.

Best practices: Use `mutating` for intentional state transitions. For complex state transitions, consider returning a new value if that reads better.

Red flag answers: "Mutating is only for structs." Enums can have mutating methods too.

Code example:

```
struct RetryPolicy {
    private(set) var attempts = 0
    let maxAttempts: Int

    mutating func recordFailure() {
        attempts += 1
    }

    var canRetry: Bool {
        attempts < maxAttempts
    }
}
```

Possible follow-up questions: Why don't classes need `mutating`? How does `mutating` interact with protocol requirements?

[Intermediate] What are closures and capture lists?

Interview answer: A closure is a block of executable code that can be passed around. It can capture values from its surrounding scope. A capture list controls how values are captured, commonly using `[weak self]`, `[unowned self]`, or `[value]`.

Deep explanation: Closures are reference types. If a closure uses a variable from outside, Swift keeps that value alive for the closure's lifetime. By default, object references captured by closures are strong, which can create retain cycles.

Why it matters: Closures are everywhere in iOS: animations, completion handlers, Combine, SwiftUI callbacks, tasks, and collection transformations. Capture behavior directly affects memory and correctness.

How it works: The compiler creates a closure object that stores captured values. Capturing a variable can capture a reference to mutable storage; using `[value]` captures the current value.

Use cases: Callbacks, sorting/filtering, dependency injection factories, delayed work, SwiftUI actions, and async task bodies.

Common mistakes: Using `[weak self]` mechanically everywhere. Capturing stale values accidentally. Creating cycles between `self`, a stored closure, and the closure body.

Best practices: Use `[weak self]` when `self` owns the closure or the closure may outlive `self`. Capture specific dependencies when possible.

Red flag answers: "Always use weak self in closures." This can drop needed work and hide lifecycle issues.

Code example:

```
final class ImageLoader {
    var onComplete: ((UIImage) -> Void)?
```

```
    func start() {
        fetchImage { [weak self] image in
            guard let self else { return }
            self.onComplete?(image)
        }
    }

    private func fetchImage(completion: @escaping (UIImage) -> Void) {
        // Asynchronous work...
    }
}
```

Possible follow-up questions: When would you use `unowned`? What does `[value]` do? Are closures value types or reference types?

[Intermediate] What is the difference between escaping and non-escaping closures?

Interview answer: A non-escaping closure must be called before the function returns. An escaping closure can outlive the function call, so it must be marked `@escaping`.

Deep explanation: Non-escaping closures are safer and easier for the compiler to optimize because their lifetime is bounded by the function call. Escaping closures are stored, dispatched later, or executed asynchronously.

Why it matters: Most async callbacks, stored handlers, and delayed work use escaping closures. Escaping closures can capture `self` strongly and create retain cycles.

How it works: Closure parameters are non-escaping by default. Marking `@escaping` changes lifetime rules and requires explicit `self` capture in many contexts.

Use cases: Non-escaping: `map`, `filter`, synchronous validation. Escaping: network completion handlers, animation completions, notification callbacks, stored SwiftUI actions.

Common mistakes: Adding `@escaping` because the compiler asks without understanding why. Storing non-escaping closures. Forgetting memory implications.

Best practices: Keep closures non-escaping by default. Use async/await instead of escaping completion handlers for new asynchronous APIs.

Red flag answers: "Escaping means it runs on another thread." It means lifetime escapes; threading is separate.

Code example:

```
func validate(_ isValid: () -> Bool) -> Bool {
    isValid()
}

final class ButtonModel {
    private let action: () -> Void

    init(action: @escaping () -> Void) {
        self.action = action
    }

    func tap() {
        action()
    }
}
```

Possible follow-up questions: Why does escaping require explicit `self`? How does async/await change old completion-handler APIs?

[Senior] What is the difference between `some` and `any`?

Interview answer: `some Protocol` is an opaque type: the concrete type is hidden from the caller but fixed by the implementation. `any Protocol` is an existential: it can hold any concrete conforming type at runtime.

Deep explanation: `some` preserves a single concrete type relationship while hiding the exact type. This enables static type checking and optimization. `any` erases the concrete type behind a protocol interface, enabling heterogeneous storage but losing some static type information.

Why it matters: SwiftUI uses `some View` because every `body` has one concrete generated type, even if it is ugly. Existentials are useful at architecture boundaries, but they have dispatch and capability limitations.

How it works: Opaque types are chosen by the function implementation and known to the compiler. Existentials store a value plus metadata/witness table information needed to call protocol requirements dynamically.

Use cases: Use `some` for returning hidden but stable concrete types, especially SwiftUI views. Use `any` for arrays of mixed conforming types, dependency properties, or plugin-like systems.

Common mistakes: Thinking `some View` means "any view." Returning different concrete types from different branches without `@ViewBuilder` or type erasure. Using `any` when generics would preserve stronger type information.

Best practices: Prefer generics or `some` when the concrete type relationship matters. Use `any` deliberately when runtime heterogeneity is required.

Red flag answers: "`some` and `any` are interchangeable." They solve opposite problems: hiding a fixed type vs storing variable conforming types.

Code example:

```
protocol ShapeStyleProvider {
    associatedtype Style: ShapeStyle
    var style: Style { get }
}

func makeTitle() -> some View {
    Text("Profile")
        .font(.title)
}

let formatters: [any FormatStyle] = [
    Date.FormatStyle(date: .abbreviated, time: .omitted),
    IntegerFormatStyle<Int>()
]
```

Possible follow-up questions: Why does SwiftUI use `some View`? When do you need `AnyView`? What limitations do existentials have with associated types?

[Senior] What is type erasure?

Interview answer: Type erasure hides a concrete generic or associated type behind a stable wrapper type. It is useful when you need to store or return values with different concrete types through one interface.

Deep explanation: Protocols with associated types and generic types preserve type information, but that can make storage and API boundaries difficult. Type erasure creates a concrete wrapper, such as `AnySequence`, `AnyPublisher`, or `AnyView`, that forwards behavior while hiding the underlying type.

Why it matters: It helps build architecture seams and heterogeneous collections, but it also removes compile-time knowledge and can hurt optimization or SwiftUI diffing.

How it works: A wrapper stores closures or a boxed base object that implements operations. Callers interact with the wrapper instead of the original generic type.

Use cases: Heterogeneous repositories, hiding Combine publisher chains, plugin systems, SwiftUI conditional storage, and public API stability.

Common mistakes: Erasing too early. Using `AnyView` to avoid understanding SwiftUI's type system. Losing useful constraints and performance.

Best practices: Keep concrete/generic types internally. Erase at module boundaries, storage boundaries, or where heterogeneity is truly needed.

Red flag answers: "Type erasure makes code simpler." It can make call sites simpler while making debugging and performance less transparent.

Code example:

```
struct AnyRepository<Entity: Identifiable> {
    private let _fetch: (Entity.ID) async throws -> Entity

    init<R: Repository>(_ repository: R) where R.Entity == Entity {
        _fetch = repository.fetch
    }

    func fetch(id: Entity.ID) async throws -> Entity {
        try await _fetch(id)
    }
}
```

Possible follow-up questions: How does `AnyView` affect SwiftUI? How is type erasure different from `any Protocol`?

[Senior] What are result builders?

Interview answer: Result builders transform a block of expressions into a single result. SwiftUI's `@ViewBuilder` lets you write multiple child views declaratively and have the compiler combine them into a concrete view type.

Deep explanation: Result builders are a compile-time DSL feature. They rewrite code like multiple expressions, `if`, `switch`, and limited loops into calls such as `buildBlock`, `buildEither`, and `buildOptional`.

Why it matters: They explain SwiftUI syntax. A `VStack { Text("A"); Text("B") }` is not magic syntax for arrays; it is builder-transformed code producing a strongly typed view tree.

How it works: The builder type defines static build methods. The compiler applies those methods to the statements in the annotated closure or property.

Use cases: SwiftUI views, HTML DSLs, layout builders, settings builders, and test-data builders.

Common mistakes: Expecting arbitrary imperative code inside a builder. Confusing the builder's declarative output with runtime rendering.

Best practices: Use builders for declarative composition, not complex business logic. Extract complex conditions into computed properties or helper methods.

Red flag answers: "ViewBuilder returns an array of views." It returns a concrete composed type such as tuples, conditionals, or specialized SwiftUI container types.

Code example:

```
@ViewBuilder
func statusView(isOnline: Bool) -> some View {
    if isOnline {
        Label("Online", systemImage: "checkmark.circle")
    } else {
        Label("Offline", systemImage: "xmark.circle")
    }
}
```

Possible follow-up questions: Why can different branches compile inside `@ViewBuilder`? What are builder limitations?

Memory Management

[Beginner] What is ARC?

Interview answer: ARC, Automatic Reference Counting, manages class instance memory by tracking strong references. When the strong reference count reaches zero, the instance is deallocated and `deinit` runs.

Deep explanation: ARC handles memory for reference types automatically, but it does not understand ownership cycles. If two objects strongly reference each other, both counts stay above zero and neither deallocates.

Why it matters: iOS apps are memory constrained. Leaks cause increased memory usage, performance degradation, and eventual termination.

How it works: The compiler inserts retain and release operations around reference lifetimes. Strong references increment the count. Weak references do not. When the count reaches zero, Swift destroys the object.

Use cases: View controllers, view models, services, delegates, closures, timers, notifications, and async callbacks.

Common mistakes: Thinking ARC prevents all leaks. Forgetting closure retain cycles. Using `unowned` when the referenced object may outlive its owner.

Best practices: Design clear ownership. Use `weak` for back-references and delegates. Verify deallocation for complex flows.

Red flag answers: "Swift has garbage collection." ARC is deterministic reference counting, not tracing garbage collection.

Code example:

```
final class DetailViewModel {
    deinit {
        print("DetailViewModel deallocated")
    }
}
```

Possible follow-up questions: How does ARC differ from garbage collection? Why do retain cycles leak?

[Intermediate] What is the difference between stack and heap memory?

Interview answer: The stack is fast, scoped storage for function calls and local values. The heap is dynamic storage for values that need flexible lifetime or reference identity. In Swift, semantics matter more than assuming exact placement.

Deep explanation: Stack allocation is cheap and follows call lifetimes. Heap allocation is more flexible but requires allocation, deallocation, and often ARC traffic. Swift can optimize storage, so "struct equals stack" is not reliable.

Why it matters: It helps explain performance trade-offs without making false claims. Large values, escaping closures, class instances, and shared buffers often involve heap allocation.

How it works: Function frames hold local storage and return addresses. Heap objects are allocated from a dynamic memory region and referenced indirectly. The compiler may promote, inline, or eliminate allocations.

Use cases: Performance analysis, memory pressure debugging, ARC traffic reduction, and avoiding unnecessary object allocation in hot paths.

Common mistakes: Equating stack/heap with struct/class in all cases. Optimizing based on folklore instead of profiling.

Best practices: Choose value/reference semantics first. Profile allocations with Instruments before changing architecture for memory placement.

Red flag answers: "Structs are always stack allocated." This is false in optimized Swift code and when values are captured, stored, boxed, or part of heap-backed storage.

Code example:

```
func makeClosure() -> () -> Int {
```

```
    let value = 42
    return { value } // Captured value must outlive the stack frame.
}
```

Possible follow-up questions: When can a closure allocate? How does escape analysis help optimization?

[Beginner] What are strong, weak, and unowned references?

Interview answer: `strong` keeps an object alive. `weak` does not keep it alive and becomes `nil` when the object deallocates. `unowned` does not keep it alive and assumes the object is still alive; accessing it after deallocation crashes.

Deep explanation: Strong references express ownership. Weak references express optional non-ownership. Unowned references express non-optional non-ownership with a strict lifetime guarantee.

Why it matters: Correct reference strength prevents memory leaks and crashes.

How it works: ARC increments counts for strong references. Weak references are tracked separately and zeroed on deallocation. Unowned references are not zeroed in the same safe optional way.

Use cases: Strong: parent owns child. Weak: delegate, parent pointer, view controller callback. Unowned: child points to parent when parent definitely outlives child.

Common mistakes: Using `unowned` to avoid optional handling. Making delegates strong. Using weak references for objects that need to be retained.

Best practices: Prefer `weak` unless you can prove the lifetime invariant. Delegates should usually be weak. Ownership should be visible in API design.

Red flag answers: "Use unowned because it is faster." Safety beats micro-optimization.

Code example:

```
protocol PaymentCoordinatorDelegate: AnyObject {
    func paymentDidFinish()
}

final class PaymentCoordinator {
    weak var delegate: PaymentCoordinatorDelegate?
}
```

Possible follow-up questions: Why must weak references be optional? When is `unowned` appropriate?

[Intermediate] What is a retain cycle?

Interview answer: A retain cycle happens when objects strongly reference each other so ARC can never reduce their reference counts to zero.

Deep explanation: ARC works locally by counting references. It does not scan object graphs looking for unreachable cycles. A cycle can involve two objects, a closure stored by an object, timers, notification tokens, tasks, or delegates.

Why it matters: Retain cycles keep screens, view models, network clients, and resources alive after users leave a flow.

How it works: If `A` strongly owns `B` and `B` strongly owns `A`, both counts remain positive. For closure cycles, `self` owns a closure property and the closure strongly captures `self`.

Use cases: Diagnosing leaked view controllers, Combine subscriptions, repeating timers, stored callbacks, and coordinator graphs.

Common mistakes: Only checking direct property cycles and ignoring closure cycles. Assuming `deinit` will always run.

Best practices: Make back-references weak. In stored closures that refer to owner, capture `self` weakly or capture specific dependencies. Invalidate timers and subscriptions.

Red flag answers: "ARC will clean it eventually." It will not if the cycle remains.

Code example:

```
final class SearchViewModel {
```

```
    var onResults: (() -> Void)?

    func configure() {
        onResults = { [weak self] in
            self?.refreshUIState()
        }
    }

    private func refreshUIState() {}
}
```

Possible follow-up questions: Can value types participate in retain cycles? How do Combine subscriptions create cycles?

[Intermediate] Why is `[weak self]` used?

Interview answer: `[weak self]` prevents a closure from strongly retaining its owner when the closure may outlive the owner or is stored by the owner.

Deep explanation: Closures strongly capture objects by default. If a view model stores a closure and that closure captures the view model, the two can retain each other. Weak capture breaks the ownership cycle.

Why it matters: It prevents leaked view controllers, view models, coordinators, and cells in common asynchronous and callback-heavy code.

How it works: The capture list stores a weak optional reference. Inside the closure, `self` is optional because the object may have deallocated by execution time.

Use cases: Network completion handlers, animation completions, Combine `sink`, timers, notification handlers, and stored callback properties.

Common mistakes: Using weak self in short non-escaping closures where it is unnecessary. Guarding self and then starting long work without thinking about desired lifetime.

Best practices: Ask: who owns the closure, and can it outlive `self`? If yes, weak is often right. Capture specific collaborators when that is clearer.

Red flag answers: "Always use weak self in async code." Sometimes the task should keep the object alive until work finishes; sometimes cancellation should control lifetime.

Code example:

```
final class FeedViewModel {
    private let service: FeedService

    init(service: FeedService) {
        self.service = service
    }

    func load() {
        service.fetch { [weak self] result in
            guard let self else { return }
            self.handle(result)
        }
    }

    private func handle(_ result: Result<[Post], Error>) {}
}
```

Possible follow-up questions: When should you avoid `[weak self]`? How do you handle optional self elegantly?

[Senior] When should you not use `[weak self]`?

Interview answer: Do not use `[weak self]` when the closure cannot outlive `self`, when `self` must stay alive for the operation to be correct, or when ownership is better expressed through cancellation or explicit lifetime management.

Deep explanation: Weak capture is a memory tool, not a default style rule. If a user taps Save and the view model should finish a write even if the view disappears, weakly dropping the closure may silently cancel important work. Conversely, if the work is view-scoped, using a cancellable task may be better than weak self.

Why it matters: Mechanical weak captures create subtle bugs: missing analytics, incomplete saves, skipped cleanup, and nondeterministic behavior.

How it works: If `self` deallocates before the closure runs, `self` becomes `nil` and the closure often returns early. That changes behavior.

Use cases: Non-escaping closures, synchronous collection methods, transactions that must complete, dependency capture, and explicit task ownership.

Common mistakes: Treating weak capture as a universal leak fix. Ignoring whether the closure is stored. Weakly capturing `self` in `Task` but not cancelling the task.

Best practices: Model lifecycle intentionally. For view-scoped async work, use `.task` or store/cancel a `Task`. For required work, move it to a service that owns the operation.

Red flag answers: "Weak self has no downside." It has semantic consequences.

Code example:

```
// No weak self needed: map's closure is non-escaping.
let names = users.map { user in
    user.name.uppercased()
}

// Capture a dependency instead of all of self.
final class PurchaseViewModel {
    private let analytics: AnalyticsTracking

    init(analytics: AnalyticsTracking) {
        self.analytics = analytics
    }

    func trackPurchase() {
        Task { [analytics] in
            await analytics.track("purchase")
        }
    }
}
```

Possible follow-up questions: How do tasks affect object lifetime? How can you prove a closure is non-escaping?

[Intermediate] How do closures capture variables?

Interview answer: Closures capture values they use from surrounding scope. By default, object references are captured strongly; capture lists can change capture strength or capture a snapshot value.

Deep explanation: Capturing a mutable local variable can preserve shared mutable storage, so later changes may be visible to the closure. A capture list like `[count]` captures the current value at closure creation.

Why it matters: Capture semantics affect memory, race conditions, stale UI data, and correctness of delayed work.

How it works: The compiler builds storage for captured variables and references it from the closure object. Capture lists initialize captured values before the closure body runs.

Use cases: Delayed logging, animation callbacks, task closures, event handlers, and retry logic.

Common mistakes: Expecting a closure to always capture a snapshot. Capturing `self` when only one property or service is needed.

Best practices: Capture specific values or services intentionally. Use `[value]` when delayed work needs a snapshot.

Red flag answers: "Closures copy everything." Capture behavior depends on variable/value/reference semantics and capture lists.

Code example:

```
var query = "swift"

let liveCapture = {
    print(query)
}

let snapshotCapture = { [query] in
    print(query)
}

query = "swiftui"
liveCapture()     // swiftui
snapshotCapture() // swift
```

Possible follow-up questions: How do capture lists interact with value types? Can a struct capture a class reference?

[Intermediate] How do you find memory leaks in iOS?

Interview answer: Use Instruments, especially Leaks and Allocations, plus Xcode's Memory Graph Debugger. Confirm expected objects deallocate, inspect retain cycles, and reproduce the flow repeatedly.

Deep explanation: Memory debugging is evidence-driven. A growing memory graph does not always mean a leak; caches and system frameworks may retain memory intentionally. A leak is memory that remains reachable unintentionally after its expected lifetime.

Why it matters: Leaks degrade performance, trigger memory warnings, and can terminate apps under pressure.

How it works: Memory Graph shows object references at a point in time. Leaks detects unreachable allocations. Allocations tracks allocation patterns over time. `deinit` logs can confirm lifetime but are not enough by themselves.

Use cases: View controller leaks, Combine subscription cycles, timers, notification observers, image caches, and retain cycles in coordinators.

Common mistakes: Assuming one retained object is a leak without knowing expected ownership. Relying only on `deinit` print statements. Ignoring repeating timers and notification tokens.

Best practices: Create a minimal reproduce path. Navigate in and out multiple times. Check owner chains. Fix one ownership problem at a time.

Red flag answers: "Just add weak everywhere." That can break ownership and hide the root cause.

Code example:

```
final class ScreenViewModel {
    deinit {
        assertionFailure("Remove after leak investigation: ScreenViewModel deallocated")
    }
}
```

Possible follow-up questions: What is the difference between Leaks and Allocations? How can a timer cause a leak?

Swift Concurrency

[Beginner] What is async/await?

Interview answer: `async/await` lets asynchronous code read like synchronous code. An `async` function can suspend, and `await` marks a possible suspension point.

Deep explanation: Before async/await, asynchronous workflows used nested completion handlers. That made error handling, cancellation, ordering, and readability harder. Async/await gives the compiler a structured model of suspension.

Why it matters: iOS apps constantly fetch data, read files, request permissions, and update UI. Async/await makes these flows clearer and safer.

How it works: When code reaches `await`, the current task may suspend and free the underlying thread for other work. The task later resumes, not necessarily on the same thread unless actor isolation requires it.

Use cases: Networking, image loading, database access, file I/O, permissions, and sequential async workflows.

Common mistakes: Thinking `await` blocks a thread. Calling async code from sync code by blocking. Forgetting `try` with throwing async functions.

Best practices: Design async APIs from the top down. Keep UI mutations on `@MainActor`. Propagate cancellation where possible.

Red flag answers: "Async/await creates a new thread." It creates suspension points in tasks; threads are runtime implementation details.

Code example:

```
func loadUser(id: User.ID) async throws -> User {
    let url = URL(string: "https://example.com/users/\(id)")!
    let (data, _) = try await URLSession.shared.data(from: url)
    return try JSONDecoder().decode(User.self, from: data)
}
```

Possible follow-up questions: What is a suspension point? Can sync code call async code directly?

[Intermediate] What is a Task?

Interview answer: A `Task` is a unit of asynchronous work managed by Swift concurrency. It has priority, cancellation state, and can return a value or throw.

Deep explanation: Tasks run async code. Structured child tasks are tied to a parent scope. Unstructured tasks created with `Task {}` are not automatically scoped to a parent function's lifetime, though they inherit some context such as priority and actor context.

Why it matters: Tasks are how async work is launched from synchronous contexts such as button taps, view lifecycle hooks, and delegate methods.

How it works: The concurrency runtime schedules tasks on executors. A task progresses until it suspends, completes, throws, or is cancelled. Cancellation is cooperative.

Use cases: Launching async work from SwiftUI actions, storing a cancellable search task, bridging UIKit callbacks to async code, and background refresh operations.

Common mistakes: Creating unstructured tasks everywhere. Not storing tasks that need cancellation. Assuming cancellation automatically stops work.

Best practices: Prefer structured concurrency. Store and cancel unstructured tasks when they are tied to object lifecycle.

Red flag answers: "Task is the same as Thread." A task is a logical unit of async work, not a dedicated OS thread.

Code example:

```
@MainActor
final class SearchViewModel: ObservableObject {
    @Published private(set) var results: [ResultItem] = []
    private var searchTask: Task<Void, Never>?

    func search(_ query: String) {
        searchTask?.cancel()
        searchTask = Task {
            do {
                results = try await SearchService().search(query)
            } catch is CancellationError {
                // Expected when a newer search starts.
            } catch {
                results = []
            }
        }
    }
}
```

Possible follow-up questions: What context does `Task {}` inherit? How is `Task.detached` different?

[Senior] What is structured concurrency?

Interview answer: Structured concurrency means child tasks are scoped to a parent operation, so lifetimes, cancellation, priority, and errors flow predictably through the task tree.

Deep explanation: Instead of launching orphaned work, structured concurrency makes async work follow lexical structure. If a parent task is cancelled, children are cancelled. If a child throws in a throwing task group, the error can cancel the group.

Why it matters: It prevents runaway work, lost errors, and lifecycle leaks. It makes async code easier to reason about in apps where screens appear/disappear frequently.

How it works: `async let` creates fixed child tasks. `withTaskGroup` and `withThrowingTaskGroup` create dynamic child tasks. The scope cannot finish until child tasks finish or are cancelled.

Use cases: Fetching profile, posts, and settings in parallel; image preloading; batch uploads; fan-out/fan-in work.

Common mistakes: Using `Task {}` inside an async function instead of `async let` or a task group. Ignoring task group cancellation and partial results.

Best practices: Use `async let` for a known small number of concurrent operations. Use task groups for dynamic collections. Keep child work independent.

Red flag answers: "Structured concurrency just means async/await syntax." It specifically refers to scoped child-task lifetimes.

Code example:

```
func loadDashboard() async throws -> Dashboard {
    async let profile = profileService.profile()
    async let messages = messageService.unreadMessages()
    async let settings = settingsService.settings()

    return try await Dashboard(
        profile: profile,
        messages: messages,
        settings: settings
    )
}
```

Possible follow-up questions: When would you use a task group instead of `async let`? How do errors propagate?

[Senior] What is a TaskGroup?

Interview answer: A task group creates a dynamic set of child tasks and lets you collect their results as they complete.

Deep explanation: Task groups are useful when the number of tasks depends on runtime data. They preserve structured concurrency: the group scope waits for children and controls cancellation/error propagation.

Why it matters: Many app operations fan out over collections: loading thumbnails, validating items, uploading attachments, or fetching pages.

How it works: Inside `withTaskGroup`, you call `group.addTask`. Results are retrieved by iterating the group. With throwing groups, a thrown error can cancel remaining work depending on how you consume results.

Use cases: Parallel image loading, batch network requests, prefetching, indexing, and local file processing.

Common mistakes: Mutating shared state from child tasks. Adding too many tasks without backpressure. Ignoring cancellation.

Best practices: Return values from child tasks and aggregate in the parent. Limit concurrency manually if work is heavy or server-sensitive.

Red flag answers: "TaskGroup is for background threads." It is for scoped child tasks, not manual thread management.

Code example:

```
func loadThumbnails(for urls: [URL]) async -> [UIImage] {
    await withTaskGroup(of: UIImage?.self) { group in
        for url in urls {
            group.addTask {
                try? await ImageService().thumbnail(from: url)
            }
        }

        var images: [UIImage] = []
        for await image in group {
            if let image {
                images.append(image)
            }
        }
        return images
    }
}
```

Possible follow-up questions: How do you limit concurrency in a task group? What happens if a child task throws?

[Intermediate] What is `@MainActor`?

Interview answer: `@MainActor` isolates code to the main actor, which is the correct place for UI state updates. It is stronger than manually dispatching to the main queue because the compiler can enforce isolation.

Deep explanation: UIKit and SwiftUI UI state must be updated from the main thread/main actor. Annotating a view model or UI-facing method with `@MainActor` expresses that requirement in the type system.

Why it matters: It prevents race conditions and "Publishing changes from background threads" issues.

How it works: The main actor is a global actor. Calls from outside its isolation require `await`, allowing the runtime to hop to the main actor executor.

Use cases: View models, UI state mutation, UIKit adapter methods, observable model updates used by SwiftUI.

Common mistakes: Wrapping everything in `DispatchQueue.main.async` inside `@MainActor` code. Marking heavy CPU/network work `@MainActor` and blocking UI responsiveness.

Best practices: Mark UI-facing view models `@MainActor`. Keep heavy work in services or nonisolated functions, then assign results on the main actor.

Red flag answers: "MainActor always means the code is currently on the main thread." It means code is isolated to the main actor; execution and hopping are managed by the runtime.

Code example:

```
@MainActor
final class ProfileViewModel: ObservableObject {
```

```
    @Published private(set) var name = ""
    private let repository: UserRepository

    init(repository: UserRepository) {
        self.repository = repository
    }

    func load(id: User.ID) async {
        do {
            name = try await repository.user(id: id).name
        } catch {
            name = "Unavailable"
        }
    }
}
```

Possible follow-up questions: How is MainActor different from `DispatchQueue.main.async`? Can a MainActor function suspend?

[Senior] What are actors?

Interview answer: Actors are reference types that protect their mutable state through actor isolation. Only code isolated to the actor can access its mutable state directly; outside access requires `await`.

Deep explanation: Actors address shared mutable state in concurrent programs. Instead of manually locking, actor isolation serializes access to actor-isolated state through an executor.

Why it matters: Race conditions are a major source of subtle bugs. Actors give a language-level way to protect mutable state.

How it works: An actor has isolated state. Calls from outside may suspend while waiting for access. Actors are reentrant, so after an `await` inside an actor method, other work may run on the actor before the method resumes.

Use cases: Token stores, caches, counters, upload coordinators, in-memory databases, and shared mutable services.

Common mistakes: Assuming actors prevent all logical races. Forgetting actor reentrancy. Exposing mutable non-Sendable state across actor boundaries.

Best practices: Keep actor methods small. Be careful with invariants across `await` points. Return immutable snapshots.

Red flag answers: "Actors are just classes on the main thread." Actors are reference types with isolated state; they are not inherently main-thread-bound.

Code example:

```
actor TokenStore {
    private var token: String?

    func update(_ token: String) {
        self.token = token
    }

    func currentToken() -> String? {
        token
    }
}
```

Possible follow-up questions: What is actor reentrancy? What is `nonisolated`? Can actors conform to protocols?

[Senior] What is `Sendable`?

Interview answer: `Sendable` marks values that can be safely passed across concurrency domains. It helps the compiler prevent data races when tasks and actors exchange data.

Deep explanation: Value-semantic types whose stored properties are Sendable are usually safe. Mutable reference types are not automatically safe unless immutable, actor-isolated, or internally synchronized.

Why it matters: Swift's concurrency safety depends on knowing whether data can cross task or actor boundaries without introducing shared mutable state.

How it works: The compiler checks Sendable conformance in concurrency-sensitive contexts. `@Sendable` closures must capture values safely for concurrent execution.

Use cases: DTOs crossing actor boundaries, task group results, async service APIs, caches, and concurrency-safe closures.

Common mistakes: Using `@unchecked Sendable` as a silencer. Capturing non-thread-safe class instances in `@Sendable` closures.

Best practices: Make data transfer objects structs with Sendable stored properties. Use actors for mutable shared state. Reserve `@unchecked Sendable` for carefully audited synchronization.

Red flag answers: "Sendable makes a type thread-safe." It indicates safe transfer; thread safety still depends on the type's design.

Code example:

```
struct UserDTO: Decodable, Sendable {
    let id: UUID
    let name: String
}

final class LegacyCache: @unchecked Sendable {
    private let lock = NSLock()
    private var storage: [String: Data] = [:]

    func value(for key: String) -> Data? {
        lock.lock()
        defer { lock.unlock() }
        return storage[key]
    }
}
```

Possible follow-up questions: When is `@unchecked Sendable` justified? Are closures Sendable by default?

[Intermediate] How does task cancellation work?

Interview answer: Task cancellation is cooperative. Cancelling a task sets its cancellation state; the task must check cancellation or call APIs that throw `CancellationError`.

Deep explanation: Cancellation does not kill running code immediately. This avoids unsafe abrupt termination. Well-designed async functions periodically check cancellation and clean up.

Why it matters: Screens disappear, searches change, users cancel uploads, and SwiftUI tasks are cancelled automatically. Ignoring cancellation wastes battery, network, and CPU.

How it works: Call `task.cancel()` or cancellation flows from parent to child. Check `Task.isCancelled`, call `try Task.checkCancellation()`, or rely on cancellable APIs such as `Task.sleep`.

Use cases: Search-as-you-type, image loading in scrolling lists, view-scoped data loading, uploads, and long computations.

Common mistakes: Assuming cancellation stops synchronous CPU loops. Swallowing `CancellationError` as a real failure. Updating UI after a cancelled result.

Best practices: Check cancellation after awaited work and inside loops. Treat cancellation as a neutral outcome.

Red flag answers: "Cancel guarantees the network request stops instantly." It requests cancellation; behavior depends on the API and timing.

Code example:

```
func filterLargeDataset(_ items: [Item], query: String) async throws -> [Item] {
    var results: [Item] = []
```

```
    for item in items {
        try Task.checkCancellation()
        if item.title.localizedCaseInsensitiveContains(query) {
            results.append(item)
        }
    }

    return results
}
```

Possible follow-up questions: How does `.task(id:)` use cancellation? How do you avoid showing stale results?

[Senior] What is a race condition?

Interview answer: A race condition happens when program correctness depends on unpredictable timing between concurrent operations. A data race is a specific unsafe case where multiple threads/tasks access the same mutable memory concurrently and at least one writes.

Deep explanation: Not all race conditions are data races. Actors can prevent data races but still allow logical races if state assumptions cross suspension points.

Why it matters: Race conditions cause flaky tests, inconsistent UI, duplicate requests, stale writes, and hard-to-reproduce crashes.

How it works: Concurrent operations interleave differently depending on scheduling, network timing, device load, and suspension points.

Use cases: Search results arriving out of order, double form submission, token refresh, cache updates, and actor methods with invariants across `await`.

Common mistakes: Using a serial queue or actor and assuming all ordering bugs are solved. Updating UI with stale async results.

Best practices: Define ordering rules. Use cancellation, request IDs, actors, locks, or serial executors appropriately. Avoid shared mutable state.

Red flag answers: "Race condition means two threads write at the same time." That describes data races; logical races are broader.

Code example:

```
@MainActor
final class SearchModel: ObservableObject {
    @Published private(set) var results: [Item] = []
    private var latestQuery = ""

    func search(_ query: String) {
        latestQuery = query

        Task {
            let response = try? await SearchService().search(query)
            guard query == latestQuery else { return }
            results = response ?? []
        }
    }
}
```

Possible follow-up questions: How can actor reentrancy cause logical races? How do you test race conditions?

[Intermediate] How does GCD compare with Swift Concurrency?

Interview answer: GCD is a lower-level queue-based API. Swift Concurrency is a language-level model with async/await, tasks, actors, structured concurrency, cancellation, priority, and compiler checks.

Deep explanation: GCD is still useful for some legacy APIs and low-level scheduling. Swift Concurrency gives clearer control flow and safety. You usually should not mix them unnecessarily in new code.

Why it matters: Modern iOS code should prefer async/await for readability and correctness, while understanding GCD for older codebases.

How it works: GCD dispatches blocks to queues. Swift Concurrency schedules tasks on executors and uses suspension points instead of explicit callback nesting.

Use cases: GCD: legacy callback APIs, dispatch sources, low-level synchronization. Swift Concurrency: networking, data loading, view tasks, actor-isolated services.

Common mistakes: Calling `DispatchQueue.main.async` from `@MainActor` code. Blocking async functions with semaphores. Wrapping every async function in `Task`.

Best practices: Use Swift Concurrency for new async APIs. Bridge legacy callbacks with continuations carefully.

Red flag answers: "GCD is obsolete." It remains part of the platform, but it is no longer the preferred high-level app concurrency model.

Code example:

```
func legacyImage(for url: URL, completion: @escaping (UIImage?) -> Void) {
    // Old callback API.
}

func image(for url: URL) async -> UIImage? {
    await withCheckedContinuation { continuation in
        legacyImage(for: url) { image in
            continuation.resume(returning: image)
        }
    }
}
```

Possible follow-up questions: What is a checked continuation? Why are semaphores dangerous with async code?

[Intermediate] How do you safely update UI from async code?

Interview answer: Keep UI state isolated to the main actor. Annotate view models or UI methods with `@MainActor`, or use `await MainActor.run` for a small UI update from non-main-actor code.

Deep explanation: SwiftUI and UIKit expect UI state changes on the main actor. Modern Swift lets you express that as part of your types instead of sprinkling dispatch calls everywhere.

Why it matters: Wrong-thread UI updates lead to warnings, races, glitches, and crashes.

How it works: Actor isolation makes cross-actor UI updates require `await`. The runtime hops to the main actor before executing isolated code.

Use cases: Assigning `@Published` properties, mutating `@Observable` models used by views, updating UIKit views after network calls.

Common mistakes: Doing heavy decoding or image processing on `@MainActor`. Updating UI after the view disappeared or task was cancelled.

Best practices: Fetch and process off the main actor where possible; assign final state on the main actor. Make view models `@MainActor`.

Red flag answers: "Use DispatchQueue.main.async after every await." That is a sign the isolation model is unclear.

Code example:

```
func loadAndAssign() async {
    let user = try? await repository.currentUser()

    await MainActor.run {
        self.user = user
    }
}
```

Possible follow-up questions: When should the whole type be `@MainActor`? What happens after `await` in a main-actor method?

SwiftUI Core

[Beginner] What is a SwiftUI `View`?

Interview answer: A SwiftUI `View` is a value that describes part of the UI. Its `body` returns another view describing the current UI for the current state.

Deep explanation: SwiftUI views are not the rendered pixels or platform view objects. They are lightweight descriptions that SwiftUI evaluates, diffs, and uses to update an internal render tree.

Why it matters: Many SwiftUI misunderstandings come from treating views like UIKit views. In SwiftUI, you do not hold and mutate view objects directly; you change state and let the framework update the UI.

How it works: The `View` protocol has an associated `Body` type. The `body` property builds a view description. SwiftUI tracks dependencies and identity to decide what to update.

Use cases: Building screens declaratively, composing UI pieces, previews, reusable components, and platform-agnostic UI.

Common mistakes: Putting imperative side effects in `body`. Assuming a view struct has stable lifetime. Storing business logic directly in a view.

Best practices: Keep views cheap and declarative. Move business logic to view models or use cases. Treat `body` as a function of state.

Red flag answers: "A SwiftUI View is like a UIView." It is more like a description used to produce/update rendered UI.

Code example:

```
struct ProfileHeader: View {
    let name: String

    var body: some View {
        Text(name)
            .font(.title)
            .bold()
    }
}
```

Possible follow-up questions: Why is `body some View`? Is `body` stored? How often can `body` run?

[Beginner] Why are SwiftUI views structs?

Interview answer: SwiftUI views are structs because they are lightweight value descriptions of UI. Recreating them is cheap, and value semantics make UI a predictable function of state.

Deep explanation: SwiftUI separates view descriptions from persistent rendering/storage. The struct can be recreated often while framework-managed storage holds state and platform resources.

Why it matters: It explains why you should not rely on view init/body lifetime for side effects and why state wrappers exist.

How it works: The view struct is evaluated to produce a tree. SwiftUI stores persistent state outside the transient struct, keyed by view identity.

Use cases: Frequent UI recomputation, previews, state-driven rendering, and lightweight composition.

Common mistakes: Adding stored mutable properties to a view and expecting them to persist. Running network calls from `init` or `body`.

Best practices: Use property wrappers for state. Use `.task`, `onAppear`, or view models for side effects. Keep view initialization cheap.

Red flag answers: "Views are structs for performance only." Performance is part of it, but the deeper reason is declarative value semantics.

Code example:

```
struct CounterView: View {
    @State private var count = 0

    var body: some View {
        Button("Count: \(count)") {
            count += 1
        }
    }
}
```

Possible follow-up questions: Where is `@State` stored if the view is a struct? Can a view have a class dependency?

[Intermediate] Is `body` the actual rendered UI?

Interview answer: No. `body` is a computed description of UI for current state. SwiftUI uses that description to update its internal view graph and platform rendering.

Deep explanation: `body` may be recomputed many times. It should be deterministic and cheap. The rendered UI is managed by SwiftUI, not by your view struct directly.

Why it matters: It prevents bad patterns like starting network calls in `body`, relying on body call counts, or doing expensive work there.

How it works: State changes invalidate dependent views. SwiftUI asks for new descriptions and reconciles them with previous identity/type structure.

Use cases: Conditional UI, state-driven modifiers, dynamic lists, and adaptive layout.

Common mistakes: Printing in `body` and assuming each print means a full screen redraw. Doing sorting, date formatting, or image processing in `body`.

Best practices: Precompute expensive data in models or view models. Keep `body` declarative. Use computed properties only when cheap.

Red flag answers: "Every body call redraws the screen." SwiftUI can recompute descriptions without necessarily repainting everything.

Code example:

```
struct UserList: View {
    let users: [User]

    var body: some View {
        List(users) { user in
            Text(user.name)
        }
    }
}
```

Possible follow-up questions: What causes body recomputation? How does SwiftUI know what changed?

[Intermediate] What is view identity vs data identity?

Interview answer: View identity tells SwiftUI whether a view is the same UI node across updates. Data identity identifies model elements, especially in collections. Stable identity lets SwiftUI preserve state, animations, focus, and scroll behavior.

Deep explanation: SwiftUI identity comes from structural position, type, and explicit IDs. In lists, each row's data ID helps SwiftUI match old rows to new rows. If identity changes unnecessarily, SwiftUI destroys and recreates state.

Why it matters: Identity mistakes cause rows to flicker, animations to reset, text fields to lose focus, navigation to pop, and tasks to restart.

How it works: SwiftUI reconciles old and new view trees. It compares identity to decide whether to update an existing node or create a new one.

Use cases: `ForEach`, `List`, `.id()`, dynamic forms, navigation paths, matched geometry, and row state.

Common mistakes: Using `UUID()` inline as an ID. Using array indices as IDs for mutable lists. Applying `.id()` to force refresh and accidentally resetting state.

Best practices: Use stable model IDs. Use `.id()` only when you intentionally want identity reset or scroll targeting.

Red flag answers: "ID is only for making ForEach compile." ID controls preservation and diffing behavior.

Code example:

```
struct Message: Identifiable {
    let id: UUID
    var text: String
}

List(messages) { message in
    Text(message.text)
}
```

Possible follow-up questions: When can array indices be safe IDs? How can `.id()` fix or break behavior?

[Senior] How does SwiftUI know what changed?

Interview answer: SwiftUI tracks state dependencies and reconciles new view descriptions against the previous view graph using type, identity, and invalidation information. It updates affected parts of the UI rather than requiring manual mutation.

Deep explanation: SwiftUI does not require you to tell a label to change text. You mutate state, and SwiftUI invalidates views that depend on that state. With Observation, SwiftUI can track property-level reads more precisely for observable models.

Why it matters: Understanding this helps write efficient SwiftUI: stable identity, clear state ownership, and minimal invalidation.

How it works: Property wrappers and observable models connect state changes to view invalidation. SwiftUI reevaluates view bodies and performs reconciliation/diffing to update the render tree.

Use cases: State-driven screens, forms, list updates, model observation, animations, and data flow debugging.

Common mistakes: Assuming SwiftUI deep-compares all views. Using broad observable objects where every change invalidates too much. Creating unstable identity.

Best practices: Keep state close to where it is used. Split large views along state boundaries. Use Observation on iOS 17+ for finer-grained model tracking.

Red flag answers: "SwiftUI just rerenders everything." It recomputes descriptions as needed and reconciles updates; the actual rendering pipeline is more selective.

Code example:

```
@Observable
final class ProfileModel {
    var name = ""
    var avatarURL: URL?
}

struct NameView: View {
    let model: ProfileModel

    var body: some View {
        Text(model.name) // Tracks this property read.
    }
}
```

Possible follow-up questions: How does Observation differ from `ObservableObject`? How do you reduce invalidation?

[Intermediate] Why does SwiftUI use `some View`?

Interview answer: SwiftUI uses `some View` to hide the exact concrete view type while preserving static type information and performance.

Deep explanation: The concrete type of a SwiftUI body can be deeply nested, like modified tuples of views. Exposing that type would be unusable. `some View` lets the implementation return one concrete type without naming it.

Why it matters: It avoids type erasure and keeps SwiftUI efficient. It also explains why branches must produce compatible types unless a result builder handles them.

How it works: The compiler knows the concrete return type, but callers only know it conforms to `View`. The concrete type must be consistent for that function/property.

Use cases: Every SwiftUI `body`, reusable view factories, computed view properties.

Common mistakes: Trying to store `some View` in a property without initialization constraints. Thinking it permits any view type dynamically.

Best practices: Return `some View` from view-building functions. Use `@ViewBuilder` for conditional branches. Avoid `AnyView` unless necessary.

Red flag answers: "`some View` is type erasure." It is opaque typing, not existential erasure.

Code example:

```
var body: some View {
    VStack {
        Text("Inbox")
        Divider()
        MessageList()
    }
}
```

Possible follow-up questions: Why does `if` sometimes work in `body`? How does `AnyView` differ?

[Senior] When should you use `AnyView`, and why can it hurt performance?

Interview answer: Use `AnyView` only when you need type erasure for storage or API boundaries. It can hurt performance because it hides concrete view structure from SwiftUI, reducing optimization and diffing precision.

Deep explanation: SwiftUI relies on static type structure to understand view trees. `AnyView` boxes the underlying view behind a uniform type. That can be useful, but overuse removes information SwiftUI could use.

Why it matters: In large lists or frequently updated views, unnecessary type erasure can increase work and make identity behavior less predictable.

How it works: `AnyView` stores a boxed view. SwiftUI sees `AnyView` instead of the original concrete type chain.

Use cases: Heterogeneous view arrays, plugin-rendered UI, APIs that must return runtime-selected views, legacy abstraction boundaries.

Common mistakes: Wrapping every conditional branch in `AnyView` instead of using `@ViewBuilder`. Using `AnyView` to silence compiler errors from overly complex view code.

Best practices: Prefer `@ViewBuilder`, enums, generic view parameters, or separate view structs. Erase only at the boundary that needs erasure.

Red flag answers: "AnyView is always bad." It is a tool with a cost; the issue is unnecessary or hot-path use.

Code example:

```
@ViewBuilder
func accessory(for state: LoadState) -> some View {
    switch state {
    case .loading:
        ProgressView()
    case .failed:
```

```
            Image(systemName: "exclamationmark.triangle")
        case .loaded:
            Image(systemName: "checkmark")
        }
    }
```

Possible follow-up questions: How would you design a heterogeneous SwiftUI plugin API? What alternatives exist to `AnyView`?

[Intermediate] How do view modifiers work, and why does modifier order matter?

Interview answer: A modifier returns a new view that wraps or transforms the previous view. Order matters because each modifier applies to the result of the modifiers before it.

Deep explanation: SwiftUI modifiers are compositional. `.padding().background()` means the background sees the padded size. `.background().padding()` means padding sits outside the background.

Why it matters: Modifier order affects layout, hit testing, animation, styling, accessibility, and performance.

How it works: Each modifier creates a new generic view type in the chain. SwiftUI evaluates the chain to produce layout and rendering behavior.

Use cases: Reusable styles, layout composition, custom controls, animations, and accessibility.

Common mistakes: Applying `.frame`, `.padding`, `.background`, and `.clipShape` in the wrong order. Expecting modifiers to mutate the original view object.

Best practices: Read modifier chains from top to bottom as transformations. Extract repeated chains into custom modifiers or view extensions.

Red flag answers: "Modifiers are just styling." Many modifiers affect layout, identity, tasks, gestures, and accessibility.

Code example:

```
Text("Pay")
    .padding(.horizontal, 16)
    .padding(.vertical, 10)
    .background(Color.blue)
    .foregroundStyle(.white)
    .clipShape(Capsule())
```

Possible follow-up questions: Why does `.background` sometimes not fill the expected area? How do custom `ViewModifiers` work?

[Intermediate] What is a custom `ViewModifier`?

Interview answer: A custom `ViewModifier` packages reusable view transformations into a named type.

Deep explanation: View modifiers keep styling consistent and reduce repeated chains. They can read environment values and accept parameters.

Why it matters: Reusable modifiers improve maintainability and design consistency in large SwiftUI apps.

How it works: A `ViewModifier` implements `body(content:)`, receives the original content, and returns a modified view.

Use cases: Button styles, card surfaces, error banners, accessibility traits, skeleton loading, and conditional styling.

Common mistakes: Putting business logic in modifiers. Creating modifiers that hide too much layout behavior.

Best practices: Name modifiers after their semantic role. Keep them focused and composable.

Red flag answers: "Use a modifier for every repeated line." Sometimes a custom view or style protocol is more appropriate.

Code example:

```
struct PrimaryCardModifier: ViewModifier {
    func body(content: Content) -> some View {
        content
            .padding(16)
            .background(.background)
            .clipShape(RoundedRectangle(cornerRadius: 8))
            .shadow(radius: 2)
    }
}

extension View {
    func primaryCard() -> some View {
        modifier(PrimaryCardModifier())
    }
}
```

Possible follow-up questions: How is a `ViewModifier` different from a custom `View`? Can modifiers have state?

[Intermediate] What are environment values and `EnvironmentObject`?

Interview answer: `@Environment` reads values supplied by the SwiftUI environment, such as color scheme, locale, dismiss action, or custom keys. `@EnvironmentObject` reads an observable object injected into the environment.

Deep explanation: Environment is a dependency propagation mechanism for values that many descendants need. It avoids passing the same dependency through every initializer. `EnvironmentObject` is older SwiftUI's way to share observable reference models by type.

Why it matters: Used well, environment reduces boilerplate. Used poorly, it hides dependencies and creates runtime crashes when objects are missing.

How it works: SwiftUI stores environment values in the view tree. Descendants read the nearest value for a key or object type.

Use cases: Theme, locale, calendar, dismiss, managed object context, app session, feature flags, and shared navigation model.

Common mistakes: Using environment for feature-specific dependencies that should be initializer-injected. Forgetting to inject `EnvironmentObject`, causing runtime failure.

Best practices: Use environment for cross-cutting UI context. Prefer explicit init dependencies for required feature dependencies. Avoid multiple environment objects of the same type unless carefully scoped.

Red flag answers: "EnvironmentObject is dependency injection for everything." It is global-ish tree injection and should be used carefully.

Code example:

```
private struct AnalyticsKey: EnvironmentKey {
    static let defaultValue: AnalyticsTracking = NoopAnalytics()
}

extension EnvironmentValues {
    var analytics: AnalyticsTracking {
        get { self[AnalyticsKey.self] }
        set { self[AnalyticsKey.self] = newValue }
    }
}

struct PayButton: View {
    @Environment(\.analytics) private var analytics

    var body: some View {
        Button("Pay") {
            analytics.track("pay_tapped")
        }
    }
}
```

Possible follow-up questions: When would you avoid `EnvironmentObject`? How do custom environment keys work?

[Senior] What are `PreferenceKey` and anchor preferences?

Interview answer: `PreferenceKey` lets child views send values up the view tree. Anchor preferences send layout anchors, such as bounds, that ancestors can resolve in a coordinate space.

Deep explanation: SwiftUI data normally flows down. Preferences are an escape hatch for child-to-parent communication, especially layout information that only children know.

Why it matters: Some layouts need parent decisions based on child size or position: tab indicators, sticky headers, scroll offset, custom overlays, and matched decorations.

How it works: A child sets a preference. Ancestors observe or transform preferences. A preference key defines a default value and a reduce function to combine multiple child values.

Use cases: Measuring child size, custom segmented controls, overlay alignment, reading scroll position, and tooltip positioning.

Common mistakes: Using preferences for ordinary state communication. Creating feedback loops where layout changes update state and trigger endless layout.

Best practices: Use preferences for layout metadata only. Keep reduced values small and stable.

Red flag answers: "PreferenceKey is like Binding upward." It is not a general state mutation channel.

Code example:

```
struct SizePreferenceKey: PreferenceKey {
    static var defaultValue: CGSize = .zero

    static func reduce(value: inout CGSize, nextValue: () -> CGSize) {
        value = nextValue()
    }
}

extension View {
    func readSize(_ onChange: @escaping (CGSize) -> Void) -> some View {
        background {
            GeometryReader { proxy in
                Color.clear.preference(key: SizePreferenceKey.self, value: proxy.size)
            }
        }
        .onPreferenceChange(SizePreferenceKey.self, perform: onChange)
    }
}
```

Possible follow-up questions: How do anchor preferences differ from normal preferences? How can preferences cause layout loops?

[Intermediate] What is `CoordinateSpace`?

Interview answer: `CoordinateSpace` defines the coordinate system used to interpret a view's frame or gestures: local, global, or named.

Deep explanation: Geometry is meaningless without knowing which coordinate system it belongs to. SwiftUI lets you ask for frames relative to local view bounds, the global screen/window context, or a named ancestor.

Why it matters: Custom layouts, scroll effects, drag gestures, overlays, and sticky headers need consistent coordinate interpretation.

How it works: `GeometryProxy.frame(in:)` converts the view's frame into a coordinate space. Named spaces are attached to ancestors with `.coordinateSpace(name:)`.

Use cases: Reading scroll offset, positioning popovers, drag-to-reorder, parallax headers, and custom gestures.

Common mistakes: Using `.global` when a named scroll view coordinate space is more stable. Comparing frames from different spaces.

Best practices: Use named coordinate spaces for components. Keep coordinate calculations isolated in layout/helper views.

Red flag answers: "GeometryReader always gives screen coordinates." It gives geometry relative to the coordinate space you ask for.

Code example:

```
ScrollView {
    GeometryReader { proxy in
        let minY = proxy.frame(in: .named("feedScroll")).minY
        Color.clear
            .onChange(of: minY) { _, value in
                print(value)
            }
    }
    .frame(height: 0)

    FeedContent()
}
.coordinateSpace(name: "feedScroll")
```

Possible follow-up questions: When should you avoid `GeometryReader`? How does coordinate space affect gestures?

[Senior] What are `EquatableView` and `.equatable()`?

Interview answer: `EquatableView` and `.equatable()` tell SwiftUI it can skip updating a view's body when its equatable input has not changed.

Deep explanation: They are performance tools for views where equality is cheaper than recomputing the body and where the view output truly depends only on equatable input.

Why it matters: Used correctly, they reduce work. Used incorrectly, they can hide real changes and make UI stale.

How it works: SwiftUI compares the new and old equatable values. If equal, it can avoid reevaluating that view subtree.

Use cases: Expensive row rendering, charts, complex static formatting, and views backed by stable value models.

Common mistakes: Applying `.equatable()` to views with hidden dependencies, environment changes, animations, or external observed state.

Best practices: Use after profiling. Ensure equality covers all input that affects rendering.

Red flag answers: "Add `.equatable()` to make SwiftUI faster." It is not a blanket optimization.

Code example:

```
struct ScoreBadge: View, Equatable {
    let score: Int
    let level: String

    var body: some View {
        Text("\(level): \(score)")
            .font(.headline)
    }
}

ScoreBadge(score: score, level: level)
    .equatable()
```

Possible follow-up questions: What hidden inputs can break `.equatable()`? How would you profile whether it helps?

SwiftUI State and Observation

[Beginner] What is `@State`?

Interview answer: `@State` is local, view-owned mutable state managed by SwiftUI. Use it for simple state that belongs to a specific view.

Deep explanation: Because views are structs and recreated often, ordinary stored properties do not persist as mutable UI state. `@State` stores the value in SwiftUI-managed storage keyed by view identity.

Why it matters: It is the basic source of truth for local UI state like toggles, selected tabs, text input, and expansion flags.

How it works: SwiftUI keeps storage outside the transient view struct. Mutating the state invalidates the view and recomputes body.

Use cases: Form fields, selected segment, local presentation booleans, temporary UI flags, and animation state.

Common mistakes: Using `@State` for data owned by a parent or shared model. Initializing `@State` from changing input and expecting it to stay synced automatically.

Best practices: Keep `@State` private. Lift state up when multiple views need to coordinate.

Red flag answers: "@State is just a stored property." It is framework-managed persistent storage.

Code example:

```
struct SettingsToggle: View {
    @State private var isEnabled = false

    var body: some View {
        Toggle("Enable notifications", isOn: $isEnabled)
    }
}
```

Possible follow-up questions: Where is `@State` stored? What happens when view identity changes?

[Beginner] What is `@Binding`?

Interview answer: `@Binding` is a two-way connection to state owned elsewhere. It lets a child read and write a parent's source of truth without owning it.

Deep explanation: Bindings avoid duplicating state. A child can edit a value, but ownership remains with the parent or model that supplied the binding.

Why it matters: It is fundamental for forms, reusable controls, sheets, and child components.

How it works: A binding wraps getter and setter closures. `$state` produces a binding to a `@State` value.

Use cases: Text fields, toggles, custom controls, selection, sheet booleans, and edit screens.

Common mistakes: Creating local copies of parent state instead of binding. Passing binding too deeply where explicit actions would be clearer.

Best practices: Use bindings for direct child editing. Use callbacks or view models for complex business actions.

Red flag answers: "Binding owns the value." It points to someone else's source of truth.

Code example:

```
struct FavoriteToggle: View {
    @Binding var isFavorite: Bool

    var body: some View {
        Toggle("Favorite", isOn: $isFavorite)
    }
}
```

Possible follow-up questions: How do you create a custom binding? When is binding overused?

[Intermediate] What are `@Observable` and `@Bindable`?

Interview answer: `@Observable` marks a reference type as observable using Swift's Observation framework. `@Bindable` creates bindings to properties of an observable model, commonly for form controls.

Deep explanation: Observation is the modern iOS 17+ replacement for many `ObservableObject` use cases. It tracks property access so SwiftUI can update views that read changed properties.

Why it matters: It reduces boilerplate and can provide more precise invalidation than `ObservableObject` with broad `objectWillChange`.

How it works: The `@Observable` macro expands the type with observation tracking. SwiftUI records observed property reads during body evaluation. `@Bindable` exposes mutable bindings to observable properties.

Use cases: View models, form models, settings models, app state objects, and feature state.

Common mistakes: Using `@StateObject` with `@Observable` models unnecessarily. Forgetting iOS 17 minimum. Making every service observable.

Best practices: For iOS 17+ SwiftUI, prefer `@Observable` for UI models. Use `@State` to own an observable model in a view.

Red flag answers: "@Observable is just ObservableObject renamed." It uses a different Observation system and does not require `@Published`.

Code example:

```
@Observable
final class ProfileForm {
    var name = ""
    var email = ""
}

struct ProfileFormView: View {
    @State private var form = ProfileForm()

    var body: some View {
        @Bindable var form = form

        Form {
            TextField("Name", text: $form.name)
            TextField("Email", text: $form.email)
        }
    }
}
```

Possible follow-up questions: How does Observation differ from Combine? When would you still use `ObservableObject`?

[Intermediate] What are `@StateObject` and `@ObservedObject`?

Interview answer: `@StateObject` creates and owns an `ObservableObject` for a view's lifetime. `@ObservedObject` observes an object owned elsewhere.

Deep explanation: With Combine-era SwiftUI, object ownership matters. If a view creates an observable object with `@ObservedObject`, SwiftUI may recreate it when the view struct is recreated. `@StateObject` preserves it for the view identity.

Why it matters: Wrong ownership causes lost state, duplicate network calls, and unstable view models.

How it works: `@StateObject` stores the object in SwiftUI-managed storage. `@ObservedObject` subscribes to an external object's `objectWillChange`.

Use cases: `@StateObject`: view-owned view model. `@ObservedObject`: object passed from parent/coordinator.

Common mistakes: Using `@ObservedObject var vm = ViewModel()` in a view. Passing `@StateObject` into child views instead of `@ObservedObject`.

Best practices: Own once with `@StateObject`, observe elsewhere with `@ObservedObject`. For iOS 17+ Observation, prefer `@State` with `@Observable` models.

Red flag answers: "They are basically the same." Observation and ownership behavior differ.

Code example:

```
final class LegacyProfileViewModel: ObservableObject {
    @Published var name = ""
}

struct ProfileScreen: View {
    @StateObject private var viewModel = LegacyProfileViewModel()

    var body: some View {
        ProfileContent(viewModel: viewModel)
    }
}

struct ProfileContent: View {
    @ObservedObject var viewModel: LegacyProfileViewModel

    var body: some View {
        Text(viewModel.name)
    }
}
```

Possible follow-up questions: What problem did `@StateObject` solve? How does this change with `@Observable`?

[Intermediate] What are `ObservableObject` and `@Published`?

Interview answer: `ObservableObject` is a Combine-based protocol for reference types whose changes SwiftUI can observe. `@Published` marks properties that emit `objectWillChange` before changing.

Deep explanation: Before iOS 17 Observation, this was the main way to model view state in reference-type view models. The object emits a broad change notification, and observing views update.

Why it matters: Many production apps still support older OS versions and use `ObservableObject`. Interviewers expect you to know both old and new systems.

How it works: `@Published` publishes changes through Combine. SwiftUI subscribes to `objectWillChange` and invalidates views observing the object.

Use cases: Apps supporting iOS 13-16, Combine pipelines, shared view models, and legacy codebases.

Common mistakes: Publishing from background threads. Marking every property as `@Published` even if UI does not observe it. Creating one huge object that invalidates many views.

Best practices: Mark UI-facing models `@MainActor`. Keep objects focused. Use `private(set)` for read-only UI state.

Red flag answers: "@Published changes after didSet only." Combine's `@Published` emits in `willSet`, which can surprise people outside SwiftUI.

Code example:

```
@MainActor
final class InboxViewModel: ObservableObject {
    @Published private(set) var messages: [Message] = []

    func load() async {
        messages = (try? await InboxService().messages()) ?? []
    }
}
```

Possible follow-up questions: What thread should `@Published` updates happen on? Why can `objectWillChange` be too broad?

[Senior] Observation framework vs `ObservableObject`: what changed?

Interview answer: Observation is macro-based, does not require Combine, does not require every observed property to be marked `@Published`, and can track property reads more precisely. `ObservableObject` is Combine-based and emits broader object-level changes.

Deep explanation: Observation was introduced to make observable reference models easier, more general, and better integrated with modern Swift. It avoids repeating `@Published` and supports computed property tracking patterns better.

Why it matters: Modern SwiftUI interviews increasingly ask about the iOS 17 data flow model. Strong candidates know how to migrate without blindly mixing both systems.

How it works: `@Observable` macro instruments property access and mutation. SwiftUI tracks which properties a view reads. `ObservableObject` uses Combine's `objectWillChange` publisher.

Use cases: Use Observation for iOS 17+ SwiftUI models. Use `ObservableObject` when supporting older OS versions or Combine-heavy code.

Common mistakes: Combining `@Observable` and `@Published` unnecessarily. Using old property wrappers with new observable models without understanding ownership.

Best practices: Choose one model per feature based on deployment target. Keep migration incremental and test UI invalidation behavior.

Red flag answers: "Observation is just syntax sugar." It changes dependency tracking and removes Combine as a requirement.

Code example:

```
@Observable
final class CounterModel {
    var count = 0
    var isEven: Bool { count.isMultiple(of: 2) }
}

struct CounterScreen: View {
    @State private var model = CounterModel()

    var body: some View {
        VStack {
            Text(model.isEven ? "Even" : "Odd")
            Button("Increment") {
                model.count += 1
            }
        }
    }
}
```

Possible follow-up questions: How do you support iOS 16 and iOS 17? How does property-level tracking affect performance?

[Intermediate] What are `@Environment`, `@AppStorage`, `@SceneStorage`, and `@FocusState`?

Interview answer: They are SwiftUI property wrappers for framework-managed state/context. `@Environment` reads contextual values, `@AppStorage` bridges to UserDefaults, `@SceneStorage` persists scene-specific UI state, and `@FocusState` manages focus.

Deep explanation: Each wrapper has a specific ownership and persistence model. Confusing them leads to hidden dependencies or incorrect persistence.

Why it matters: These wrappers are common in real apps: themes, settings, scene restoration, keyboard focus, and form UX.

How it works: SwiftUI connects property wrappers to environment, storage, or focus systems and invalidates views when values change.

Use cases: `@Environment(\.dismiss)`, dark mode, persisted toggles, selected tab restoration, focused text field.

Common mistakes: Storing sensitive data in `@AppStorage`. Using `@SceneStorage` for business data. Treating focus as model state.

Best practices: Use `@AppStorage` for lightweight non-sensitive preferences only. Use Keychain for secrets. Use `@FocusState` for UI focus, not validation state.

Red flag answers: "AppStorage is a database." It is a convenience wrapper over UserDefaults.

Code example:

```
struct LoginForm: View {
    enum Field {
        case email
        case password
    }

    @AppStorage("hasSeenTips") private var hasSeenTips = false
    @FocusState private var focusedField: Field?

    var body: some View {
        Form {
            TextField("Email", text: .constant(""))
                .focused($focusedField, equals: .email)
            SecureField("Password", text: .constant(""))
                .focused($focusedField, equals: .password)
        }
    }
}
```

Possible follow-up questions: What should not go in UserDefaults? How do you manage focus in a dynamic form?

[Senior] What are source-of-truth mistakes in SwiftUI?

Interview answer: A source-of-truth mistake happens when the same state is duplicated in multiple places or owned by the wrong view/model, causing stale UI, conflicting updates, or reset behavior.

Deep explanation: SwiftUI works best with a single authoritative owner for each piece of mutable state. Other views should derive from it, bind to it, or send actions to mutate it.

Why it matters: Most SwiftUI bugs in forms, navigation, and lists come from duplicated state.

How it works: If parent and child both keep independent copies, SwiftUI cannot know which one is authoritative. Updates may flow one way but not back.

Use cases: Editable forms, sheet presentation, selected row, navigation path, filters, and toggles.

Common mistakes: Initializing child `@State` from parent props. Keeping both `selectedID` and `selectedItem` mutable without synchronization. Making every child own its own view model.

Best practices: Lift shared state to the nearest common owner. Use derived values where possible. Use bindings for simple edits and actions for domain changes.

Red flag answers: "Just add another @State." That often hides the ownership problem.

Code example:

```
struct ParentView: View {
    @State private var name = ""

    var body: some View {
        ChildEditor(name: $name)
    }
}

struct ChildEditor: View {
```

```
    @Binding var name: String

    var body: some View {
        TextField("Name", text: $name)
    }
}
```

Possible follow-up questions: When should child state stay local? How do you model derived state?

SwiftUI Navigation and Presentation

[Intermediate] NavigationStack vs NavigationView?

Interview answer: `NavigationStack` is the modern navigation API introduced in iOS 16. It supports value-driven navigation and `NavigationPath`. `NavigationView` is older and should generally be avoided in new iOS 16+ code.

Deep explanation: `NavigationView` was view-link driven and often awkward for programmatic navigation. `NavigationStack` models navigation as data, which is better for deep linking, restoration, and testability.

Why it matters: Modern interviews expect value-driven navigation patterns, especially for SwiftUI apps.

How it works: `NavigationLink(value:)` appends a value to the navigation stack. `navigationDestination(for:)` maps values to destination views.

Use cases: Hierarchical flows, settings screens, detail navigation, deep links, and state restoration.

Common mistakes: Mixing old and new APIs. Using hidden `NavigationLinks` for programmatic navigation in new code. Storing whole view models in paths.

Best practices: Use route enums or stable IDs in paths. Keep navigation state separate from destination view construction.

Red flag answers: "NavigationStack is just renamed NavigationView." It is a more data-driven API.

Code example:

```
enum Route: Hashable {
    case profile(User.ID)
    case settings
}

struct RootView: View {
    @State private var path: [Route] = []

    var body: some View {
        NavigationStack(path: $path) {
            List {
                Button("Settings") {
                    path.append(.settings)
                }
            }
            .navigationDestination(for: Route.self) { route in
                switch route {
                case .profile(let id):
                    ProfileView(userID: id)
                case .settings:
                    SettingsView()
                }
            }
        }
    }
}
```

Possible follow-up questions: How would you deep link into a NavigationStack? What should go into a NavigationPath?

[Senior] What is NavigationPath and when would you use it?

Interview answer: `NavigationPath` is a type-erased stack of hashable navigation values. Use it when a stack can contain multiple route types or needs dynamic path manipulation.

Deep explanation: For simple apps, `[Route]` is often clearer and more type-safe. `NavigationPath` is useful for heterogeneous stacks, but type erasure makes it easier to lose compile-time clarity.

Why it matters: Navigation state is app state. A robust path model enables deep linking, restoration, and deterministic navigation tests.

How it works: The path stores `Hashable` values. SwiftUI matches each value type to a registered destination.

Use cases: Deep links, restoration, heterogeneous routes, large apps with modular destinations.

Common mistakes: Using `NavigationPath` when a route enum is simpler. Putting non-stable data into the path.

Best practices: Prefer route enums until heterogeneity requires `NavigationPath`. Store IDs/routes, not full views.

Red flag answers: "NavigationPath stores views." It stores navigation data values.

Code example:

```
@State private var path = NavigationPath()

func handleDeepLink(_ link: AppLink) {
    path.removeLast(path.count)
    path.append(Route.profile(link.userID))
    path.append(Route.order(link.orderID))
}
```

Possible follow-up questions: How do you restore a path? How do route enums improve testability?

[Intermediate] How do sheets, fullScreenCover, popovers, alerts, and confirmation dialogs differ?

Interview answer: They are presentation APIs for different UX intents. Sheets are modal partial flows, full-screen covers are immersive modal flows, popovers are contextual floating UI, alerts communicate urgent information, and confirmation dialogs present action choices.

Deep explanation: Presentation should be state-driven. A boolean or optional item controls whether UI is presented. Optional item presentation is often safer because it carries the data needed for the destination.

Why it matters: Bad presentation state causes duplicate sheets, wrong item content, and inconsistent dismissal.

How it works: SwiftUI attaches presentation modifiers to a view. When bound state becomes active, SwiftUI presents. When dismissed, SwiftUI updates the binding.

Use cases: Edit forms, onboarding, destructive confirmations, share menus, contextual actions, and error display.

Common mistakes: Multiple competing sheet booleans. Presenting alerts from stale async results. Keeping presentation state in the wrong child view.

Best practices: Use enum or identifiable item state for complex presentations. Keep presentation state near the owner of the flow.

Red flag answers: "Use alerts for all errors." Many errors are better as inline state, banners, or retry UI.

Code example:

```
struct UserListView: View {
    @State private var selectedUser: User?
    @State private var isConfirmingDelete = false

    var body: some View {
        List(users) { user in
            Button(user.name) {
                selectedUser = user
            }
        }
        .sheet(item: $selectedUser) { user in
            UserDetailView(user: user)
        }
        .confirmationDialog("Delete user?", isPresented: $isConfirmingDelete) {
            Button("Delete", role: .destructive) {}
        }
    }
}
```

Possible follow-up questions: Why is item-based sheet presentation useful? How do you coordinate multiple modals?

[Intermediate] What are toolbars and menus in SwiftUI?

Interview answer: `toolbar` adds actions and content to platform-specific toolbar areas, while `Menu` presents a compact list of actions or options.

Deep explanation: Toolbars adapt to navigation bars, bottom bars, keyboard toolbars, and macOS toolbars. Menus are good for secondary actions that should not dominate the main UI.

Why it matters: Good action placement affects usability and platform feel.

How it works: `ToolbarItem` placement tells SwiftUI where the item belongs. `Menu` builds actions lazily from a view builder.

Use cases: Navigation actions, edit buttons, keyboard Done button, overflow actions, sorting, filtering, and contextual commands.

Common mistakes: Putting primary actions only inside overflow menus. Using toolbar placements that behave differently across platforms without testing.

Best practices: Keep the primary action visible. Use menus for secondary or grouped actions. Test compact and regular size classes.

Red flag answers: "Toolbar is just a top bar." It is a platform-adaptive command surface.

Code example:

```
.toolbar {
    ToolbarItem(placement: .primaryAction) {
        Button("Save") {
            save()
        }
    }

    ToolbarItem(placement: .secondaryAction) {
        Menu("Sort") {
            Button("Newest", action: sortNewest)
            Button("Oldest", action: sortOldest)
        }
    }
}
```

Possible follow-up questions: How do toolbar placements differ? How do you add keyboard toolbar actions?

SwiftUI Layout

[Beginner] How do HStack, VStack, and ZStack work?

Interview answer: `HStack` lays children horizontally, `VStack` vertically, and `ZStack` overlays children along the z-axis. They propose sizes to children, collect child sizes, and position them based on alignment and spacing.

Deep explanation: SwiftUI layout is a proposal/response system. Parents propose a size. Children choose their size. Parents place children. Stacks are basic layout containers implementing this negotiation.

Why it matters: Understanding layout negotiation prevents hacks with fixed frames and GeometryReader.

How it works: Stacks measure children in order, account for spacing, alignment, layout priority, and flexible frames, then choose final size and placement.

Use cases: Rows, columns, overlays, badges, cards, form groups, and simple screen composition.

Common mistakes: Using fixed frames to force layout. Assuming `Spacer` has intrinsic size. Misunderstanding alignment vs padding.

Best practices: Prefer flexible layout. Use alignment and layout priority before hard-coded sizes. Test Dynamic Type.

Red flag answers: "Stacks are just Auto Layout stack views." They are declarative layout containers with SwiftUI's proposal system.

Code example:

```
HStack(alignment: .firstTextBaseline, spacing: 12) {
    Image(systemName: "person.crop.circle")
    VStack(alignment: .leading) {
        Text("Ari")
        Text("iOS Engineer").font(.subheadline)
    }
    Spacer()
}
```

Possible follow-up questions: What is layout priority? How does Spacer work?

[Intermediate] List, Grid, LazyVStack, and Lazy grids: how do you choose?

Interview answer: Use `List` for platform-native rows with built-in editing, swipe actions, sections, accessibility, and cell reuse behavior. Use `ScrollView` plus lazy stacks/grids for custom layouts. Use `Grid` for two-dimensional non-lazy layout; lazy grids for large scrollable grids.

Deep explanation: `List` is opinionated and platform-adaptive. Lazy containers create child views as needed, but they are not a complete replacement for all `List` features.

Why it matters: Wrong container choice causes styling fights, performance issues, or missing platform behavior.

How it works: Lazy stacks/grids defer child creation. `List` has additional platform-backed behavior and row management. `Grid` lays out all cells and is better for finite grids.

Use cases: Settings: `List`. Pinterest-like grid: `LazyVGrid`. Custom feed: `ScrollView` + `LazyVStack`. Dashboard matrix: `Grid`.

Common mistakes: Using `VStack` inside `ScrollView` for thousands of rows. Fighting `List` styling for highly custom layouts. Assuming lazy means free.

Best practices: Choose based on UX and data size. Profile large lists. Keep row views cheap.

Red flag answers: "List is always faster." It depends on content, platform, modifiers, and behavior needed.

Code example:

```
let columns = [
    GridItem(.adaptive(minimum: 140), spacing: 12)
```

```
]

ScrollView {
    LazyVGrid(columns: columns, spacing: 12) {
        ForEach(products) { product in
            ProductTile(product: product)
        }
    }
    .padding()
}
```

Possible follow-up questions: How do lazy containers affect `onAppear`? How would you debug a slow list?

[Intermediate] What are alignment guides?

Interview answer: Alignment guides let a child tell its parent where a specific alignment line should be, enabling custom alignment beyond default leading, center, trailing, or baseline behavior.

Deep explanation: SwiftUI alignment is not only about a parent's setting. Children can customize alignment positions using guide closures.

Why it matters: Alignment guides solve precise layout problems without hard-coded offsets.

How it works: The guide closure receives view dimensions and returns the coordinate SwiftUI should use for the selected alignment.

Use cases: Aligning labels and icons, custom baselines, timeline layouts, badges, and mixed-size content.

Common mistakes: Using offsets for alignment. Offsets move drawing but often do not affect layout size or parent placement as expected.

Best practices: Use alignment guides when the relationship is layout-level. Use offset for visual effects, not structural alignment.

Red flag answers: "Alignment guide is just padding." It participates in parent alignment calculation.

Code example:

```
HStack(alignment: .top) {
    Image(systemName: "exclamationmark.circle")
        .alignmentGuide(.top) { dimensions in
            dimensions[VerticalAlignment.center]
        }

    Text("Important message that may wrap onto multiple lines.")
}
```

Possible follow-up questions: How are custom alignments defined? Offset vs alignment guide?

[Intermediate] What is GeometryReader and when should you avoid it?

Interview answer: `GeometryReader` gives access to parent-proposed size and coordinate-space frames. Avoid using it as a default layout tool because it expands to available space and can make layouts harder to reason about.

Deep explanation: GeometryReader is useful for measurement and coordinate calculations, not for ordinary spacing. Many uses are better solved with stacks, `ViewThatFits`, custom `Layout`, or container-relative APIs.

Why it matters: Overusing GeometryReader causes unexpected full-size containers, layout loops, and fragile device-specific code.

How it works: GeometryReader receives a `GeometryProxy`. Its content can query size, safe area, and frames.

Use cases: Scroll offset, responsive drawing, custom effects, anchor resolution, and container measurements.

Common mistakes: Using screen width for all responsive layout. Reading geometry and writing state every layout pass. Assuming proxy size is final child size.

Best practices: Prefer modern layout tools first. If measuring, isolate GeometryReader in a background or overlay when possible.

Red flag answers: "Use GeometryReader whenever layout is hard." It often makes layout harder.

Code example:

```
Text("Progress")
    .background {
        GeometryReader { proxy in
            Color.clear
                .preference(key: SizePreferenceKey.self, value: proxy.size)
        }
    }
```

Possible follow-up questions: Why does GeometryReader expand? What alternatives exist in iOS 16/17?

[Intermediate] How do safe area and keyboard avoidance work in SwiftUI?

Interview answer: SwiftUI automatically respects safe areas for most containers. You can customize with `safeAreaInset`, `ignoresSafeArea`, and scroll/form behavior. Keyboard avoidance is often automatic in scrollable containers, but custom layouts may need focus-aware scrolling or safe-area insets.

Deep explanation: Safe areas represent system UI and device constraints. Keyboard appearance changes available space. SwiftUI containers adapt differently depending on whether content is scrollable, fixed, or ignoring safe areas.

Why it matters: Poor safe area handling causes clipped buttons, hidden text fields, and inaccessible bottom actions.

How it works: SwiftUI proposes layout within safe bounds unless told otherwise. `safeAreaInset` reserves space for custom bars. Keyboard changes layout environment.

Use cases: Bottom call-to-action bars, chat input bars, full-bleed images, login forms, and custom tab bars.

Common mistakes: Using `.ignoresSafeArea()` too broadly. Hard-coding bottom padding. Not testing Dynamic Type and keyboard.

Best practices: Use `safeAreaInset` for persistent bottom controls. Use `ScrollViewReader` or focus management for complex forms.

Red flag answers: "Just add 34 points of padding." Safe areas vary by device, orientation, and context.

Code example:

```
ScrollView {
    FormContent()
}
.safeAreaInset(edge: .bottom) {
    Button("Continue", action: submit)
        .buttonStyle(.borderedProminent)
        .padding()
        .background(.bar)
}
```

Possible follow-up questions: When should a background ignore safe area but content not? How do you keep a focused text field visible?

[Senior] What is the Layout protocol?

Interview answer: The `Layout` protocol lets you build custom SwiftUI layout containers by measuring subviews and placing them yourself. It is available from iOS 16.

Deep explanation: Before `Layout`, custom layout often required GeometryReader/preferences. `Layout` gives a first-class API for implementing layout negotiation.

Why it matters: Senior SwiftUI engineers should know when to move from layout hacks to explicit custom layout.

How it works: A layout implements `sizeThatFits(proposal:subviews:cache:)` and `placeSubviews(in:proposal:subviews:cache:)`.

Use cases: Flow layouts, custom grids, tag wrapping, radial layouts, adaptive controls, and performance-sensitive custom containers.

Common mistakes: Not respecting proposed size. Measuring subviews multiple times unnecessarily. Ignoring spacing and layout direction.

Best practices: Keep layout deterministic. Use cache for expensive calculations. Test with Dynamic Type and right-to-left languages.

Red flag answers: "Use GeometryReader for all custom layouts." `Layout` is usually cleaner for reusable custom containers.

Code example:

```
struct EqualWidthHStack: Layout {
    func sizeThatFits(
        proposal: ProposedViewSize,
        subviews: Subviews,
        cache: inout ()
    ) -> CGSize {
        let sizes = subviews.map { $0.sizeThatFits(.unspecified) }
        let maxWidth = sizes.map(\.width).max() ?? 0
        let maxHeight = sizes.map(\.height).max() ?? 0
        return CGSize(width: maxWidth * CGFloat(subviews.count), height: maxHeight)
    }

    func placeSubviews(
        in bounds: CGRect,
        proposal: ProposedViewSize,
        subviews: Subviews,
        cache: inout ()
    ) {
        let width = bounds.width / CGFloat(max(subviews.count, 1))
        for (index, subview) in subviews.enumerated() {
            subview.place(
                at: CGPoint(x: bounds.minX + CGFloat(index) * width, y: bounds.minY),
                proposal: ProposedViewSize(width: width, height: bounds.height)
            )
        }
    }
}
```

Possible follow-up questions: How does `Layout` compare with alignment guides? What should go in layout cache?

[Intermediate] What are `ViewThatFits` and `containerRelativeFrame`?

Interview answer: `ViewThatFits` chooses the first child that fits the proposed space. `containerRelativeFrame` sizes a view relative to a container, useful in scroll views and adaptive layouts.

Deep explanation: These APIs reduce manual geometry calculations. `ViewThatFits` supports graceful fallback UI. `containerRelativeFrame` lets child views size themselves based on container dimensions.

Why it matters: They help build responsive SwiftUI without hard-coded device checks.

How it works: `ViewThatFits` evaluates children in order against the proposal. `containerRelativeFrame` derives dimensions from the nearest relevant container.

Use cases: Compact vs expanded labels, responsive cards, carousel items, adaptive buttons, and dynamic type fallback.

Common mistakes: Putting expensive alternatives in `ViewThatFits`. Using these APIs when simple layout priority is enough.

Best practices: Order `ViewThatFits` from preferred to fallback. Keep alternatives semantically equivalent.

Red flag answers: "Responsive design means checking screen width." Prefer layout-driven adaptation.

Code example:

```
ViewThatFits {
    Label("Download Invoice", systemImage: "arrow.down.doc")
    Image(systemName: "arrow.down.doc")
}
```

Possible follow-up questions: How does Dynamic Type affect `ViewThatFits`? When would you use layout priority instead?

[Intermediate] What is `ScrollViewReader`?

Interview answer: `ScrollViewReader` gives a proxy that can programmatically scroll to child views with stable IDs.

Deep explanation: SwiftUI scrolling is usually user-driven. When app logic needs to reveal a selected row, focused field, or new message, `ScrollViewReader` bridges state to scroll position.

Why it matters: Programmatic scrolling is common in chat, forms, deep links, validation errors, and onboarding.

How it works: Child views must have `.id(...)`. The proxy calls `scrollTo(id, anchor:)`.

Use cases: Chat autoscroll, jump to validation error, index navigation, focused field reveal.

Common mistakes: Using unstable IDs. Calling `scrollTo` before the target exists. Overusing it to fight layout issues.

Best practices: Use stable IDs and trigger scrolling after data/state updates. Consider animation carefully.

Red flag answers: "ScrollViewReader can scroll to any view." It scrolls to views with known IDs in its scroll content.

Code example:

```
ScrollViewReader { proxy in
    ScrollView {
        ForEach(messages) { message in
            MessageRow(message: message)
                .id(message.id)
        }
    }
    .onChange(of: messages.last?.id) { _, id in
        guard let id else { return }
        withAnimation {
            proxy.scrollTo(id, anchor: .bottom)
        }
    }
}
```

Possible follow-up questions: How do you avoid fighting the user's scroll position in chat? What happens if IDs change?

[Senior] What Dynamic Type layout issues do you watch for?

Interview answer: I watch for clipped text, fixed-height containers, horizontal-only layouts that cannot wrap, icon/text overlap, inaccessible hit targets, and custom controls that ignore font scaling.

Deep explanation: Dynamic Type changes intrinsic sizes. Layouts must adapt by wrapping, stacking, scrolling, or showing alternative UI.

Why it matters: Accessibility is part of production quality and often legally/business critical.

How it works: Text sizes come from environment dynamic type size. SwiftUI relayouts views based on changed intrinsic sizes.

Use cases: Forms, settings, buttons, tabular content, cards, and dense dashboards.

Common mistakes: Fixed heights. `lineLimit(1)` on important content. Small tap targets. Text inside decorative containers with no room to grow.

Best practices: Test large accessibility sizes. Use flexible stacks, `ViewThatFits`, scroll containers, and minimum hit targets.

Red flag answers: "We can just reduce the font size to fit." Users chose larger text for a reason.

Code example:

```
ViewThatFits(in: .horizontal) {
    HStack {
        Text(title)
        Spacer()
        Text(value)
    }

    VStack(alignment: .leading) {
        Text(title)
        Text(value).bold()
    }
}
```

Possible follow-up questions: How do you test Dynamic Type? How would you handle complex tables?

SwiftUI Lists and Collections

[Beginner] What is `ForEach` and why do stable IDs matter?

Interview answer: `ForEach` creates views for a collection. Stable IDs let SwiftUI match data elements across updates so it can preserve row state, animate changes, and diff correctly.

Deep explanation: SwiftUI needs to know whether an item is the same logical item after insertion, deletion, sorting, or update. Stable identity should come from the domain model, not transient position.

Why it matters: Bad IDs cause wrong row updates, broken animations, lost focus, and corrupted local row state.

How it works: `ForEach` uses `Identifiable` or an explicit `id` key path. SwiftUI reconciles child views by ID.

Use cases: Lists, grids, menus, forms, and dynamic sections.

Common mistakes: Using `\.self` for mutable values. Using indices as IDs when the array can reorder. Creating `UUID()` in `body`.

Best practices: Use stable model identifiers. If data has no ID, create one when data is loaded, not during rendering.

Red flag answers: "Use `id: \.self` whenever possible." It is safe only for stable, unique, value-identity items.

Code example:

```
ForEach(viewModel.messages) { message in
    MessageRow(message: message)
}
```

Possible follow-up questions: When is `\.self` acceptable? Why are index IDs risky?

[Intermediate] What is SwiftUI diffing in lists?

Interview answer: Diffing is how SwiftUI determines inserts, deletes, moves, and updates between old and new collection states using identity and structure.

Deep explanation: SwiftUI does not mutate row views manually. It receives a new list description and reconciles it against the old one. Stable IDs make this reconciliation meaningful.

Why it matters: Correct diffing enables smooth animations and state preservation. Bad diffing creates visual bugs.

How it works: Framework internals compare old/new identity and view structure. The exact algorithm is private, but stable IDs and consistent view types are key inputs.

Use cases: Animated insertions, deletion, drag reorder, filtering, sorting, and live updates.

Common mistakes: Changing IDs when content changes. Using `.id()` to force reloads. Wrapping rows in `AnyView` unnecessarily.

Best practices: Keep identity separate from display content. Use Equatable models where helpful. Avoid row side effects based on body calls.

Red flag answers: "Diffing compares every property deeply." Identity and view structure are central; do not rely on deep comparison.

Code example:

```
struct Todo: Identifiable, Equatable {
    let id: UUID
    var title: String
    var isDone: Bool
}
```

Possible follow-up questions: How does `.id()` affect diffing? How do animations use identity?

[Intermediate] How do swipe actions, pull to refresh, search, and sections work?

Interview answer: SwiftUI provides list modifiers for common platform behavior: `swipeActions`, `refreshable`, `searchable`, and `Section`. They work best with `List` and state-driven data.

Deep explanation: These APIs integrate with platform conventions, accessibility, and system gestures. They are preferable to custom reimplementations unless design requirements are truly custom.

Why it matters: Interviewers look for platform-native judgment, not just custom UI ability.

How it works: Modifiers attach behavior to rows or containers. `refreshable` runs async work. `searchable` binds search text. `Section` groups content semantically.

Use cases: Mail-like lists, settings, inboxes, searchable catalogs, grouped forms, refreshable feeds.

Common mistakes: Doing destructive work without confirmation. Not handling refresh cancellation/errors. Filtering data destructively instead of deriving filtered state.

Best practices: Keep search query as state and derive results. Make refresh idempotent. Use roles for destructive actions.

Red flag answers: "Build custom swipe actions for full control first." Native behavior is usually better unless requirements demand custom interaction.

Code example:

```
List {
    Section("Unread") {
        ForEach(unreadMessages) { message in
            MessageRow(message: message)
                .swipeActions {
                    Button("Archive") {
                        archive(message)
                    }
                    .tint(.blue)
                }
        }
    }
}
.searchable(text: $query)
.refreshable {
    await viewModel.reload()
}
```

Possible follow-up questions: How do you avoid duplicate refresh calls? How do you test searchable filtering?

SwiftUI Animation and Gestures

[Intermediate] Implicit vs explicit animation?

Interview answer: Implicit animation attaches animation to state changes through `.animation(_:value:)`. Explicit animation wraps the state mutation in `withAnimation`.

Deep explanation: SwiftUI animations are state transitions. You do not animate properties directly; you change state and SwiftUI interpolates animatable values between old and new UI descriptions.

Why it matters: Understanding this prevents misplaced animation modifiers and unexpected global animations.

How it works: SwiftUI uses transactions to carry animation context during state updates. Animatable values interpolate over time.

Use cases: Expanding rows, toggles, matched transitions, onboarding, loading states, and gesture-driven movement.

Common mistakes: Using deprecated broad `.animation(_:)`. Animating too much by attaching animation high in the tree. Expecting non-animatable changes to animate.

Best practices: Use `.animation(_:value:)` scoped to specific values. Use `withAnimation` around intentional mutations.

Red flag answers: "Animation changes the view directly." State changes; SwiftUI animates the transition.

Code example:

```
@State private var isExpanded = false

VStack {
    Button("Toggle") {
        withAnimation(.spring()) {
            isExpanded.toggle()
        }
    }

    if isExpanded {
        DetailsView()
            .transition(.opacity.combined(with: .move(edge: .top)))
    }
}
```

Possible follow-up questions: What is a transaction? Why does an animation affect unexpected children?

[Senior] What are Transactions and transitions?

Interview answer: A transaction carries animation and update context through a SwiftUI state change. A transition defines how a view is inserted or removed from the hierarchy.

Deep explanation: Animations describe interpolation. Transitions describe enter/exit behavior when identity changes and a view appears or disappears.

Why it matters: Senior SwiftUI work often involves debugging why a transition does not run or why an animation leaks into unrelated updates.

How it works: `withAnimation` creates a transaction with animation. `.transaction` can customize or disable animation in a subtree. A transition applies when SwiftUI detects insertion/removal of a view identity.

Use cases: Toasts, expandable sections, overlays, route changes, conditional controls, and disabling animations for specific updates.

Common mistakes: Adding `.transition` without animating the state change. Expecting transition to run when a view is merely updated, not inserted/removed.

Best practices: Pair transitions with stable identity and animated state mutations. Scope transaction changes narrowly.

Red flag answers: "Transition is just animation." Transition is specifically insertion/removal behavior.

Code example:

```
content
    .transaction { transaction in
        if reduceMotionEnabled {
            transaction.animation = nil
        }
    }
```

Possible follow-up questions: Why does a transition not fire? How do you disable animation in one subtree?

[Senior] What is `matchedGeometryEffect`?

Interview answer: `matchedGeometryEffect` animates geometry changes between two views that represent the same logical element in the same namespace.

Deep explanation: It creates a visual continuity effect between different view positions, sizes, or containers. The views are not literally the same instance; SwiftUI matches their geometry by namespace and ID.

Why it matters: It enables polished transitions like grid-to-detail hero animations.

How it works: SwiftUI records geometry for matched IDs and interpolates between source and destination frames during an animated state change.

Use cases: Hero image transitions, tab indicators, card expansion, selected item highlights.

Common mistakes: Using unstable IDs. Having multiple active sources. Expecting it to work across unrelated view hierarchies without shared namespace/state.

Best practices: Keep matched elements simple. Ensure one clear source/destination pair. Test interruptible transitions.

Red flag answers: "It moves the actual view object." It animates geometry between matched descriptions.

Code example:

```
@Namespace private var namespace
@State private var selected: Photo?

// In grid and detail:
Image(uiImage: photo.image)
    .matchedGeometryEffect(id: photo.id, in: namespace)
```

Possible follow-up questions: How do you handle multiple matched elements? Why can layout changes break the effect?

[Intermediate] How do SwiftUI gestures compose?

Interview answer: Gestures are values attached to views. You can compose them with `simultaneousGesture`, `highPriorityGesture`, `sequenced`, and `exclusively` to control recognition behavior.

Deep explanation: Gesture composition determines whether gestures compete, run together, or wait for each other. This matters in scroll views, draggable cards, long press menus, and custom controls.

Why it matters: Gesture conflicts are common in production SwiftUI.

How it works: Gesture values produce callbacks like `onChanged`, `updating`, and `onEnded`. SwiftUI coordinates recognition based on composition.

Use cases: Drag-to-dismiss, long-press selection, pinch-to-zoom, custom sliders, and reorder interactions.

Common mistakes: Breaking scroll gestures with a high-priority drag. Updating permanent state during every drag instead of using gesture state.

Best practices: Use `@GestureState` for transient gesture values. Keep gesture interactions accessible and provide non-gesture alternatives for important actions.

Red flag answers: "Attach a DragGesture and it will just work." Gesture priority and container interactions matter.

Code example:

```
@GestureState private var dragOffset: CGSize = .zero
@State private var position: CGSize = .zero

let drag = DragGesture()
    .updating($dragOffset) { value, state, _ in
        state = value.translation
    }
    .onEnded { value in
        position.width += value.translation.width
        position.height += value.translation.height
    }

Circle()
    .offset(x: position.width + dragOffset.width, y: position.height + dragOffset.height)
    .gesture(drag)
```

Possible follow-up questions: When use `@GestureState`? How do you combine long press and drag?

SwiftUI Lifecycle and Tasks

[Intermediate] What are `.onAppear` and `.onDisappear`?

Interview answer: They run closures when SwiftUI considers a view appeared or disappeared. They are not equivalent to UIKit view controller lifecycle and can run multiple times.

Deep explanation: SwiftUI views are value descriptions and can appear/disappear due to scrolling, conditional rendering, navigation, or identity changes. Lifecycle callbacks are useful but should not be treated as one-time constructors/destructors.

Why it matters: Incorrect assumptions cause duplicate network calls, repeated analytics, and missing cleanup.

How it works: SwiftUI invokes callbacks based on view graph insertion/removal and platform visibility decisions.

Use cases: Light analytics, starting/stopping simple work, row visibility, playback pause/resume.

Common mistakes: Starting non-idempotent network calls in `onAppear` without guarding. Assuming `onDisappear` always means deallocation.

Best practices: Prefer `.task` for async loading. Make appearance work idempotent. Keep lifecycle side effects small.

Red flag answers: "onAppear is like viewDidLoad." It can fire multiple times.

Code example:

```
.onAppear {
    analytics.track("profile_appeared")
}
```

Possible follow-up questions: Why can `onAppear` fire in a lazy list? How do you avoid duplicate loads?

[Intermediate] What is `.task` and `.task(id:)`?

Interview answer: `.task` starts async work tied to a view's lifecycle. `.task(id:)` restarts the task when the ID changes and cancels the previous task.

Deep explanation: This is the preferred SwiftUI mechanism for view-scoped async work. It gives cancellation for free when the view disappears or the ID changes.

Why it matters: It avoids manual task storage for many screen-loading and search scenarios.

How it works: SwiftUI creates a task when the view appears. It cancels the task when the view disappears or when the ID changes for `.task(id:)`.

Use cases: Initial data load, search query changes, refresh on selected account change, async setup.

Common mistakes: Ignoring cancellation. Doing non-view-scoped work in `.task`. Triggering duplicate tasks through unstable identity.

Best practices: Use `.task(id:)` for state-dependent async work. Handle `CancellationError` as neutral.

Red flag answers: "`.task` runs once." It runs according to view lifecycle and identity.

Code example:

```
struct SearchResultsView: View {
    let query: String
    @State private var results: [Item] = []

    var body: some View {
        List(results) { Text($0.title) }
            .task(id: query) {
                results = (try? await SearchService().search(query)) ?? []
            }
    }
}
```

Possible follow-up questions: How does `.task(id:)` cancel stale work? What makes an ID unstable?

[Senior] How do you avoid duplicate network calls in SwiftUI?

Interview answer: Make loading idempotent, tie work to stable view identity, use `.task(id:)` carefully, cache or deduplicate in the data layer, and avoid starting work in `body` or unstable row lifecycle callbacks.

Deep explanation: SwiftUI may recreate views and call lifecycle hooks multiple times. Network deduplication should not depend only on view lifecycle because scrolling, navigation, and state changes can retrigger views.

Why it matters: Duplicate calls waste bandwidth, drain battery, create inconsistent state, and can double-submit user actions.

How it works: State changes cause recomputation. Lifecycle-triggered work may restart. Data services can track in-flight requests and return existing results.

Use cases: Feed loading, image loading, detail screens, search, checkout submit, and pagination.

Common mistakes: Calling `Task { await load() }` in `body`. Using `onAppear` for row fetches without caching. Not cancelling stale searches.

Best practices: Use view models/services to own loading state. Protect submit actions with `isSubmitting`. Cache in-flight image requests.

Red flag answers: "Use a boolean in every view." Local booleans help but do not solve shared data or in-flight deduplication.

Code example:

```
@MainActor
@Observable
final class DetailModel {
    enum LoadState {
        case idle
        case loading
        case loaded(Item)
        case failed(Error)
    }

    private(set) var state: LoadState = .idle

    func load(id: Item.ID) async {
        guard case .idle = state else { return }
        state = .loading
        do {
            state = .loaded(try await ItemService().item(id: id))
        } catch {
            state = .failed(error)
        }
    }
}
```

Possible follow-up questions: Where should request deduplication live? How do you handle pull-to-refresh differently from initial load?

SwiftUI Performance and Debugging

[Intermediate] Why can `body` be recomputed often, and how do you handle it?

Interview answer: `body` can be recomputed whenever dependent state changes or SwiftUI needs a fresh description. Keep body cheap, deterministic, and free of side effects.

Deep explanation: Body recomputation is normal, not automatically a performance bug. The issue is expensive work inside body or broad state invalidation.

Why it matters: Heavy body work causes scrolling stutter and battery drain.

How it works: State invalidation triggers body reevaluation. SwiftUI then reconciles the resulting view tree.

Use cases: Lists, dynamic forms, animations, observable models, and frequently changing state.

Common mistakes: Sorting, filtering, date formatting, image decoding, or network calls inside body. Logging body calls and assuming full redraw.

Best practices: Precompute expensive values. Split views. Use stable identity. Profile before adding optimizations.

Red flag answers: "Prevent body from running." You generally design body to be safe to run often.

Code example:

```
struct FeedView: View {
    let sections: [FeedSection] // Already prepared by model/view model.

    var body: some View {
        List(sections) { section in
            Section(section.title) {
                ForEach(section.items) { item in
                    FeedRow(item: item)
                }
            }
        }
    }
}
```

Possible follow-up questions: How do you identify expensive body work? How does Observation help?

[Senior] How do you diagnose slow SwiftUI lists?

Interview answer: Check row identity, row body cost, image loading/decoding, excessive state invalidation, `onAppear` work, type erasure, layout complexity, and main-thread blocking. Use Instruments and targeted simplification.

Deep explanation: Slow lists are usually a combination of unstable IDs, expensive row construction, asynchronous image churn, and too much state changing at the list level.

Why it matters: Lists are among the most common high-performance surfaces in iOS apps.

How it works: Scrolling needs rapid layout and rendering. Any main-thread work, repeated decoding, or large invalidation can miss frames.

Use cases: Feeds, chats, search results, settings, marketplaces, and media galleries.

Common mistakes: Starting network calls directly in every row. Recomputing formatters in row body. Using index IDs. Applying heavy shadows/masks to many rows.

Best practices: Cache images. Preformat data. Keep rows small. Use stable IDs. Profile Time Profiler, SwiftUI instruments, Allocations, and Main Thread Checker.

Red flag answers: "Replace everything with UIKit." Sometimes needed, but first identify the actual bottleneck.

Code example:

```
struct MessageRowModel: Identifiable, Equatable {
    let id: UUID
    let title: String
    let subtitle: String
    let formattedDate: String
}
```

Possible follow-up questions: How does image loading affect scrolling? What Instruments template would you use?

[Intermediate] How do you debug SwiftUI layout and rendering issues?

Interview answer: Isolate the view, add temporary borders/backgrounds, inspect sizes with GeometryReader carefully, test Dynamic Type and device sizes, and simplify modifiers until the issue is clear.

Deep explanation: SwiftUI layout bugs often come from modifier order, fixed frames, GeometryReader expansion, safe-area handling, or conflicting priorities.

Why it matters: Debugging layout by guesswork leads to fragile device-specific fixes.

How it works: SwiftUI layout is a size proposal and response system. Visual debugging helps reveal which parent or modifier is changing size.

Use cases: Clipping, unexpected spacing, invisible views, overlays, scroll problems, and adaptive layout.

Common mistakes: Adding random frames/padding. Using `.fixedSize()` without understanding consequences. Ignoring accessibility text sizes.

Best practices: Work from parent to child. Verify modifier order. Use previews with multiple sizes and dynamic type.

Red flag answers: "SwiftUI layout is unpredictable." It is deterministic, but the rules differ from Auto Layout.

Code example:

```
extension View {
    func debugBorder(_ color: Color = .red) -> some View {
        overlay(Rectangle().stroke(color, lineWidth: 1))
    }
}
```

Possible follow-up questions: How does `.fixedSize()` work? Why does modifier order affect size?

[Senior] How can `.id()` fix or break behavior?

Interview answer: `.id()` changes a view's explicit identity. It can force SwiftUI to treat a view as new, which can reset state or trigger transitions. It can fix stale identity problems but break state preservation if misused.

Deep explanation: Identity controls lifecycle. Changing ID destroys old state and creates new state. That is useful for resetting forms or reloading a view, but harmful if accidental.

Why it matters: Misused `.id()` causes disappearing focus, scroll resets, task restarts, animation glitches, and lost local state.

How it works: SwiftUI includes explicit ID in reconciliation. A different ID means different view identity.

Use cases: Resetting a form after switching accounts, scroll targets, forcing a new web view, restarting an animation intentionally.

Common mistakes: Using `.id(UUID())` in body. Applying `.id()` high in the tree to "refresh" everything. Changing IDs based on display text.

Best practices: Use stable IDs for identity and deliberate changed IDs for resets. Keep `.id()` scope narrow.

Red flag answers: "`.id()` just labels a view." It affects lifecycle and state.

Code example:

```
ProfileEditor(userID: selectedUserID)
    .id(selectedUserID) // Intentionally reset editor state when user changes.
```

Possible follow-up questions: How does `.id()` affect `.task`? How does it interact with transitions?

UIKit and SwiftUI Interoperability

[Intermediate] What are `UIViewRepresentable` and `UIViewControllerRepresentable`?

Interview answer: They wrap UIKit views or view controllers so they can be used in SwiftUI.

Deep explanation: SwiftUI does not replace every UIKit control. Representables bridge APIs such as maps, web views, text views, camera controllers, or custom legacy components.

Why it matters: Most real apps are hybrid. Senior engineers need clean interop patterns.

How it works: You implement creation and update methods. SwiftUI creates the UIKit object and calls update when SwiftUI state changes. A Coordinator can bridge delegates and callbacks.

Use cases: `WKWebView`, `MKMapView`, `UITextView`, camera/image picker, custom UIKit controls, legacy modules.

Common mistakes: Creating the UIKit view repeatedly in update. Not syncing state both ways. Forgetting coordinator lifetime and delegate retention.

Best practices: Keep `make` for construction, `update` for state synchronization. Use Coordinator for delegate callbacks. Avoid storing source of truth inside UIKit view when SwiftUI owns it.

Red flag answers: "Just put UIKit code in SwiftUI body." The representable lifecycle exists for a reason.

Code example:

```
struct WebView: UIViewRepresentable {
    let url: URL

    func makeUIView(context: Context) -> WKWebView {
        WKWebView()
    }

    func updateUIView(_ webView: WKWebView, context: Context) {
        if webView.url != url {
            webView.load(URLRequest(url: url))
        }
    }
}
```

Possible follow-up questions: When is `makeUIView` called? How do you avoid reload loops?

[Senior] What is the Coordinator pattern in SwiftUI representables?

Interview answer: A Coordinator is an object that bridges UIKit delegate/data source callbacks to SwiftUI state and actions.

Deep explanation: UIKit often communicates through delegates, target-action, or notifications. SwiftUI representables are structs, so a stable reference object is needed for delegate identity and callback coordination.

Why it matters: Without a coordinator, data flow between UIKit and SwiftUI becomes fragile or leaks.

How it works: `makeCoordinator()` creates a coordinator retained by SwiftUI for the representable's lifecycle. The context passes it to `make` and `update`.

Use cases: Text view delegate, map annotations, scroll view offset, image picker result, web navigation delegate.

Common mistakes: Coordinator strongly retaining parent objects unnecessarily. Updating SwiftUI state recursively during UIKit updates.

Best practices: Use coordinator as a bridge, not a dumping ground. Make feedback loops explicit and guard repeated updates.

Red flag answers: "Coordinator is just optional boilerplate." It is essential for delegate-heavy UIKit components.

Code example:

```
struct TextView: UIViewRepresentable {
    @Binding var text: String

    func makeCoordinator() -> Coordinator {
        Coordinator(text: $text)
    }

    func makeUIView(context: Context) -> UITextView {
        let view = UITextView()
        view.delegate = context.coordinator
        return view
    }

    func updateUIView(_ view: UITextView, context: Context) {
        if view.text != text {
            view.text = text
        }
    }

    final class Coordinator: NSObject, UITextViewDelegate {
        var text: Binding<String>

        init(text: Binding<String>) {
            self.text = text
        }

        func textViewDidChange(_ textView: UITextView) {
            text.wrappedValue = textView.text
        }
    }
}
```

Possible follow-up questions: How do you prevent update loops? Who owns the coordinator?

[Intermediate] What is `UIHostingController`?

Interview answer: `UIHostingController` hosts SwiftUI content inside UIKit.

Deep explanation: It is the bridge for adopting SwiftUI in existing UIKit apps. UIKit owns the hosting controller lifecycle; SwiftUI renders the root view inside it.

Why it matters: Most production apps migrate incrementally, not all at once.

How it works: Create `UIHostingController(rootView:)` and present, push, embed, or assign it like any view controller.

Use cases: New SwiftUI screen in UIKit navigation, SwiftUI cells/headers, settings screens, feature modules.

Common mistakes: Not managing sizing when embedding as child. Replacing root view to push data instead of using observable state. Ignoring environment injection.

Best practices: Define clear data flow from UIKit to SwiftUI. Use hosting configuration for cells when appropriate on modern iOS.

Red flag answers: "You cannot mix UIKit and SwiftUI." You can, but lifecycle boundaries matter.

Code example:

```
let screen = ProfileView(userID: userID)
let hostingController = UIHostingController(rootView: screen)
navigationController?.pushViewController(hostingController, animated: true)
```

Possible follow-up questions: How do you pass UIKit callbacks into SwiftUI? How do you embed a hosting controller as a child?

SwiftUI Accessibility

[Intermediate] What accessibility basics should every SwiftUI view handle?

Interview answer: Provide meaningful labels, values, hints when needed, traits, actions, logical VoiceOver order, Dynamic Type support, Reduce Motion alternatives, and sufficient color contrast.

Deep explanation: Accessibility is not only labels. It includes navigation order, semantics, motion sensitivity, hit targets, contrast, text scaling, and alternative actions for gestures.

Why it matters: Accessible apps are higher quality for everyone and required by many organizations.

How it works: SwiftUI exposes accessibility modifiers that map view semantics to assistive technologies.

Use cases: Custom controls, icon-only buttons, charts, gestures, dynamic content, and forms.

Common mistakes: Labeling decorative images. Using color alone to convey status. Adding verbose hints everywhere. Breaking VoiceOver order with visual-only layout.

Best practices: Test with VoiceOver. Use native controls when possible. Make important icon buttons explicit.

Red flag answers: "SwiftUI handles accessibility automatically." It helps, but custom UI still needs semantic work.

Code example:

```
Button {
    favorite.toggle()
} label: {
    Image(systemName: favorite ? "heart.fill" : "heart")
}
.accessibilityLabel(favorite ? "Remove from favorites" : "Add to favorites")
.accessibilityHint("Updates this item in your favorites list.")
```

Possible follow-up questions: How do you test VoiceOver order? How do you support Reduce Motion?

[Senior] How do you make custom gestures accessible?

Interview answer: Provide an equivalent accessible action, clear labels/values, and avoid making essential functionality only available through complex gestures.

Deep explanation: A drag, pinch, or long press may be unavailable or difficult for VoiceOver, Switch Control, or motor-impaired users. Accessibility actions expose the same capability semantically.

Why it matters: Gesture-only UI can block users from core workflows.

How it works: SwiftUI's `.accessibilityAction` adds named actions that assistive technologies can invoke.

Use cases: Swipe-to-delete alternatives, reorder controls, rating sliders, custom media scrubbers, map controls.

Common mistakes: Assuming swipe actions are enough. No visible fallback. Missing accessibility value updates.

Best practices: Use native controls where possible. For custom controls, expose label, value, adjustable actions, and named actions.

Red flag answers: "Users can just use the gesture." Not all users can.

Code example:

```
RatingView(value: rating)
    .accessibilityLabel("Rating")
    .accessibilityValue("\(rating) out of 5")
    .accessibilityAdjustableAction { direction in
        switch direction {
        case .increment:
            rating = min(rating + 1, 5)
        case .decrement:
            rating = max(rating - 1, 0)
        @unknown default:
```

```
            break
        }
    }
```

Possible follow-up questions: What is an adjustable accessibility action? How do you test Switch Control?

SwiftUI Testing and Previews

[Intermediate] How do you test SwiftUI screens?

Interview answer: Test business logic in view models/use cases, use previews with mock dependencies for visual states, use snapshot tests where appropriate, and use XCUITest for critical user flows.

Deep explanation: SwiftUI views are declarative and hard to unit test directly without relying on implementation details. The testable surface should be state transformation and user-visible behavior.

Why it matters: Good test strategy catches regressions without brittle tests that break on harmless layout refactors.

How it works: View models expose state and actions. Previews render different states. UI tests interact through accessibility identifiers/labels.

Use cases: Loading/error/success states, forms, navigation, accessibility, purchase flows, onboarding.

Common mistakes: Testing private view hierarchy. Depending only on previews. Overusing snapshot tests for dynamic content.

Best practices: Inject dependencies. Keep view models deterministic. Add accessibility identifiers for UI tests when labels are insufficient.

Red flag answers: "You do not test SwiftUI." You test the logic and critical UI behavior around it.

Code example:

```
@MainActor
@Test
func loadDisplaysName() async throws {
    let repository = StubUserRepository(user: User(id: UUID(), name: "Ari"))
    let viewModel = ProfileViewModel(repository: repository)

    await viewModel.load(id: repository.user.id)

    #expect(viewModel.name == "Ari")
}
```

Possible follow-up questions: When do snapshots add value? How do you make previews realistic?

[Intermediate] What is preview-driven development?

Interview answer: Preview-driven development uses SwiftUI previews to build and verify UI states quickly with mock data and dependencies.

Deep explanation: Previews are not tests, but they speed feedback. A strong preview setup shows loading, empty, error, success, long text, Dynamic Type, dark mode, and localization variants.

Why it matters: It improves UI quality and catches edge cases before simulator testing.

How it works: Preview providers or `#Preview` instantiate views with controlled state and dependencies.

Use cases: Component libraries, state-heavy screens, accessibility checks, design review.

Common mistakes: Only previewing the happy path. Using live network dependencies. Letting previews bit-rot.

Best practices: Create mock fixtures. Preview edge cases. Keep preview dependencies local and deterministic.

Red flag answers: "If preview looks good, the app works." Previews complement tests; they do not replace runtime verification.

Code example:

```
#Preview("Error") {
    ProfileView(
        model: .preview(state: .failed("Unable to load profile."))
    )
```

```
}
```

Possible follow-up questions: How do you inject mock dependencies into previews? What preview states do you always add?

UIKit and iOS Fundamentals

[Beginner] What is the UIViewController lifecycle?

Interview answer: Key lifecycle methods include `viewDidLoad`, `viewWillAppear`, `viewDidAppear`, `viewWillDisappear`, and `viewDidDisappear`. `viewDidLoad` is for one-time view setup; appearance methods are for work tied to visibility.

Deep explanation: UIKit view controllers have reference identity and lifecycle callbacks driven by containment, presentation, navigation, and view loading.

Why it matters: Many iOS apps still use UIKit. SwiftUI interop also requires understanding UIKit lifecycle.

How it works: The view loads lazily. Appearance callbacks may run multiple times as screens appear/disappear.

Use cases: Setup UI, start/stop observers, analytics, refresh visible content, manage child controllers.

Common mistakes: Doing layout before views have final bounds. Starting duplicate network work in `viewWillAppear`. Forgetting to remove observers.

Best practices: Use `viewDidLoad` for setup, `viewWillAppear` for refresh, `viewDidLayoutSubviews` for frame-dependent adjustments, and explicit cancellation for tasks.

Red flag answers: "viewDidLoad runs every time the screen appears." It runs when the view is loaded, not every appearance.

Code example:

```
final class ProfileViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .systemBackground
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        refreshIfNeeded()
    }
}
```

Possible follow-up questions: When does `viewDidLayoutSubviews` run? How does containment affect lifecycle?

[Intermediate] What is the app lifecycle?

Interview answer: The app lifecycle describes transitions such as launch, active, inactive, background, and termination. Modern apps usually manage scenes through `UISceneDelegate` or SwiftUI's `Scene` API.

Deep explanation: iOS apps can have multiple scenes/windows. App-level lifecycle and scene-level lifecycle are related but not identical.

Why it matters: Lifecycle affects saving state, background tasks, deep links, notifications, and resource management.

How it works: The system sends delegate callbacks or SwiftUI scene phase updates. Apps respond by pausing work, saving state, or refreshing data.

Use cases: Persist drafts on background, refresh on active, handle URLs, manage background uploads.

Common mistakes: Assuming one window/scene. Doing heavy work during launch. Not handling background expiration.

Best practices: Keep launch fast. Use scene phase for SwiftUI. Save critical state proactively.

Red flag answers: "AppDelegate handles everything." Modern iOS separates app and scene responsibilities.

Code example:

```
@Environment(\.scenePhase) private var scenePhase
```

```
.onChange(of: scenePhase) { _, phase in
    if phase == .background {
        draftStore.save()
    }
}
```

Possible follow-up questions: What is the difference between AppDelegate and SceneDelegate? How do background tasks work?

[Beginner] What are delegates and NotificationCenter?

Interview answer: Delegates are one-to-one callback relationships, often weak. NotificationCenter broadcasts events to many observers without tight coupling.

Deep explanation: Delegation is explicit and type-safe when modeled with protocols. NotificationCenter is useful for cross-cutting events but can become hard to trace.

Why it matters: UIKit uses delegation heavily. Choosing between delegate, closure, notification, Combine, or async stream affects architecture.

How it works: A delegate property points to an object implementing a protocol. NotificationCenter posts named notifications to observers.

Use cases: Delegates: table view events, coordinator callbacks, child-to-parent communication. Notifications: app-wide system events, keyboard, logout broadcast.

Common mistakes: Strong delegates. Using notifications for request/response flows. Forgetting observer cleanup in old APIs.

Best practices: Use delegates for direct ownership relationships. Use notifications sparingly for broadcast events.

Red flag answers: "NotificationCenter is simpler than dependency injection." It often hides dependencies.

Code example:

```
protocol LoginViewControllerDelegate: AnyObject {
    func loginViewControllerDidFinish(_ controller: LoginViewController)
}

final class LoginViewController: UIViewController {
    weak var delegate: LoginViewControllerDelegate?
}
```

Possible follow-up questions: Why are delegates weak? When would you use Combine instead?

[Intermediate] What are Combine basics relevant to iOS interviews?

Interview answer: Combine is Apple's reactive framework using publishers, subscribers, operators, and cancellables to model asynchronous streams of values.

Deep explanation: Even with Swift Concurrency, Combine remains in many apps, especially for `ObservableObject`, event streams, and older reactive architectures.

Why it matters: You may need to maintain Combine code or bridge it to async/await.

How it works: A publisher emits values/completion. Operators transform streams. A subscriber receives values. `AnyCancellable` controls subscription lifetime.

Use cases: Search debouncing, form validation, notification streams, `@Published` view model state, legacy reactive pipelines.

Common mistakes: Not storing cancellables. Creating retain cycles in `sink`. Updating UI off main thread.

Best practices: Store cancellables intentionally. Use `[weak self]` in sinks where needed. Use `receive(on:)` for UI updates in non-main contexts.

Red flag answers: "Combine and async/await are the same." Combine models streams; async/await models asynchronous operations. AsyncSequence bridges some stream use cases.

Code example:

```
searchTextPublisher
    .debounce(for: .milliseconds(300), scheduler: RunLoop.main)
    .removeDuplicates()
    .sink { [weak self] query in
        self?.search(query)
    }
    .store(in: &cancellables)
```

Possible follow-up questions: How do you bridge Combine to async/await? What is backpressure?

[Intermediate] UIKit vs SwiftUI: how do you choose?

Interview answer: Use SwiftUI for modern declarative UI, rapid iteration, previews, and state-driven screens. Use UIKit when you need mature imperative control, specific components, advanced collection behavior, or existing codebase consistency.

Deep explanation: This is not a religion. Many apps are hybrid. The right choice depends on deployment target, team experience, component complexity, and long-term maintenance.

Why it matters: Senior engineers make trade-offs, not blanket framework claims.

How it works: SwiftUI describes UI as values and state. UIKit builds object hierarchies and mutates them directly.

Use cases: SwiftUI: settings, forms, simple-to-medium screens, new features. UIKit: complex collection layouts, mature custom controls, legacy modules, highly tuned interactions.

Common mistakes: Forcing SwiftUI into places where UIKit would be simpler. Rebuilding UIKit patterns imperatively inside SwiftUI.

Best practices: Use the framework that best fits the feature and team. Keep boundaries clean through representables and hosting controllers.

Red flag answers: "SwiftUI replaces UIKit completely." Real apps still use both.

Code example:

```
// UIKit app presenting a SwiftUI feature:
let controller = UIHostingController(rootView: SettingsView())
navigationController?.pushViewController(controller, animated: true)
```

Possible follow-up questions: What SwiftUI limitations have you hit? How do you migrate incrementally?

[Intermediate] What are Auto Layout, UITableView, and UICollectionView core concepts?

Interview answer: Auto Layout defines constraints between views. UITableView and UICollectionView display reusable cells for large data sets; collection views are more flexible and power grids/custom layouts.

Deep explanation: UIKit layout is constraint- or frame-based. Table/collection views rely on cell reuse, data sources, delegates, and layout objects to efficiently display scrolling content.

Why it matters: UIKit fundamentals remain common interview topics and production requirements.

How it works: Auto Layout solves constraints to produce frames. Table/collection views ask data sources for cell counts and cells, reuse offscreen cells, and delegate user interactions.

Use cases: Legacy feeds, complex grids, compositional layouts, high-performance lists, custom interactions.

Common mistakes: Creating new cells manually instead of dequeuing. Ambiguous constraints. Doing heavy work during cell configuration.

Best practices: Keep cells reusable and stateless. Use diffable data sources for modern UIKit lists. Precompute view models for cells.

Red flag answers: "SwiftUI means I do not need UIKit knowledge." Many apps and frameworks still rely on UIKit.

Code example:

```
let registration = UICollectionView.CellRegistration<UICollectionViewListCell, Item> { cell, _, item in
    var content = cell.defaultContentConfiguration()
    content.text = item.title
    cell.contentConfiguration = content
}
```

Possible follow-up questions: What is cell reuse? How does diffable data source work?

[Beginner] What are localization basics?

Interview answer: Localization adapts user-facing text, dates, numbers, plurals, layout direction, and resources for different languages/regions.

Deep explanation: Localization is more than translating strings. It includes formatting, pluralization, right-to-left layout, text expansion, and cultural expectations.

Why it matters: Poor localization breaks UI and user trust in global apps.

How it works: Use localized string resources, formatters, and environment locale. SwiftUI automatically responds to some locale/layout direction changes.

Use cases: App strings, currency, dates, measurements, plural messages, onboarding, error messages.

Common mistakes: String concatenation. Hard-coded date formats. Not testing long translations or RTL. Putting user-facing text in code without localization.

Best practices: Use string catalogs where available. Use formatters. Test pseudolocalization and RTL.

Red flag answers: "Localization is just Localizable.strings." It is a full product quality concern.

Code example:

```
Text("checkout.items.count \(itemCount)")
```

Possible follow-up questions: How do you handle plurals? What UI issues appear in German or Arabic?

Architecture

[Beginner] What is MVC in iOS?

Interview answer: MVC separates Model, View, and Controller. In UIKit, view controllers often become large because they mediate lifecycle, UI updates, navigation, and business logic.

Deep explanation: Apple's UIKit MVC can work for small screens, but without discipline controllers become "Massive View Controllers."

Why it matters: Interviewers ask MVC to see whether you understand trade-offs and why teams adopt MVVM or coordinators.

How it works: Model holds data/business concepts. View displays UI. Controller coordinates between them.

Use cases: Simple UIKit screens, prototypes, small components.

Common mistakes: Putting networking, parsing, validation, and navigation all in one view controller.

Best practices: Keep controllers thin. Extract services, view models, coordinators, and repositories as complexity grows.

Red flag answers: "MVC is bad." MVC is fine when kept small; the problem is uncontrolled responsibilities.

Code example:

```
final class ProfileViewController: UIViewController {
    private let viewModel: ProfileViewModel

    init(viewModel: ProfileViewModel) {
        self.viewModel = viewModel
        super.init(nibName: nil, bundle: nil)
    }

    @available(*, unavailable)
    required init?(coder: NSCoder) {
        fatalError("init(coder:) has not been implemented")
    }
}
```

Possible follow-up questions: How do you prevent Massive View Controllers? How does MVC differ from MVVM?

[Intermediate] What is MVVM?

Interview answer: MVVM separates UI rendering from presentation logic. The View displays state and forwards user actions; the ViewModel owns UI state, formatting, validation, and calls use cases/services.

Deep explanation: In SwiftUI, MVVM can be natural if the ViewModel is the state and action layer. But overusing view models for every small view can add unnecessary complexity.

Why it matters: MVVM improves testability and keeps views declarative.

How it works: View observes view model state. User actions call view model methods. View model updates published/observable state.

Use cases: Forms, network-backed screens, complex UI state, testable presentation logic.

Common mistakes: Putting navigation, business logic, and persistence all into the ViewModel. Making ViewModel depend on SwiftUI types unnecessarily.

Best practices: Keep ViewModel `@MainActor` for UI state. Inject dependencies. Move domain logic into use cases when it grows.

Red flag answers: "MVVM means every view must have a ViewModel." Not every small component needs one.

Code example:

```
@MainActor
```

```
@Observable
final class LoginViewModel {
    var email = ""
    var password = ""
    var errorMessage: String?

    private let login: LoginUseCase

    init(login: LoginUseCase) {
        self.login = login
    }

    func submit() async {
        do {
            try await login(email: email, password: password)
        } catch {
            errorMessage = "Unable to sign in."
        }
    }
}
```

Possible follow-up questions: Where should navigation live in MVVM? How do you test ViewModels?

[Senior] What are VIPER and Clean Architecture?

Interview answer: VIPER separates View, Interactor, Presenter, Entity, and Router. Clean Architecture separates domain, data, and presentation with dependencies pointing inward toward business rules.

Deep explanation: Both aim to control dependencies and improve testability in large codebases. VIPER is more prescriptive and can be verbose. Clean Architecture is more conceptual: domain should not depend on UI or infrastructure.

Why it matters: Senior engineers should choose architecture based on scale and team needs, not trend.

How it works: VIPER assigns roles to specific components. Clean Architecture uses entities/use cases/repositories and dependency inversion.

Use cases: Large teams, complex business domains, modular apps, high test coverage requirements.

Common mistakes: Applying VIPER to tiny screens. Creating ceremony without business complexity. Letting domain import UIKit/SwiftUI.

Best practices: Keep domain pure. Use protocols at boundaries. Avoid architecture that hides simple flows behind excessive files.

Red flag answers: "VIPER is always better because more layers." More layers are a cost unless they manage real complexity.

Code example:

```
protocol FetchOrdersUseCase {
    func execute() async throws -> [Order]
}

struct DefaultFetchOrdersUseCase: FetchOrdersUseCase {
    let repository: OrderRepository

    func execute() async throws -> [Order] {
        try await repository.orders()
    }
}
```

Possible follow-up questions: How do dependencies point in Clean Architecture? When is VIPER overkill?

[Intermediate] What is dependency injection?

Interview answer: Dependency injection means providing an object's dependencies from the outside instead of creating them internally. It improves testability, configurability, and separation of concerns.

Deep explanation: Constructor injection is usually clearest because dependencies are explicit and immutable. Environment/service-locator patterns can be useful but hide requirements if overused.

Why it matters: Testing requires substituting network, storage, clock, analytics, and feature flags.

How it works: Objects depend on protocols or concrete collaborators passed through initializers. Tests pass mocks/stubs.

Use cases: Repositories, API clients, view models, use cases, coordinators, analytics.

Common mistakes: Using singletons for everything. Creating dependencies inside view models. Over-abstracting every type.

Best practices: Prefer constructor injection. Use protocols at meaningful seams. Keep composition at app/feature entry points.

Red flag answers: "DI means using a dependency injection framework." Manual DI is often enough in Swift apps.

Code example:

```
@MainActor
final class OrdersViewModel: ObservableObject {
    private let fetchOrders: FetchOrdersUseCase

    init(fetchOrders: FetchOrdersUseCase) {
        self.fetchOrders = fetchOrders
    }
}
```

Possible follow-up questions: How do you inject dependencies into SwiftUI? When are singletons acceptable?

[Intermediate] What are Repository and Coordinator patterns?

Interview answer: A Repository abstracts data access behind a domain-friendly interface. A Coordinator owns navigation flow and screen assembly, especially in UIKit or hybrid apps.

Deep explanation: Repositories hide whether data comes from network, cache, database, or mock. Coordinators keep navigation out of view controllers/view models when flows are complex.

Why it matters: Both patterns reduce coupling and improve testability when used at the right scale.

How it works: Repository protocols live near the domain/use case. Implementations live in data/infrastructure. Coordinators hold navigation controllers or route state and create screens.

Use cases: Offline caching, API migration, test doubles, onboarding flows, checkout flows, deep links.

Common mistakes: Repository that simply mirrors API endpoints without domain meaning. Coordinator that becomes a global app god object.

Best practices: Give repositories domain methods. Keep coordinators flow-scoped. Avoid business logic in coordinators.

Red flag answers: "Repository is just a network service." It should abstract data source decisions for domain needs.

Code example:

```
protocol OrderRepository {
    func orders() async throws -> [Order]
    func order(id: Order.ID) async throws -> Order
}
```

Possible follow-up questions: Where do you put caching? How do SwiftUI navigation models compare to coordinators?

[Senior] How do you manage state in a large SwiftUI app?

Interview answer: Use clear ownership boundaries: local `@State` for local UI, observable feature models for screen state, domain use cases for business logic, repositories for data access, and explicit route/navigation state. Avoid one giant global state unless the app architecture intentionally supports it.

Deep explanation: Large SwiftUI apps fail when state is duplicated, hidden, or overly global. Good architecture defines what owns each state, how actions mutate it, and what is derived.

Why it matters: State management affects correctness, performance, testability, navigation, and team scalability.

How it works: Views read state and send actions. Feature models/use cases mutate state and call dependencies. Shared app state is injected deliberately, not discovered randomly.

Use cases: Authentication, feature flags, profile, cart, checkout, navigation, offline sync.

Common mistakes: EnvironmentObject for everything. Multiple sources of truth. View models talking directly to each other. Business logic in views.

Best practices: Keep feature state modular. Use derived state. Prefer explicit actions. Test state transitions.

Red flag answers: "Use Redux for all SwiftUI apps." Centralized state can help some apps, but it is not automatically the right choice.

Code example:

```
@Observable
final class CartFeatureModel {
    private let pricing: PricingUseCase
    var items: [CartItem] = []
    var total: Money = .zero

    init(pricing: PricingUseCase) {
        self.pricing = pricing
    }

    func updateQuantity(itemID: CartItem.ID, quantity: Int) async {
        items[id: itemID]?.quantity = quantity
        total = await pricing.total(for: items)
    }
}
```

Possible follow-up questions: How do you avoid EnvironmentObject overuse? How would you modularize a SwiftUI feature?

Networking and Persistence

[Intermediate] How do you design a URLSession networking layer?

Interview answer: Create a small API client that builds requests, validates HTTP responses, decodes typed `Decodable` responses, maps errors, and is injected behind a protocol for testing.

Deep explanation: Networking code should separate transport concerns from domain concerns. URLSession performs requests; repositories decide which endpoints matter to the app.

Why it matters: Clean networking makes error handling, retries, authentication, and testing manageable.

How it works: `URLSession.data(for:)` returns data and response asynchronously. You validate status code/content, decode, and map failures.

Use cases: REST APIs, authenticated calls, pagination, uploads/downloads, retry logic.

Common mistakes: Ignoring status codes. Decoding error bodies as success. Using `URLSession.shared` directly everywhere. Not testing invalid responses.

Best practices: Centralize request construction and response validation. Keep endpoints typed. Inject sessions/clients.

Red flag answers: "If decoding succeeds, the request succeeded." HTTP status still matters.

Code example:

```
struct HTTPClient {
    let session: URLSession

    func send<Response: Decodable>(_ request: URLRequest) async throws -> Response {
        let (data, response) = try await session.data(for: request)
        guard let http = response as? HTTPURLResponse,
              (200..<300).contains(http.statusCode) else {
            throw URLError(.badServerResponse)
        }
        return try JSONDecoder().decode(Response.self, from: data)
    }
}
```

Possible follow-up questions: How do you test URLSession code? How do you handle auth refresh?

[Beginner] What is Codable?

Interview answer: `Codable` is a type alias for `Encodable & Decodable`. It lets Swift types encode to and decode from external formats such as JSON.

Deep explanation: The compiler can synthesize Codable implementations when stored properties are Codable and keys match. Custom coding keys and custom init/encode handle mismatches.

Why it matters: Most iOS apps parse API responses and persist structured data.

How it works: Decoders walk keyed/unkeyed containers and construct Swift values. Encoding does the reverse.

Use cases: JSON APIs, local cache files, app state export, persistence.

Common mistakes: Using API DTOs as domain models everywhere. Force-trying decoding. Not handling date/key strategies.

Best practices: Separate DTOs from domain when API shape is unstable or awkward. Test decoding with sample payloads.

Red flag answers: "Codable always maps JSON automatically." It works automatically only when shapes and types align.

Code example:

```
struct UserResponse: Decodable {
```

```
    let id: UUID
    let displayName: String

    enum CodingKeys: String, CodingKey {
        case id
        case displayName = "display_name"
    }
}
```

Possible follow-up questions: How do you decode dates? How do you handle optional vs missing fields?

[Intermediate] How do you handle networking errors?

Interview answer: Separate transport errors, HTTP errors, decoding errors, and domain errors. Preserve technical details for logging while mapping to user-friendly UI state.

Deep explanation: Users need actionable messages. Engineers need diagnostics. A good error layer serves both without leaking backend internals into the UI.

Why it matters: Error handling determines retry behavior, analytics, supportability, and user trust.

How it works: Catch lower-level errors, map them to typed app errors, and expose presentation state to the UI.

Use cases: Offline handling, authentication expiration, validation errors, server failures, rate limits.

Common mistakes: Showing raw error descriptions. Treating all errors as generic. Losing underlying error context.

Best practices: Use typed errors where they drive behavior. Log underlying errors. Design retry rules.

Red flag answers: "Just show an alert for catch." That ignores context and recovery.

Code example:

```
enum APIError: Error {
    case transport(URLError)
    case server(statusCode: Int, data: Data)
    case decoding(Error)
}
```

Possible follow-up questions: Where should error mapping happen? How do you handle validation errors from the backend?

[Intermediate] How do you approach caching?

Interview answer: Choose cache strategy based on freshness, consistency, cost, and offline needs. Use memory cache for short-lived expensive data, disk/database cache for persistence, and HTTP caching where server headers support it.

Deep explanation: Caching is a product behavior, not only a performance trick. You must define when cached data is valid and how it is invalidated.

Why it matters: Good caching improves speed and offline UX. Bad caching shows stale or incorrect data.

How it works: Caches map keys to values plus metadata such as timestamps, ETags, versions, or dependencies.

Use cases: Images, profile data, feed pages, feature flags, static config, offline drafts.

Common mistakes: No invalidation plan. Caching personalized sensitive data insecurely. Duplicate caches at multiple layers with inconsistent rules.

Best practices: Define TTL/staleness behavior. Respect server caching where possible. Keep sensitive data encrypted or avoid caching it.

Red flag answers: "Cache everything for performance." Correctness and privacy come first.

Code example:

```
actor ImageMemoryCache {
    private var images: [URL: UIImage] = [:]
```

```
    func image(for url: URL) -> UIImage? {
        images[url]
    }

    func insert(_ image: UIImage, for url: URL) {
        images[url] = image
    }
}
```

Possible follow-up questions: How do ETags work? How do you handle cache invalidation after mutation?

[Beginner] UserDefaults vs Keychain?

Interview answer: Use UserDefaults for lightweight non-sensitive preferences. Use Keychain for secrets such as tokens, credentials, and keys.

Deep explanation: UserDefaults is convenient preference storage, not secure storage. Keychain is designed for sensitive data and can persist across app reinstalls depending on configuration.

Why it matters: Storing tokens in UserDefaults is a security risk.

How it works: UserDefaults stores property-list values. Keychain stores secure items protected by system security/accessibility classes.

Use cases: UserDefaults: toggles, onboarding seen flag, last selected tab. Keychain: auth token, refresh token, private key.

Common mistakes: Storing PII/secrets in UserDefaults. Overusing Keychain for ordinary preferences. Not handling Keychain errors.

Best practices: Keep stored data minimal. Use Keychain wrappers carefully. Consider access group and accessibility requirements.

Red flag answers: "UserDefaults is fine because app sandbox protects it." It is not appropriate for secrets.

Code example:

```
@AppStorage("hasCompletedOnboarding")
private var hasCompletedOnboarding = false
```

Possible follow-up questions: What Keychain accessibility class would you choose? What happens on reinstall?

[Intermediate] Core Data vs SwiftData?

Interview answer: Core Data is the mature object graph and persistence framework available for many OS versions. SwiftData is the modern Swift-native persistence framework introduced in iOS 17, built to integrate with SwiftUI and macros.

Deep explanation: SwiftData simplifies common persistence with `@Model`, model containers, and query integration. Core Data remains more mature, configurable, and necessary for older OS support or advanced scenarios.

Why it matters: Persistence choice affects deployment target, migration, performance, tooling, and team familiarity.

How it works: Core Data uses managed object models and contexts. SwiftData uses model types and model contexts with macro-generated persistence metadata.

Use cases: Core Data: legacy apps, complex migrations, older OS support. SwiftData: new iOS 17+ apps with SwiftUI-first data models.

Common mistakes: Treating persistence models as domain models without boundaries. Doing heavy fetches on main context. Ignoring migration strategy.

Best practices: Design persistence separately from API DTOs. Test migrations. Keep writes controlled and observable.

Red flag answers: "SwiftData replaces Core Data for every app." Deployment target and complexity matter.

Code example:

```
@Model
final class Note {
    var title: String
    var createdAt: Date

    init(title: String, createdAt: Date = .now) {
        self.title = title
        self.createdAt = createdAt
    }
}
```

Possible follow-up questions: How do you handle migrations? How do you keep persistence off the UI hot path?

Testing and Debugging

[Beginner] What is XCTest and what is Swift Testing?

Interview answer: XCTest is Apple's long-standing testing framework. Swift Testing is the newer Swift-native testing framework with `@Test` and `#expect`, designed for expressive tests in modern Swift.

Deep explanation: Many projects still use XCTest, especially for UI tests and older infrastructure. Swift Testing can make unit tests more concise where supported.

Why it matters: Interviewers want to know you can test production code using the project's existing tools.

How it works: XCTest uses test case classes and assertions. Swift Testing uses annotated test functions and expectation macros.

Use cases: Unit tests, integration tests, view model tests, repository tests, UI tests with XCUITest.

Common mistakes: Testing implementation details. Using sleeps instead of deterministic async testing. Not isolating dependencies.

Best practices: Prefer deterministic tests. Mock dependencies through protocols. Test behavior and edge cases.

Red flag answers: "I only test manually." Manual QA is not a substitute for automated tests.

Code example:

```
@Test
func fullNameCombinesFirstAndLastName() {
    let user = User(firstName: "Ari", lastName: "Tan")
    #expect(user.fullName == "Ari Tan")
}
```

Possible follow-up questions: How do you test async code? What belongs in UI tests vs unit tests?

[Intermediate] How do you unit test ViewModels?

Interview answer: Inject dependencies, control inputs, call view model actions, and assert published/observable state. Keep tests on the main actor if the view model is main-actor isolated.

Deep explanation: ViewModel tests should prove state transitions and side effects at boundaries, not SwiftUI rendering details.

Why it matters: This is the highest-value test layer for most SwiftUI features.

How it works: Use fake repositories/services that return known results. Await async actions. Assert final state and important intermediate states if relevant.

Use cases: Loading success/failure, form validation, retry, navigation events, analytics calls.

Common mistakes: Using real network. Relying on timers/sleeps. Testing private properties instead of public state.

Best practices: Use protocol-based mocks/stubs. Keep test data small. Test cancellation if relevant.

Red flag answers: "SwiftUI views cannot be tested, so no tests." Test the state model and critical UI flows.

Code example:

```
@MainActor
@Test
func loadFailureShowsError() async {
    let viewModel = ProfileViewModel(repository: FailingUserRepository())

    await viewModel.load(id: UUID())

    #expect(viewModel.errorMessage == "Unable to load profile.")
}
```

Possible follow-up questions: How do you test published changes over time? How do you test cancellation?

[Intermediate] How do you mock dependencies in Swift?

Interview answer: Define small protocols at meaningful boundaries and provide fake, stub, spy, or mock implementations in tests.

Deep explanation: Mocks are not magic. A fake has working simplified behavior. A stub returns fixed data. A spy records calls. A mock usually verifies interactions.

Why it matters: Good test doubles make tests deterministic without over-coupling to implementation.

How it works: Production code receives protocol-conforming dependencies through initializers. Tests pass a test double.

Use cases: Repositories, clocks, API clients, analytics, persistence, feature flags.

Common mistakes: Huge protocols that are painful to mock. Verifying every internal call. Third-party mocking frameworks where simple fakes suffice.

Best practices: Keep protocols small and consumer-owned. Use spies for side effects like analytics.

Red flag answers: "Use a mocking library for everything." Manual test doubles are often clearer in Swift.

Code example:

```
struct StubClock: ClockProviding {
    let now: Date
}

final class AnalyticsSpy: AnalyticsTracking {
    private(set) var events: [String] = []

    func track(_ event: String) {
        events.append(event)
    }
}
```

Possible follow-up questions: What is the difference between fake and mock? Where should protocols live?

[Intermediate] How do you debug crashes?

Interview answer: Start with the crash log: exception type, signal, crashed thread, stack trace, app version, device/OS, and reproduction steps. Symbolicate if needed, then connect the stack to recent code and runtime state.

Deep explanation: Crash debugging is evidence-driven. The top frame may not be root cause; look for the first app frame and surrounding context.

Why it matters: Production crash fixes require precision, not guesses.

How it works: Crash reports capture thread stacks and exception data. Symbolication maps addresses to function/file/line using dSYMs.

Use cases: Force unwrap crashes, array out of bounds, main-thread violations, memory pressure, Objective-C exceptions, concurrency precondition failures.

Common mistakes: Fixing the symptom without understanding input state. Ignoring breadcrumbs/logs. Not matching dSYM to build.

Best practices: Symbolicate correctly. Add defensive handling where input is invalid. Add regression tests when possible.

Red flag answers: "The stack trace says line X, so just nil-check it." You need to know why invalid state reached that line.

Code example:

```
guard users.indices.contains(index) else {
    assertionFailure("Invalid selected user index: \(index)")
    return
}
```

Possible follow-up questions: What is symbolication? How do you debug crashes you cannot reproduce?

[Senior] How do you debug SwiftUI performance issues?

Interview answer: Use profiling and reduction: identify main-thread work, excessive invalidation, unstable identity, expensive body work, image decoding, layout complexity, and animation cost. Validate with Instruments, not intuition.

Deep explanation: SwiftUI performance issues are often architectural. One broad observable object or unstable list ID can cause more damage than a single expensive modifier.

Why it matters: Smooth UI is core iOS quality.

How it works: Instruments shows CPU, allocations, rendering, hangs, and SwiftUI update patterns. You can isolate a subtree, replace data, or remove modifiers to find the bottleneck.

Use cases: Scrolling feeds, charts, complex forms, animated dashboards, image-heavy screens.

Common mistakes: Prematurely rewriting in UIKit. Optimizing without measuring. Ignoring image size/decoding.

Best practices: Profile on device. Keep body cheap. Use stable IDs. Split state. Cache expensive work.

Red flag answers: "AnyView is the reason" without evidence. It may be a factor, but profile first.

Code example:

```
// Move this out of body if it is expensive or called frequently.
let rowModels = messages.map(MessageRowModel.init)
```

Possible follow-up questions: Which Instruments would you open first? How do you distinguish body recomputation from rendering cost?

Additional Deep-Dive FAQs for Missing Interview Topics

[Beginner] What does declarative UI mean in SwiftUI?

Interview answer: Declarative UI means you describe what the UI should look like for a given state, not the step-by-step mutations needed to get there. In SwiftUI, the view is a function of state: change the state, and SwiftUI updates the UI.

Deep explanation: UIKit code often says "find this label and set its text." SwiftUI code says "if state is loading, show a spinner; if loaded, show content." The framework owns the update process. Your job is to model state clearly and render it consistently.

Why it matters: This is the core mindset shift from UIKit to SwiftUI. Without it, developers put side effects in `body`, mutate views directly, or duplicate state.

How it works: SwiftUI evaluates view descriptions from current state, tracks dependencies, and reconciles those descriptions against the previous view graph.

Use cases: Loading states, form validation, feature flags, conditional navigation, empty states, and error banners.

Common mistakes: Trying to hold references to views. Imperatively hiding/showing subviews. Updating state inside `body`. Treating `body` as a lifecycle callback.

Best practices: Model UI states explicitly. Keep `body` deterministic. Move effects into `.task`, actions, view models, or use cases.

Red flag answers: "Declarative UI means less code." Sometimes it does, but the real point is state-driven rendering.

Code example:

```
enum ScreenState {
    case loading
    case loaded([Message])
    case failed(String)
}

struct InboxView: View {
    let state: ScreenState

    var body: some View {
        switch state {
        case .loading:
            ProgressView()
        case .loaded(let messages):
            List(messages) { Text($0.title) }
        case .failed(let message):
            ContentUnavailableView("Error", systemImage: "exclamationmark.triangle", description: Text(
message))
        }
    }
}
```

Possible follow-up questions: How does declarative UI change testing? Why should `body` avoid side effects?

[Intermediate] What is the difference between structured and unstructured concurrency?

Interview answer: Structured concurrency scopes child tasks to a parent operation, so cancellation, priority, and errors flow predictably. Unstructured concurrency creates work whose lifetime is not tied to the current scope, such as `Task {}` stored or launched from synchronous code.

Deep explanation: Structured concurrency makes async lifetimes visible in code. `async let` and task groups cannot outlive their scope. Unstructured tasks can be useful at UI boundaries, but they need explicit lifecycle and cancellation.

Why it matters: Unstructured tasks are a common cause of duplicate work, leaked work, stale UI updates, and hard-to-debug cancellation behavior.

How it works: Child tasks are attached to parent tasks. Unstructured tasks are scheduled independently, though `Task {}` may inherit priority and actor context from the creation point. `Task.detached` does not inherit actor isolation in the same way.

Use cases: Structured: parallel loading inside one async function. Unstructured: starting work from a button tap or UIKit delegate method.

Common mistakes: Using `Task {}` inside an async function instead of `async let` or a task group. Forgetting to store/cancel view-scoped tasks. Using `Task.detached` to escape actor rules.

Best practices: Prefer structured concurrency inside async code. Use unstructured tasks only at boundaries and give them clear ownership.

Red flag answers: "Unstructured means bad." It is not bad; it is just easier to misuse.

Code example:

```
func loadScreen() async throws -> ScreenData {
    async let profile = api.profile()
    async let settings = api.settings()
    return try await ScreenData(profile: profile, settings: settings)
}

final class LegacyController: UIViewController {
    private var loadTask: Task<Void, Never>?

    func reloadTapped() {
        loadTask?.cancel()
        loadTask = Task { await reload() }
    }
}
```

Possible follow-up questions: When would you use `Task.detached`? How does cancellation propagate through task groups?

[Senior] What are common Swift concurrency mistakes in iOS apps?

Interview answer: Common mistakes include updating UI off the main actor, launching unstructured tasks without cancellation, ignoring cancellation, sharing mutable non-Sendable state, using `Task.detached` unnecessarily, blocking async code with semaphores, and assuming actors solve logical races.

Deep explanation: Swift Concurrency improves safety, but it does not remove the need for ownership and lifecycle design. Many bugs come from mixing old callback/GCD code with async code without clear boundaries.

Why it matters: Concurrency bugs are often intermittent and production-only. Interviewers want to know you can prevent them, not just explain syntax.

How it works: Tasks suspend and resume. Actors serialize isolated state but can be reentrant. Cancellation is cooperative. Sendable checking prevents some unsafe transfers but does not validate all business invariants.

Use cases: Search-as-you-type, image loading, checkout submit, token refresh, shared caches, and analytics pipelines.

Common mistakes: Using `DispatchQueue.main.async` inside `@MainActor` code. Swallowing `CancellationError` as a failure. Mutating shared arrays from multiple tasks. Starting a new task on every body recomputation.

Best practices: Mark UI models `@MainActor`. Use structured concurrency. Make cancellation explicit. Use actors for shared mutable state. Return values from child tasks instead of mutating shared state.

Red flag answers: "Just wrap it in a Task." That may hide the real lifecycle problem.

Code example:

```
@MainActor
@Observable
final class SearchModel {
    var results: [Item] = []
    private var task: Task<Void, Never>?
```

```
    func search(_ query: String) {
        task?.cancel()
        task = Task {
            do {
                results = try await service.search(query)
            } catch is CancellationError {
                // Neutral outcome.
            } catch {
                results = []
            }
        }
    }
}
```

Possible follow-up questions: Why is blocking with semaphores dangerous in async code? How can actor reentrancy still create bugs?

[Intermediate] What is `deinit`, and how does it help with memory debugging?

Interview answer: `deinit` runs before a class instance is deallocated. It is useful for cleanup and for confirming whether objects such as view controllers, coordinators, or view models are actually released.

Deep explanation: Value types do not have `deinit`; reference types do. In iOS debugging, adding temporary `deinit` logs can reveal retained screens, but it does not tell you why the object is retained.

Why it matters: If `deinit` never runs after leaving a flow, there may be a retain cycle, pending task, timer, notification observer, or long-lived owner.

How it works: ARC calls `deinit` when the strong reference count reaches zero. Stored properties are released after `deinit` begins.

Use cases: Invalidating timers, closing resources, ending observation, debug logs, and verifying screen lifetime.

Common mistakes: Relying on `deinit` for critical business work. Assuming missing `deinit` always means a leak; the object may still have legitimate ownership.

Best practices: Use `deinit` for cleanup and debugging, not core app behavior. Pair `deinit` checks with Memory Graph or Instruments.

Red flag answers: "If `deinit` does not print immediately, there is definitely a leak." Navigation controllers, caches, tasks, and transitions can legitimately extend lifetime.

Code example:

```
final class CheckoutCoordinator {
    deinit {
        print("CheckoutCoordinator deallocated")
    }
}
```

Possible follow-up questions: Why do structs not have `deinit`? What keeps a view controller alive after dismissal?

[Intermediate] What should you know about Instruments for leaks and performance?

Interview answer: Instruments gives evidence about allocations, leaks, CPU, hangs, time spent on the main thread, and rendering performance. For leaks, use Leaks, Allocations, and Memory Graph together. For SwiftUI performance, use Time Profiler, Allocations, Hangs, and SwiftUI-related templates where available.

Deep explanation: Instruments helps distinguish real leaks from caches, and real performance bottlenecks from harmless body recomputation. The key is to reproduce a small flow repeatedly and compare before/after behavior.

Why it matters: Senior engineers do not guess at performance and memory problems; they measure.

How it works: Instruments samples or tracks runtime behavior while the app runs. Different instruments answer different questions: "what allocated?", "what stayed alive?", "what consumed CPU?", "what blocked the main thread?"

Use cases: Leaked view controllers, slow scrolling lists, image decoding spikes, launch performance, memory pressure, and animation hitches.

Common mistakes: Looking only at one snapshot. Treating all retained memory as a leak. Profiling only on simulator. Ignoring device class and release builds.

Best practices: Profile on device when possible. Reproduce the same flow multiple times. Keep notes of baseline, hypothesis, change, and result.

Red flag answers: "I would just add weak self everywhere." That is not evidence-based debugging.

Code example:

```
// Temporary diagnostic only.
deinit {
    os_log("Profile screen deallocated")
}
```

Possible follow-up questions: How do Leaks and Allocations differ? How would you profile a slow SwiftUI list?

[Senior] What are SwiftUI view lifecycle misconceptions?

Interview answer: The biggest misconception is treating SwiftUI views like long-lived UIKit view objects. SwiftUI view structs are transient descriptions; `body`, `init`, `onAppear`, and `onDisappear` can run more often or differently than UIKit lifecycle methods.

Deep explanation: SwiftUI manages persistent identity and state separately from the view struct. A view can be recreated without losing `@State`, or lose state when identity changes. Lazy containers can trigger appearance callbacks as rows enter and leave the visible region.

Why it matters: Misunderstanding lifecycle leads to duplicate network calls, repeated analytics, lost state, and unexpected task cancellation.

How it works: SwiftUI inserts, updates, and removes nodes in its view graph. Lifecycle modifiers correspond to graph/visibility behavior, not object lifetime.

Use cases: Screen loading, row prefetching, video playback, analytics, form state, and navigation.

Common mistakes: Starting network calls in `init` or `body`. Assuming `.onAppear` runs once. Assuming `.onDisappear` means deallocation.

Best practices: Use `.task` for view-scoped async work. Make loading idempotent. Store durable state in an owned model, not in incidental view lifetime.

Red flag answers: "`onAppear` is SwiftUI's `viewDidLoad`." It is not.

Code example:

```
DetailView(id: id)
    .task(id: id) {
        await model.load(id: id)
    }
```

Possible follow-up questions: Why can lazy list rows call `onAppear` many times? How does `.id()` affect lifecycle?

[Senior] How does task cancellation work when SwiftUI views disappear?

Interview answer: Tasks created by `.task` are view-scoped. SwiftUI cancels them when the view disappears or when a `.task(id:)` value changes. The task still has to cooperate with cancellation.

Deep explanation: Cancellation is not a crash or forced thread kill. It marks the task as cancelled. Awaited APIs may throw `CancellationError`, and your own loops should check cancellation.

Why it matters: View-scoped cancellation prevents stale network results, battery waste, and UI updates after navigation.

How it works: SwiftUI owns the task lifetime. When identity changes or the view leaves the graph, the framework requests cancellation. If your code ignores cancellation, work may continue longer than intended.

Use cases: Search screens, detail loading, image loading, async validation, and pagination.

Common mistakes: Treating cancellation as an error alert. Updating state with stale results. Starting unstructured tasks inside `.task` and losing cancellation benefits.

Best practices: Use `.task(id:)` for state-dependent async work. Handle `CancellationError` separately. Check cancellation after expensive awaits or inside loops.

Red flag answers: "SwiftUI kills the task immediately." Cancellation is cooperative.

Code example:

```
.task(id: query) {
    do {
        let results = try await searchService.search(query)
        try Task.checkCancellation()
        model.results = results
    } catch is CancellationError {
        // Ignore stale query.
    } catch {
        model.errorMessage = "Search failed."
    }
}
```

Possible follow-up questions: What happens if you start `Task {}` inside `.task`? How do you prevent stale search results?

[Senior] How do you decide parent vs child state ownership in SwiftUI?

Interview answer: The owner should be the closest component that needs to make authoritative decisions about the state. If only the child cares, use local `@State`. If parent and child both care, lift state to the parent and pass a binding or action.

Deep explanation: SwiftUI works best with one source of truth. Parent-owned state is useful for coordination, persistence, navigation, and business decisions. Child-owned state is useful for local UI details such as expansion, focus, or temporary animation.

Why it matters: Incorrect ownership causes stale copies, unexpected resets, conflicting updates, and hard-to-test flows.

How it works: `@State` stores state for a view identity. `@Binding` lets a child edit state owned elsewhere. Observable models can own feature-level state across multiple views.

Use cases: Forms, sheet presentation, selected row, filters, tabs, child controls, and reusable components.

Common mistakes: Initializing child `@State` from a parent prop and expecting it to stay synced. Passing bindings through many layers for domain actions. Giving every row its own source of truth for shared selection.

Best practices: Lift state only as high as needed. Use bindings for simple edits. Use actions for meaningful business events.

Red flag answers: "Put all state at the top." That creates broad invalidation and over-coupling.

Code example:

```
struct Parent: View {
    @State private var selectedID: Item.ID?

    var body: some View {
        ItemList(selectedID: $selectedID)
    }
}

struct ItemList: View {
    @Binding var selectedID: Item.ID?
```

```
}
```

Possible follow-up questions: When should a child not receive a binding? How do you model derived state?

[Intermediate] How do you implement programmatic navigation in SwiftUI?

Interview answer: Use `NavigationStack` with route values and a bound path. Programmatic navigation means mutating route state, not pushing a view object directly.

Deep explanation: SwiftUI navigation is most robust when represented as data. A route enum lets you append, pop, replace, or deep link into a stack in a testable way.

Why it matters: Programmatic navigation is needed for login redirects, deep links, form completion, notifications, and coordinator-style flows.

How it works: `NavigationStack(path:)` binds to an array or `NavigationPath`. `navigationDestination(for:)` maps route values to views. Mutating the path updates navigation.

Use cases: Deep links, wizard flows, push after save, notification routing, settings subpages.

Common mistakes: Storing destination views in the path. Using many hidden `NavigationLinks`. Mutating navigation state from random child views without a clear owner.

Best practices: Store stable route data such as IDs. Keep route definitions small and Hashable. Centralize navigation mutations for complex flows.

Red flag answers: "I push a SwiftUI view like UIKit." SwiftUI pushes route data, and the destination builder creates the view.

Code example:

```
enum AppRoute: Hashable {
    case order(Order.ID)
    case settings
}

@State private var path: [AppRoute] = []

NavigationStack(path: $path) {
    HomeView(openOrder: { path.append(.order($0)) })
        .navigationDestination(for: AppRoute.self) { route in
            switch route {
            case .order(let id): OrderView(id: id)
            case .settings: SettingsView()
            }
        }
}
```

Possible follow-up questions: When would you use `NavigationPath` instead of `[Route]`? How do you pop to root?

[Senior] How do you handle deep linking in a SwiftUI app?

Interview answer: Parse the incoming URL or notification into app routes, validate required data, update app/session state if needed, and then mutate navigation state such as `NavigationStack` path or selected tab.

Deep explanation: Deep linking is not just navigation. It may require authentication, feature availability, data preloading, error fallback, and routing across tabs or modal flows.

Why it matters: Real apps handle links from push notifications, universal links, widgets, email, and shortcuts. A strong answer shows you can route safely.

How it works: The app receives a URL through SwiftUI `onOpenURL`, scene delegate, app delegate, or notification handling. A router converts it to route state.

Use cases: Open order details, join invite, payment result, promo page, reset password, support conversation.

Common mistakes: Parsing URLs inside views. Assuming the user is authenticated. Pushing a route before the root UI is ready. Ignoring invalid or unsupported links.

Best practices: Create a dedicated deep-link parser/router. Route by stable IDs. Make failure states user-visible and safe.

Red flag answers: "Just append the destination to the path." That ignores authentication, validation, and app readiness.

Code example:

```
enum DeepLink {
    case order(Order.ID)
}

func handle(_ url: URL) {
    guard let link = DeepLinkParser().parse(url) else { return }
    switch link {
    case .order(let id):
        selectedTab = .orders
        path = [.order(id)]
    }
}
```

Possible follow-up questions: How do you deep link into a tab? What if required data is missing?

[Intermediate] When should you use `LazyVStack` and `LazyHStack`?

Interview answer: Use lazy stacks inside scroll views when you need custom scrollable layouts with many children and do not need all of `List`'s platform behavior. `LazyVStack` scrolls vertically; `LazyHStack` is useful for horizontal carousels.

Deep explanation: Lazy stacks defer creating views until they are needed, which helps with large collections. They are still SwiftUI containers, not cell reuse APIs identical to `UITableView`.

Why it matters: Choosing between `List`, lazy stacks, and grids affects performance, styling, accessibility, and built-in behaviors.

How it works: Children are produced lazily as they approach the visible region. Identity still matters for state preservation and diffing.

Use cases: Custom feeds, horizontal product shelves, chat-like layouts, carousels, and mixed-content screens.

Common mistakes: Using `VStack` for thousands of rows. Starting expensive work in every row `onAppear`. Assuming lazy means no memory/performance concerns.

Best practices: Keep row views cheap. Use stable IDs. Cache images. Prefer `List` when you need native row behavior.

Red flag answers: "Lazy stacks are always better than List." They provide flexibility, but not every platform feature.

Code example:

```
ScrollView(.horizontal) {
    LazyHStack(spacing: 12) {
        ForEach(products) { product in
            ProductCard(product: product)
        }
    }
    .padding(.horizontal)
}
```

Possible follow-up questions: How do lazy stacks affect `onAppear`? When would you choose `List` instead?

[Intermediate] How do you build sectioned lists in SwiftUI?

Interview answer: Use `Section` inside `List` or lazy containers when content is grouped by a meaningful category. A section can have a header, footer, and rows.

Deep explanation: Sectioning is both visual and semantic. In `List`, sections integrate with platform styling and accessibility. In custom scroll views, sections are just view composition unless you add behavior.

Why it matters: Settings, forms, grouped search results, transaction history, and inboxes are commonly sectioned in interviews and production apps.

How it works: SwiftUI renders each `Section` according to the container and style. In `List`, section headers may become sticky depending on platform/style.

Use cases: Settings groups, date-grouped messages, categorized search results, profile forms, and grouped notifications.

Common mistakes: Using visual headings without semantic sections. Giving section rows unstable IDs. Over-customizing `List` when a custom scroll layout is more appropriate.

Best practices: Model section data explicitly. Give sections and rows stable IDs. Test VoiceOver navigation through sections.

Red flag answers: "A section is just a Text heading." In `List`, it carries container semantics and styling.

Code example:

```
struct MessageSection: Identifiable {
    let id: Date
    let title: String
    let messages: [Message]
}

List(sections) { section in
    Section(section.title) {
        ForEach(section.messages) { message in
            MessageRow(message: message)
        }
    }
}
```

Possible follow-up questions: How do sticky headers work? How do you preserve row identity across sections?

[Intermediate] How does `.searchable` work in SwiftUI?

Interview answer: `.searchable` adds platform search UI bound to search text. The view should derive filtered results from that query or ask a model/service to perform debounced async search.

Deep explanation: Search is state. For local data, filtering can be derived from the query. For remote data, the query should trigger cancellable async work, usually with `.task(id:)` or model-level task management.

Why it matters: Search screens often reveal stale-result, cancellation, and performance bugs.

How it works: SwiftUI binds user input to a `String`. The placement and UI style adapt to platform/container.

Use cases: Lists, settings, contacts, orders, help center, product catalogs.

Common mistakes: Filtering destructively by replacing the original data. Firing a network request for every keystroke without debounce/cancellation. Showing stale results after query changes.

Best practices: Keep original data separate from filtered results. Debounce remote search where appropriate. Treat cancellation as neutral.

Red flag answers: "Searchable automatically filters the list." It provides UI and binding; filtering is your logic.

Code example:

```
@State private var query = ""

var filteredItems: [Item] {
    guard !query.isEmpty else { return items }
    return items.filter { $0.title.localizedCaseInsensitiveContains(query) }
}
```

```
List(filteredItems) { item in
    Text(item.title)
}
.searchable(text: $query)
```

Possible follow-up questions: How do you debounce remote search? How do you avoid stale async results?

[Intermediate] What does `withAnimation` do?

Interview answer: `withAnimation` wraps a state mutation in an animation transaction. SwiftUI then animates animatable changes caused by that state update.

Deep explanation: You are not animating a view object directly. You change state, and SwiftUI interpolates between the old and new rendered values when possible.

Why it matters: It explains why animations sometimes do not run: there may be no animatable value, no identity change for transitions, or the mutation happened outside an animation transaction.

How it works: `withAnimation` sets animation context for updates performed inside the closure. SwiftUI propagates that context through the affected view tree.

Use cases: Expand/collapse, route transitions, row insertion, banners, selected states, and matched geometry.

Common mistakes: Wrapping asynchronous work instead of the final state mutation. Expecting every property to animate. Applying animation too high in the tree.

Best practices: Animate the smallest meaningful state change. Use `.transaction` to disable or tune animation in a subtree.

Red flag answers: "`withAnimation` animates this block of code." It animates UI changes resulting from state mutations in the transaction.

Code example:

```
Button("Show details") {
    withAnimation(.spring(response: 0.35, dampingFraction: 0.85)) {
        isExpanded.toggle()
    }
}
```

Possible follow-up questions: Why would a transition not animate? How does Reduce Motion affect animation choices?

[Intermediate] What is `MagnificationGesture` or `MagnifyGesture`?

Interview answer: It is a SwiftUI gesture for pinch-to-zoom interactions. In modern SwiftUI, `MagnifyGesture` is the newer API; older code often uses `MagnificationGesture`.

Deep explanation: Pinch gestures produce a scale value. You usually keep transient gesture scale separate from committed scale so the view feels smooth during interaction and persists the final value on end.

Why it matters: It tests whether you understand gesture state, not just tap buttons.

How it works: The gesture reports magnification changes. SwiftUI updates transient gesture state during the gesture and lets you commit final state in `onEnded`.

Use cases: Photo viewer, map-like content, canvas, document preview, image cropper.

Common mistakes: Accumulating scale incorrectly so zoom jumps. Not clamping min/max scale. Making zoom-only interactions inaccessible.

Best practices: Use `@GestureState` for transient scale. Clamp committed scale. Provide accessibility alternatives.

Red flag answers: "Pinch is enough for zoom." You still need state management, limits, and accessibility.

Code example:

```
@State private var scale: CGFloat = 1
@GestureState private var gestureScale: CGFloat = 1

Image(uiImage: image)
    .resizable()
    .scaledToFit()
    .scaleEffect(scale * gestureScale)
    .gesture(
        MagnificationGesture()
            .updating($gestureScale) { value, state, _ in
                state = value
            }
            .onEnded { value in
                scale = min(max(scale * value, 1), 4)
            }
    )
```

Possible follow-up questions: How do you combine pinch and drag? How do you make zoom accessible?

[Intermediate] What is `LongPressGesture` used for?

Interview answer: `LongPressGesture` recognizes a press held for a minimum duration. It is useful for secondary actions, selection mode, previews, and gesture sequences.

Deep explanation: A long press often changes interaction mode rather than performing a primary action. It can be combined with drag for reorder or press-and-hold behavior.

Why it matters: Interviewers may use it to test gesture composition and accessibility thinking.

How it works: The gesture succeeds after the configured duration and movement tolerance. SwiftUI can call handlers during pressing and at completion.

Use cases: Context actions, reorder handles, press-to-record, preview reveal, multi-select mode.

Common mistakes: Hiding essential actions behind long press only. Conflicting with scroll gestures. Not giving visual feedback while pressing.

Best practices: Provide visible or accessible alternatives. Use gesture priority carefully inside scroll views. Give press feedback.

Red flag answers: "Long press is a replacement for menus." It can reveal menus, but the action should remain discoverable.

Code example:

```
Text(message.title)
    .onLongPressGesture(minimumDuration: 0.5) {
        selectedMessage = message
        isShowingActions = true
    }
```

Possible follow-up questions: How do you combine long press then drag? How do you expose the action to VoiceOver?

[Senior] How do you avoid unnecessary state changes in SwiftUI?

Interview answer: Avoid writing state when the value has not logically changed, keep state scoped to the smallest owner, derive values instead of storing duplicates, and avoid broad observable objects that invalidate large view trees.

Deep explanation: Every state write can invalidate views that depend on it. The goal is not to prevent updates entirely; it is to make updates meaningful and scoped.

Why it matters: Unnecessary state changes cause janky lists, repeated tasks, broken animations, and battery drain.

How it works: SwiftUI tracks dependencies and invalidates affected views. Coarse objects or top-level state can cause more recomputation than needed.

Use cases: Search fields, timers, scroll tracking, forms, async loading, live dashboards.

Common mistakes: Storing derived values as mutable state. Writing state inside layout callbacks every pass. Publishing from a huge view model for unrelated changes.

Best practices: Compare before assigning when updates are noisy. Split state by feature. Use derived computed values for cheap transformations.

Red flag answers: "Body recomputation is always the problem." Often the problem is expensive work or broad invalidation, not recomputation itself.

Code example:

```
func updateQuery(_ newValue: String) {
    guard query != newValue else { return }
    query = newValue
}
```

Possible follow-up questions: How can Observation reduce invalidation? When is `.equatable()` appropriate?

[Senior] How do you use Instruments to debug SwiftUI performance?

Interview answer: I start by reproducing the slow interaction on device, then inspect main-thread time, allocations, layout/rendering cost, image decoding, and state update frequency. I change one hypothesis at a time and re-profile.

Deep explanation: SwiftUI performance problems can come from many layers: model updates, view recomputation, layout, rendering, image work, or system controls. Instruments helps locate the expensive layer.

Why it matters: Senior SwiftUI work requires performance proof, not superstition.

How it works: Time Profiler shows CPU hot spots. Allocations shows churn. Hangs and animation/rendering tools show main-thread stalls. Signposts can mark app-specific flows.

Use cases: Slow feeds, animation hitches, launch stalls, repeated image decoding, heavy charts, complex forms.

Common mistakes: Profiling only on simulator. Blaming `body` prints. Changing architecture before measuring. Ignoring image sizes.

Best practices: Profile release-like builds on device. Add signposts around important flows. Precompute and cache expensive row data.

Red flag answers: "AnyView is slow, so remove it." It might help, but profiling should identify the bottleneck.

Code example:

```
let log = OSLog(subsystem: "com.example.app", category: "performance")
os_signpost(.begin, log: log, name: "FeedRender")
// Work being measured.
os_signpost(.end, log: log, name: "FeedRender")
```

Possible follow-up questions: What would you check first in a slow list? How do signposts help?

[Intermediate] How do you test SwiftUI screens with XCUITest?

Interview answer: Use XCUITest for critical user-visible flows: launch the app in a known state, interact through accessibility identifiers or labels, and assert visible outcomes.

Deep explanation: UI tests should cover integration behavior that unit tests cannot: navigation, form entry, accessibility wiring, system keyboards, and end-to-end flows. They are slower and more brittle, so keep them focused.

Why it matters: Interviewers expect judgment about the test pyramid, not just "test everything in UI tests."

How it works: XCUITest runs the app as a black box. Test code queries accessibility elements and performs taps, swipes, and text input.

Use cases: Login, checkout, onboarding, deep links, critical navigation, accessibility identifiers.

Common mistakes: Depending on arbitrary sleeps. Testing every small UI detail. Using labels that change with localization as the only selectors.

Best practices: Use deterministic launch arguments. Prefer waiting for expectations over sleeps. Add stable identifiers for non-obvious elements.

Red flag answers: "UI tests replace unit tests." They complement unit tests.

Code example:

```
func testLoginSuccessShowsInbox() {
    let app = XCUIApplication()
    app.launchArguments = ["UITestMode", "LoginSuccess"]
    app.launch()

    app.textFields["emailField"].tap()
    app.textFields["emailField"].typeText("user@example.com")
    app.secureTextFields["passwordField"].tap()
    app.secureTextFields["passwordField"].typeText("password")
    app.buttons["loginButton"].tap()

    XCTAssertTrue(app.staticTexts["Inbox"].waitForExistence(timeout: 3))
}
```

Possible follow-up questions: How do you make UI tests less flaky? What should be unit-tested instead?

[Intermediate] What is snapshot testing?

Interview answer: Snapshot testing compares a rendered UI or output against a recorded reference. It is useful for visual regression checks, but it should be used selectively because snapshots can be brittle.

Deep explanation: A snapshot test does not prove business correctness. It proves that output did not change unexpectedly. It is strongest for design-system components, stable states, and layout edge cases.

Why it matters: SwiftUI interviews often ask how to verify UI beyond view model tests.

How it works: The test renders a view in a controlled environment, captures an image or textual representation, and compares it to a stored baseline.

Use cases: Design system controls, empty/error states, Dynamic Type variants, dark mode, localized layouts.

Common mistakes: Snapshotting too many screens. Recording baselines without review. Ignoring device, OS, font, and scale differences.

Best practices: Snapshot stable components and key states. Use deterministic data. Review diffs carefully.

Red flag answers: "Snapshot tests guarantee the UI works." They catch visual changes, not interaction correctness.

Code example:

```
// Conceptual example; exact API depends on the snapshot library.
let view = ProfileHeader(name: "Ari", role: "iOS Engineer")
assertSnapshot(matching: view, as: .image(layout: .device(config: .iPhone13)))
```

Possible follow-up questions: What makes snapshots flaky? What should you not snapshot?

[Intermediate] How do you accessibility-test a SwiftUI screen?

Interview answer: Test with VoiceOver or Accessibility Inspector, verify labels/values/hints/actions, check Dynamic Type, Reduce Motion, color contrast, focus order, and ensure gesture-only actions have accessible alternatives.

Deep explanation: Accessibility testing combines automated checks, manual assistive-tech testing, and design review. SwiftUI provides many defaults, but custom controls require explicit semantics.

Why it matters: Accessibility is user experience, quality, and often compliance. It is a senior signal in interviews.

How it works: SwiftUI maps accessibility modifiers to the platform accessibility tree. Assistive technologies navigate that tree, not the visual hierarchy directly.

Use cases: Custom buttons, charts, gesture controls, forms, modals, dynamic content, lists.

Common mistakes: Using color alone for status. Missing labels on icon-only buttons. Bad VoiceOver order. Clipped Dynamic Type text. No Reduce Motion alternative.

Best practices: Use native controls where possible. Test large accessibility text sizes. Add accessibility actions for custom gestures.

Red flag answers: "The compiler will handle accessibility." It helps, but it cannot infer every product meaning.

Code example:

```
Image(systemName: "exclamationmark.triangle.fill")
    .foregroundStyle(.yellow)
    .accessibilityLabel("Warning")

Text(errorMessage)
    .accessibilityHint("Resolve this issue before continuing.")
```

Possible follow-up questions: How do you test VoiceOver order? How do you handle Reduce Motion?

[Intermediate] How do you use mocks in SwiftUI previews?

Interview answer: Inject preview-specific dependencies or state models so previews render deterministic loading, success, empty, error, long text, dark mode, and Dynamic Type states without hitting real services.

Deep explanation: Previews should be fast and reliable. If a preview depends on live network, real authentication, or production persistence, it will be flaky and slow.

Why it matters: Good previews improve UI quality and speed, especially in SwiftUI.

How it works: Views receive models or dependencies through initializers/environment. Preview code provides fake implementations and fixture data.

Use cases: Design review, edge-case layout, reusable components, accessibility states, localization.

Common mistakes: Using production API clients in previews. Only previewing the happy path. Building fixtures inline until previews are unreadable.

Best practices: Create small fixture factories. Keep preview dependencies local. Preview failure and empty states.

Red flag answers: "Previews are only for quick visual checks." They can be a strong development workflow when deterministic.

Code example:

```
#Preview("Empty") {
    InboxView(model: .preview(state: .empty))
}

#Preview("Error") {
    InboxView(model: .preview(state: .failed("Unable to load messages.")))
}
```

Possible follow-up questions: How do you inject environment dependencies into previews? How do previews complement tests?

[Intermediate] How do you design REST API integration on iOS?

Interview answer: Model endpoints as typed requests, validate HTTP status, decode DTOs with `Codable`, map DTOs to domain models, handle error bodies, and keep URLSession details out of views/view models.

Deep explanation: REST integration is more than `URLSession.data(from:)`. Real APIs need authentication, pagination, retries, caching, idempotency, versioning, rate-limit handling, and error mapping.

Why it matters: Networking is one of the most common interview areas because it connects architecture, async/await, error handling, testing, and product behavior.

How it works: The app builds an HTTP request, sends it through a client, validates transport and server response, decodes success/error payloads, and returns domain-friendly results.

Use cases: Login, feeds, order details, search, profile updates, file uploads, pagination.

Common mistakes: Ignoring non-2xx status codes. Letting API DTOs leak everywhere. Force-unwrapping URLs. Not testing error responses.

Best practices: Keep DTOs separate from domain when shapes differ. Centralize auth headers and response validation. Use async/await and typed errors.

Red flag answers: "If decoding works, the request succeeded." HTTP status and semantic errors still matter.

Code example:

```
struct Endpoint<Response: Decodable> {
    let path: String
    let method: String
}

func send<Response>(_ endpoint: Endpoint<Response>) async throws -> Response {
    var request = URLRequest(url: baseURL.appending(path: endpoint.path))
    request.httpMethod = endpoint.method
    let (data, response) = try await session.data(for: request)
    guard let http = response as? HTTPURLResponse,
          (200..<300).contains(http.statusCode) else {
        throw APIError.invalidResponse
    }
    return try decoder.decode(Response.self, from: data)
}
```

Possible follow-up questions: How do you handle pagination? How do you refresh auth tokens safely?

[Senior] How do you separate business logic from UI?

Interview answer: Views should render state and send user actions. View models should manage presentation state. Business rules belong in use cases/domain services, and data access belongs in repositories or API/storage clients.

Deep explanation: Business logic is the part that should remain true if the UI changes from SwiftUI to UIKit or from mobile to another client. If logic is embedded in views, it becomes hard to test and reuse.

Why it matters: This is a core architecture signal. It shows you can scale a codebase beyond one screen.

How it works: Dependencies point inward: UI depends on presentation/domain; domain does not depend on UI frameworks. The UI layer adapts domain results into user-visible state.

Use cases: Checkout validation, eligibility rules, pricing, cancellation reasons, profile completion, offline sync.

Common mistakes: Putting validation in button actions. Making views call API clients directly. Using view models as a dumping ground for all logic.

Best practices: Put durable rules in use cases. Inject dependencies. Unit test business rules without rendering UI.

Red flag answers: "SwiftUI views are simple, so logic in body is fine." Simple conditionals are fine; business workflows are not.

Code example:

```
struct CheckoutEligibilityUseCase {
    func canCheckout(cart: Cart, user: User) -> Bool {
        !cart.items.isEmpty && user.hasVerifiedPaymentMethod
    }
}
```

Possible follow-up questions: Where does form validation live? How do you avoid Massive ViewModels?

[Senior] How would you structure a large SwiftUI app?

Interview answer: I would organize by feature, keep UI/presentation/domain/data responsibilities separate, inject dependencies at composition boundaries, use explicit navigation routes, and keep shared app state small and intentional.

Deep explanation: Large SwiftUI apps need clear ownership more than clever abstraction. The structure should help teams find code, test behavior, and change features without cross-module side effects.

Why it matters: Interviewers ask this to evaluate architecture judgment and team-scale thinking.

How it works: Each feature owns its screens and feature model. Domain/use cases define behavior. Repositories abstract data. App composition wires concrete implementations.

Use cases: Banking, delivery, commerce, ride-hailing, productivity, enterprise apps.

Common mistakes: One global environment object for everything. Circular dependencies between features. View models importing data-layer details. Over-modularizing too early.

Best practices: Use feature folders/modules. Keep domain models UI-free. Define routes as data. Add architecture only where complexity justifies it.

Red flag answers: "I would use MVVM and EnvironmentObject." That is not enough for large-app structure.

Code example:

```
Feature/
-- Presentation/
-- Domain/
-- Data/
-- Tests/
```

Possible follow-up questions: How do features communicate? Where does navigation live? How do you avoid global state?

[Intermediate] How should data flow between UIKit and SwiftUI?

Interview answer: Use explicit bindings, callbacks, observable models, or coordinators at the boundary. Decide which side owns the source of truth and keep synchronization one-directional where possible.

Deep explanation: Interop bugs happen when UIKit and SwiftUI both think they own the same state. A hosting controller can pass SwiftUI state/actions down, while representables use coordinators to send UIKit delegate events back up.

Why it matters: Most production apps are hybrid, so data flow boundaries matter.

How it works: UIKit can host SwiftUI with `UIHostingController`. SwiftUI can host UIKit with representables. Bindings/callbacks/coordinators bridge changes.

Use cases: Embedding SwiftUI screens in UIKit navigation, wrapping `UITextView`, sharing selection state, presenting SwiftUI modals from UIKit.

Common mistakes: Updating UIKit and SwiftUI state independently. Reloading wrapped UIKit views unnecessarily. Creating retain cycles through coordinators.

Best practices: Make ownership explicit. Keep representable `updateUIView` idempotent. Use coordinators for delegate callbacks.

Red flag answers: "Just share a singleton." That hides ownership and makes tests harder.

Code example:

```
struct LegacyTextView: UIViewRepresentable {
    @Binding var text: String

    func updateUIView(_ uiView: UITextView, context: Context) {
        if uiView.text != text {
            uiView.text = text
        }
    }
```

```
}
```

Possible follow-up questions: How do you prevent update loops? Who owns state in a hybrid screen?

[Intermediate] How do Reduce Motion and color contrast affect SwiftUI design?

Interview answer: Respect Reduce Motion by avoiding essential motion-heavy feedback and offering simpler transitions. Respect contrast by ensuring text and controls remain readable in light/dark mode, disabled states, and custom themes.

Deep explanation: Accessibility is not only VoiceOver. Motion can cause discomfort, and low contrast can make text unusable. SwiftUI exposes environment values and accessibility APIs to adapt.

Why it matters: Strong iOS candidates treat accessibility as product quality, not an afterthought.

How it works: SwiftUI can read accessibility-related environment values and render alternative UI. System colors and materials adapt better than hard-coded colors.

Use cases: Animated transitions, skeleton loading, charts, status badges, custom buttons, onboarding.

Common mistakes: Using color alone for errors/success. Hard-coding low-contrast colors. Forcing large motion for navigation or state changes.

Best practices: Use semantic colors. Pair color with text/icons. Check contrast in both appearances. Disable or simplify animations when Reduce Motion is enabled.

Red flag answers: "Accessibility is handled by system controls." Custom UI still needs review.

Code example:

```
@Environment(\.accessibilityReduceMotion) private var reduceMotion

withAnimation(reduceMotion ? nil : .spring()) {
    isExpanded.toggle()
}
```

Possible follow-up questions: How do you test contrast? How do you avoid color-only communication?

Most Common iOS Interview Questions

[Beginner] Explain struct vs class in one minute.

Interview answer: Structs are value types and classes are reference types. I default to structs for data because value semantics reduce accidental shared mutation. I use classes when I need identity, shared mutable lifetime, inheritance, Objective-C/UIKit interop, or `deinit`.

Deep explanation: Mention both capabilities and costs. Structs and classes share many features, but classes add inheritance, type casting, deinit, identity, and ARC.

Why it matters: This question tests whether you understand Swift's design philosophy.

How it works: Struct assignments behave as independent values. Class assignments copy references to one instance.

Use cases: Struct: `User`, `Order`, `ViewState`. Class: `UIViewController`, cache, coordinator, shared service.

Common mistakes: Saying only "struct stack, class heap." Missing identity and ARC.

Best practices: Tie answer to ownership and mutation.

Red flag answers: "Structs cannot have methods." They can.

Code example:

```
let copy = originalStruct
let alias = originalClass
```

Possible follow-up questions: How does copy-on-write affect arrays? What does `let` mean for a class?

[Intermediate] Explain ARC and retain cycles in one minute.

Interview answer: ARC automatically retains and releases class instances based on strong references. It deallocates when the strong count reaches zero. Retain cycles happen when objects or closures strongly keep each other alive, so we use weak/unowned references where ownership should not be strong.

Deep explanation: Closures are a common source: object owns closure, closure captures object. Delegates should usually be weak.

Why it matters: Tests memory and closure knowledge.

How it works: Strong references increment count; weak references do not and zero out.

Use cases: Delegates, callbacks, timers, Combine sinks.

Common mistakes: Weak everywhere. Strong delegates.

Best practices: Design ownership deliberately and verify deinit for complex flows.

Red flag answers: "Swift has garbage collection."

Code example:

```
callback = { [weak self] in
    self?.reload()
}
```

Possible follow-up questions: When not to use weak self? When use unowned?

[Intermediate] Explain async/await and MainActor in one minute.

Interview answer: Async/await marks asynchronous suspension points without nested callbacks. `@MainActor` isolates UI state updates to the main actor so Swift can enforce safe UI mutation.

Deep explanation: `await` suspends a task, not necessarily a thread. MainActor is a global actor, not just a dispatch call.

Why it matters: Tests modern concurrency fluency.

How it works: Tasks suspend/resume at awaits. Actor hops require await.

Use cases: Networking, view models, UI updates.

Common mistakes: Blocking async work with semaphores. Updating published state off main actor.

Best practices: Mark UI-facing models `@MainActor`.

Red flag answers: "await blocks the main thread."

Code example:

```
@MainActor
func load() async {
    title = (try? await service.title()) ?? "Unavailable"
}
```

Possible follow-up questions: What is cancellation? What is actor reentrancy?

[Intermediate] Explain SwiftUI data flow in one minute.

Interview answer: SwiftUI views are value descriptions of UI. State is the source of truth. Views read state and send actions. When state changes, SwiftUI recomputes affected view bodies and reconciles the UI using identity and dependencies.

Deep explanation: Use `@State` for local ownership, `@Binding` for child editing, `@Observable` or `ObservableObject` for reference models, and environment for cross-cutting context.

Why it matters: Tests whether you understand declarative UI.

How it works: State invalidates view descriptions; SwiftUI updates render graph.

Use cases: Forms, lists, navigation, async loading.

Common mistakes: Duplicating source of truth. Side effects in body.

Best practices: Keep ownership clear and body cheap.

Red flag answers: "Views update themselves like UIKit views."

Code example:

```
@State private var isOn = false
Toggle("Enabled", isOn: $isOn)
```

Possible follow-up questions: Where is @State stored? What is view identity?

[Senior] Explain how you would structure a medium-to-large SwiftUI app.

Interview answer: I would structure by feature, keep views declarative, use `@Observable`/view models for presentation state, use use cases for business logic, repositories for data access, constructor injection for dependencies, and route/navigation state separate from UI rendering.

Deep explanation: The goal is controlled dependencies and testable state transitions without unnecessary ceremony.

Why it matters: Tests senior architecture judgment.

How it works: Feature modules own views and feature models. Domain protocols define use cases/repositories. Data layer implements APIs/persistence.

Use cases: Authenticated apps, marketplaces, banking, delivery, productivity apps.

Common mistakes: Global environment object for everything. Business logic in views.

Best practices: Make dependencies explicit. Keep domain independent of UI.

Red flag answers: "Use MVVM everywhere" without explaining ownership or dependency boundaries.

Code example:

```
Feature/
+-- Views
+-- Model
+-- UseCases
+-- Data
```

Possible follow-up questions: How do you handle navigation? How do you test features?

Senior SwiftUI Interview Questions

[Senior] How would you explain SwiftUI identity to a team debugging list bugs?

Interview answer: I would explain that SwiftUI preserves state and animations by matching old and new views using identity. In collections, model IDs must represent stable logical identity. If IDs change, SwiftUI thinks the row is a new row.

Deep explanation: Many list bugs are not rendering bugs; they are identity bugs.

Why it matters: Senior engineers teach mental models and prevent repeated classes of bugs.

How it works: Reconciliation uses identity/type/position. `ForEach` IDs and `.id()` influence this.

Use cases: Chat rows, editable lists, reorderable lists, navigation detail state.

Common mistakes: `id: \.self` for mutable models. `UUID()` in body. Index IDs in mutable arrays.

Best practices: Use stable domain IDs and avoid accidental identity resets.

Red flag answers: "SwiftUI list bugs are random."

Code example:

```
struct RowModel: Identifiable {
    let id: Message.ID
    let title: String
}
```

Possible follow-up questions: How do transitions depend on identity? How does `.id()` affect tasks?

[Senior] How would you design a SwiftUI screen with loading, error, empty, and content states?

Interview answer: I would model the screen state explicitly with an enum, render each state declaratively, and keep async loading in a main-actor model. This avoids invalid combinations like loading and error at the same time.

Deep explanation: Multiple booleans scale poorly. Enums encode mutually exclusive states directly.

Why it matters: State modeling is a senior-level SwiftUI skill.

How it works: View reads one state enum. View model transitions state through loading, loaded, empty, failed.

Use cases: Any network-backed screen.

Common mistakes: `isLoading`, `error`, `items`, `isEmpty` all independent.

Best practices: Use explicit state machines and test transitions.

Red flag answers: "Just show a spinner until data arrives."

Code example:

```
enum Loadable<Value> {
    case idle
    case loading
    case loaded(Value)
    case empty
    case failed(String)
}
```

Possible follow-up questions: How do you handle refresh while content is visible? How do you preserve stale data on error?

[Senior] How do you prevent SwiftUI view models from becoming massive?

Interview answer: Keep view models focused on presentation state and user actions. Move business rules into use cases, data access into repositories, formatting into helpers where needed, and navigation into route/coordinator state for complex flows.

Deep explanation: MVVM can become Massive ViewModel if every responsibility is dumped there.

Why it matters: Large view models are hard to test and change.

How it works: Dependencies perform specialized work. View model orchestrates and exposes UI state.

Use cases: Checkout, onboarding, search, order tracking, profile editing.

Common mistakes: View model directly parses JSON, writes database, navigates, validates business rules, and tracks analytics.

Best practices: Split by responsibility, not by arbitrary file count.

Red flag answers: "Just make more ViewModels." Splitting without clear boundaries does not fix responsibility problems.

Code example:

```
struct SubmitOrderUseCase {
    let repository: OrderRepository
    let validator: OrderValidating

    func execute(_ draft: OrderDraft) async throws -> Order {
        try validator.validate(draft)
        return try await repository.submit(draft)
    }
}
```

Possible follow-up questions: Where should validation live? How do you test analytics?

[Senior] How would you migrate from ObservableObject to Observation?

Interview answer: I would migrate feature-by-feature where the deployment target allows iOS 17+, replace `ObservableObject`/`@Published` with `@Observable`, update ownership from `@StateObject` to `@State` where appropriate, and verify invalidation behavior with tests and UI checks.

Deep explanation: Migration is not a search-and-replace. Ownership wrappers and data flow change.

Why it matters: Teams need safe incremental migration strategies.

How it works: Observation tracks property reads and mutations through macro-generated code.

Use cases: Modernizing SwiftUI screens, reducing Combine dependency, improving invalidation precision.

Common mistakes: Mixing wrappers blindly. Breaking iOS 16 support. Not testing computed property updates.

Best practices: Define OS support first. Keep migration PRs scoped. Compare before/after UI behavior.

Red flag answers: "Just replace `ObservableObject` with `@Observable` everywhere."

Code example:

```
@Observable
final class SettingsModel {
    var isBiometricsEnabled = false
}

struct SettingsView: View {
    @State private var model = SettingsModel()
}
```

Possible follow-up questions: How do you support older OS versions? What about Combine pipelines?

Common Bad Answers and Better Answers

Topic	Bad answer	Better answer
Struct vs class	Structs are stack, classes are heap.	Structs have value semantics; classes have reference identity and ARC. Storage is an optimization detail.
ARC	Swift has garbage collection.	Swift uses deterministic ARC for reference types; cycles still need weak/unowned or lifecycle fixes.
weak self	Always use weak self.	Use weak when the closure may outlive self or create a cycle; avoid it when the operation must retain its owner intentionally.
async/await	Await blocks the thread.	Await marks a suspension point; the task can suspend and free the thread.
MainActor	Same as DispatchQueue.main.async.	MainActor is compiler-enforced isolation to the main actor; dispatch is a lower-level scheduling call.
Actors	Actors make all concurrency safe.	Actors protect isolated mutable state, but logical races and reentrancy still need design.
SwiftUI View	A View is the actual UI object.	A View is a value description SwiftUI uses to update an internal render tree.
body	Body should run once.	Body can run often; it must be cheap, deterministic, and side-effect-free.
@State	State is just a property.	@State is SwiftUI-managed persistent storage keyed by view identity.
AnyView	Use AnyView to fix type errors.	Prefer ViewBuilder/generics; erase only when storage/API heterogeneity requires it.
GeometryReader	Use it whenever layout is hard.	Use it for measurement/coordinates; prefer stacks, Layout, ViewThatFits, and safe area APIs for normal layout.
Testing	SwiftUI cannot be tested.	Test view models/use cases, use previews for visual states, and XCUITest for critical flows.

Final Mock Interview

Question 1: You are building a profile screen. Would you make the model a struct or a class?

Strong sample answer: I would make the domain `UserProfile` a struct because it represents data and benefits from value semantics. If the screen needs observable mutable UI state, I would use a separate `@Observable` view model or feature model as a class, marked `@MainActor` if it owns UI state. That separates value data from reference-owned presentation state.

Follow-up answer: If the profile contains large arrays, Swift's copy-on-write collections keep ordinary copies cheap until mutation. I would still avoid embedding mutable class references inside the struct unless that reference behavior is intentional.

Question 2: Why did this SwiftUI list lose text field focus after data changed?

Strong sample answer: I would first check row identity. If the list uses indices or creates new UUIDs during rendering, SwiftUI may treat rows as new views and recreate their state, which loses focus. The fix is to use stable domain IDs and avoid unnecessary `.id()` changes. I would also check whether the parent state invalidates too broad a subtree.

Follow-up answer: If focus is part of the screen behavior, I would model it with `@FocusState` using stable row IDs, not row indices.

Question 3: A view model uses `[weak self]` inside a save task, and sometimes saving silently does not happen. What do you think?

Strong sample answer: Weak capture might be wrong if the save operation should complete even after the view model is released. I would clarify the intended lifecycle. If save is user-critical, move the operation to a service that owns the work or make the task explicitly retained until completion. If the work is view-scoped, cancellation should be explicit and handled as a neutral outcome.

Follow-up answer: Weak self prevents cycles, but it also changes behavior by allowing the closure to do nothing. It is not a universal fix.

Question 4: How would you design async loading for a SwiftUI detail screen?

Strong sample answer: I would use a main-actor observable model with an explicit load state enum. The view starts loading with `.task(id: itemID)`, so if the ID changes the previous task is cancelled. The model guards duplicate initial loads and treats cancellation separately from failure. The repository handles caching/deduplication if multiple screens request the same item.

Follow-up answer: I would avoid starting the request in `body` or in unguarded `onAppear`, because view lifecycle can trigger multiple times.

Question 5: What is actor reentrancy and why can it matter?

Strong sample answer: An actor protects its isolated state so only one task accesses it at a time, but actor methods can suspend at `await`. While suspended, other actor-isolated work may run and mutate state before the original method resumes. That means actors prevent data races, but not every logical race. I avoid holding invariants across await points or revalidate state after awaiting.

Follow-up answer: For example, if an actor checks that a token is valid, awaits refresh, then assumes state is unchanged, another task may have updated the token in between.

Question 6: How would you explain `some View` to someone coming from UIKit?

Strong sample answer: `some View` means the property returns one concrete view type, but the exact type is hidden. SwiftUI view types become extremely complex after modifiers and builders, so opaque return types keep APIs readable while preserving static type information. It is not the same as returning any view at runtime; it is a hidden but fixed concrete type.

Follow-up answer: When different branches need to return different views, `@ViewBuilder` can combine them into a concrete conditional view type. `AnyView` is type erasure and should be reserved for true heterogeneity.

Question 7: How do you test a SwiftUI feature that calls a network API?

Strong sample answer: I would inject a repository protocol into the view model or use case, pass a stub/fake in tests, and assert state transitions for success, loading, empty, and failure. I would keep real networking out of unit tests. For UI, I would add previews for visual states and XCUITests for critical flows.

Follow-up answer: If the view model is `@MainActor`, the test should run on the main actor. For async code, I would await the action directly instead of sleeping.

Question 8: When would you choose UIKit over SwiftUI?

Strong sample answer: I would choose UIKit when the feature requires mature imperative control, very complex collection layouts, existing UIKit infrastructure, or deployment/behavior constraints that make SwiftUI risky. I would choose SwiftUI for new state-driven screens, forms, settings, and components where previews and declarative data flow improve speed and maintainability. In many apps, a hybrid boundary is the pragmatic answer.

Follow-up answer: I would avoid wrapping a complex UIKit mental model inside SwiftUI. If SwiftUI is chosen, use SwiftUI data flow; if UIKit is chosen, keep UIKit ownership and lifecycle clear.

Reference Links Used

- Swift Book: Structures and Classes
- Swift Book: Concurrency
- Swift Evolution SE-0395: Observation
- Swift Evolution SE-0302: Sendable and @Sendable closures
- Swift Evolution SE-0306: Actors
- Apple Developer Documentation: NavigationStack
- Apple Developer Documentation: NavigationPath
- Apple Developer Documentation: Layout
- Apple Developer Documentation: ViewThatFits
- Apple Developer Documentation: MagnifyGesture
- Apple Developer Documentation: LongPressGesture
- Apple Developer Documentation Search: SwiftUI searchable
- Apple Developer Documentation: XCUITest

Generated with AI coding assistant